GATOH Documentation

Release 2026.09.0

Manuel Munizaga

Nov 06, 2026

CONTENTS

Version: 2026.09.0

Useful links: Source Repository | Issue Tracker |

GATOH is a Python package that provides a framework for creating complex, multilayer agent-based models.

It is licensed under the MIT license, and the source code is hosted on GitHub at:

https://github.com/JungleHollow/GATOH

Contents:

CHAPTER

ONE

GATOH USER GUIDE

This guide is an overview, and explains the important features of the GATOH Agent-Based Modelling package.

1.1 What is GATOH?

1.2 Installing GATOH

1.3 GATOH Quickstart

The Generalised Agent Transformation of Opinions in Hierarchies, or GATOH, framework is an agent-based modeling library for Python. It is designed to provide a highly flexible and generalisable framework that can define and simulate a wide variety of agent-based models centered on opinion dynamics.

1.3.1 Installing GATOH

GATOH has been primarily developed and tested on a x86_64 Linux system, but support for 64 bit Windows has also been confirmed. The different methods for installing GATOH are outlined below:

From source

1. Ensure that Python >=3.14 is installed on your system.
2. Clone the library's code from the git repository: `git clone https://github.com/JungleHollow/GATOH`.
3. If needed, create a virtual environment to install the library to.
4. From the target python environment, simply install GATOH using pip as normal, but specify the path to the GATOH directory rather than the package name: `python -m pip install <path to GATOH directory>`.
5. At this point, all required dependencies will be installed alongside GATOH, and you can begin using the library.

PyPi

1. Ensure that Python >= 3.14 is installed on your system.
2. ...

Work in progress

1.3.2 Using GATOH

Once you have installed GATOH, you can use it by importing gatoh in your python scripts. Almost all functions and classes in GATOH are currently organised into their own submodules, meaning that these will need to be imported and used accordingly. Below is a simple example of what very basic GATOH usage could look like:

```
from gatoh.model import ABModel
from gatoh.utils import plot_graph

HIERARCHIES = ["Religious", "Neighbours", "Family", "Friends"]
HIERARCHY_RW = {
    "Religious": (0.0, 0.3),
    "Neighbours": (0.0, 0.2),
    "Family": (0.0, 0.01),
    "Friends": (0.0, 0.1),
}

model = ABModel(
    HIERARCHIES,
    HIERARCHY_RW,
    iterations=50,
    model_id="TEST_MODEL",
)
model.generate_agents(
    "TEST",
    {
        "neutral": 0.4,
        "social": 0.4,
        "impulsive": 0.2,
    },
    number=20,
)
model.generate_graphs(
    HIERARCHIES,
    model.agents,
    method="blockmodel",
    agent_subsetting=True,
)

model.iterate()

x_vals = {"iterations": [i for i in range(50)]}
y_vals = {"aggregate_opinions": model.logger.variables.aggregate_opinions}

plot_graph(
    x_vals,
    y_vals,
    ...
)
```

You can refer to the *Basic tutorial* for more information on how to use GATOH, or the *Absolute beginner's guide* for an in-depth explanation of agent-based modeling and the GATOH framework.

1.4 GATOH: The absolute basics for beginners

1.4.1 Prerequisites

A basic knowledge of Python is assumed. For further reference, see the Python tutorial.

To work the examples, you'll need the GATOH (Generalised Agent Transformation of Opinions in Hierarchies) framework and its included dependencies installed.

Learner Profile

This is a quick overview of agent-based modelling, and its implementation in GATOH. It demonstrates how agents are defined, and how they interact with each other and their environment to produce emergent behaviours. If you are looking for a basic introduction to ABM (Agent-Based Modelling) principles in the GATOH framework, then this article may be of use to you.

Learning Objectives

After reading, you should be able to:

- Understand what an "agent" is, and how they are defined;
- Understand the basic process of ABM from start to finish;
- Understand the different use-cases for ABM;

1.4.2 The Basics

To start with, the fundamental modelling method that GATOH is based around is that of ABM. The implementation of ABMs (Agent-Based Models) can be approached in a variety of ways depending on the nature of the scenario that they are simulating.

The scenario must be properly defined – and there must be a clear idea of the type of behaviours that emerge from the scenario, as well as who or what produces those behaviours. It is also important to consider the environment in the scenario, and whether or not it has a significant impact on the emergent behaviours.

GATOH is primarily designed with sociological or anthropological simulations in mind. The simplest general scenario that is intended to be modelled with GATOH is that of social contagion in some community or social network; with the emergent behaviour in this scenario being the spread of some idea or belief between the people who live in the community.

But, social interactions between community members are not simple, and are heavily influenced by a number of factors – primarily the different social hierarchies that exist in the community, that agents may or may not belong to.

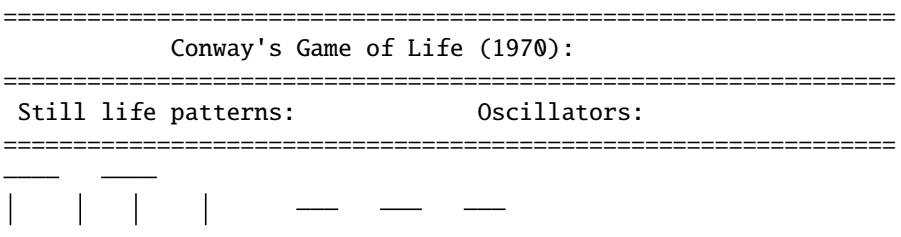
In the next sections, we will explore the different components that make up a GATOH model, and the ways that they interact with each other.

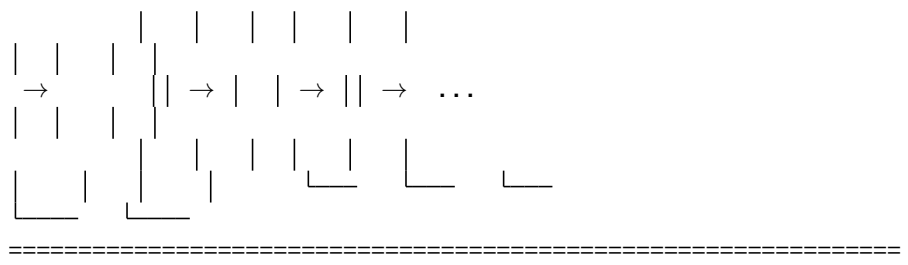
Agent-Based Modelling

ABM is a computational modelling strategy that has grown in popularity in recent years as computer hardware has become more powerful.

The first ABMs were simple, cell-based systems, where each cell represented a single agent. The most famous example of these types of ABMs is Conway's Game of Life[1] . In this rudimentary ABM, simple rules are defined which determine how and when each cell 'comes to life' or 'dies' based on the cell's current state, and the state of its immediate neighbours.

From these simple rules, relatively complex behaviours can emerge. In the depiction below, we can see some examples of the behaviours that can emerge from such a simple system:





Whilst it was revolutionary at its time of creation, the Game of Life was developed in a context where computers were still in their infancy, and they were still basically just a step above calculators. To give an example, the Cray-1 – a 5.5 ton supercomputer released in 1976 – had a processor capable of running 80 million operations per second, and had 8.4 megabytes of memory available.

Now, in 2026, the newest consumer CPUs (Computer Processing Units) are capable of performing around 5 billion operations per second, and they have more memory available in just their L3 cache than 20 Cray-1 supercomputers would have had collectively in the 1970s(!). Considering this, it is easy to see how ABMs would have started evolving past these simple 2-dimensional, binary cells in the following decades.

Now it is possible to model extremely complex systems such as the spread of global pandemics, or the nature of peak-hour traffic in a large city. Although ABM can be applied to many use-cases, there is still some care that must be taken when creating such models.

Each area of interest has very specific characterisations, rules, and interactions, which must all be appropriately quantified and defined for the model to be representative of real life. Using the examples of global pandemics and car traffic, we can see that models for these scenarios would have inherently different approaches.

A model that simulates a global pandemic might treat interactions between agents as the driving force that spreads the 'emergent behaviour' (as interactions between agents would enable viral transmissions), whereas the 'interaction' between agents in a traffic model would in itself be the emergent behaviour (as the 'interaction' of a large number of agents would imply that traffic has built up).

For the purposes of this guide, let's look at another famous ABM from the same era as Conway's game of life – Schelling's model of segregation [2]. In the following sections, we will break down ABMs into their core components, and see how each fits into the overall modeling process; using Schelling's model as an example.

Defining the problem

The first, and arguably most important, step in the agent-based modeling process is the concrete definition of the problem and its computational representation.

If the problem is not defined correctly from the start, then this just leads to a lot of headache and time lost further along the line, as any changes that have to be made later – no matter how minor or isolated they may appear – will likely be connected to a number of other components that will have to be redefined and tweaked as well.

This is particularly important when we are talking about highly complicated models that take multiple hours to run *one* iteration; it's not a great experience when you realise that a model parameter is wrong and has to be changed when your model is halfway done with its iterations and has already been running for the past week…

All of this to say that it is very important and worthwhile to spend some extra time at the start to think through, and properly define the representation of the problem for your model.

In the case of Schelling's model of segregation, the foundation of the model is based on the assumption that no outside influences or explicit prejudices exist (for the purposes of the model); with the only factor affecting possible segregation being the presence of "mild in-group preferences" in people. Following this assumption, the model is defined as follows:

- A community is represented by a $N \times N$ grid of cells.
- Each cell in the community represents a space that can be inhabitted by an agent, or be empty.
- Every agent in the community is assigned to one of two "groups".

- At every iteration, each agent:
    1. Observes its immediate neighbours (ignoring empty spaces).
    2. If the proportion of neighbours belonging to the same "group" is equal or greater to some threshold, the agent is content and remains stationary.
    3. Otherwise, the agent is dissatisfied, and will move from its current space to another empty space in the community.

In this way, a seemingly complex problem (such as racial segregation in the case of Schelling) can be defined in a more deterministic manner that can be more easily managed computationally.

In some cases, this may mean a simplification (i.e. reducing the level of detail of, or completely removing a concept from the model), whilst in other cases this may mean having to think outside the box a bit to be able to quantify some property or behaviour that by nature isn't already quantified (e.g. treating the strength of a person's beliefs as a value on a scale from 0-10).

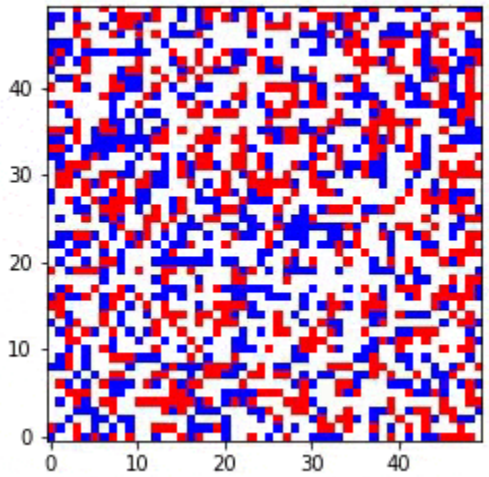


Fig. 1: **Fig.** Schelling's model of segregation in action[?]

*Above, I have included a short GIF (if viewing in HTML) depicting the runtime of Schelling's model of segregation so that you can understand better how everything will fit together. As you can see, the agents start in a relatively random distribution within the community, but as more iterations happen, the community slowly becomes more and more segregated into distinct "tribes".

Agents

Definition: An agent is an entity that is capable of acting without external input, and can interact with other agents and the environment around it.

One of the core objects of the GATOH framework is the "Agent". In the GATOH framework, an Agent represents an "average" person that lives in some community, and interacts with their neighbours in different social contexts.

Agents can be defined in a number of different ways depending on the exact nature of the problem, but there is rarely an 'optimal' definition; each alternative has its own pros and cons.

The core definition of an agent will obviously vary depending on the problem definition, as different types of problems will be interested in tracking and simulating different types of characteristics or traits in their agents. For example, if the model is being constructed to simulate car traffic, each agent might represent an individual car on the streets.

[0] By Blaqdolphin - Own work, CC BY-SA 4.0, Link

But even if the problem definition remains fixed, there are multiple ways of defining agents. If a model is instead simulating the spread of avian flu in birds, each agent could represent a single bird, or a flock of birds that are migrating in an area. In this way, the exact definition of the agents in the model will have an impact on what exactly happens when different "agents" interact with each other; as the interaction can be representative of individual or group effects.

Another thing to keep in mind when defining an agent is how, or if, the agent will interact with the space around them (if there even is a "space" defined for your particular model).

Depending on the mechanism that you have chosen to use for agent-space interaction, it may be necessary for agents to hold additional attributes so that they can differentiate the information that they provide for agent-agent interactions versus agent-space interactions, as these will likely not involve the same behaviours:

```
================================================================
                Example Held Attributes
================================================================
          Agent                       Agent (with space)
================================================================
    - Name                      - Name
    - Age                       - Age
                                - X coordinate
                                - Y coordinate
                                - Travel distance
                                - Travel probability
                                - Travel frequency
================================================================
```

In the case of Schelling's model of segregation, an agent represents a person that lives within some arbitrary community, and they may belong to one of two different groups. The agents in Schelling's model do not "interact" with each other in the traditional sense, but rather they observe the traits of their neighbours.

These agents do, however, interact with the space around them; moving from their current cells to empty ones if they are not satisfied with the group makeup of their neighbours.

Although, in this particular case no additional information is needed to differentiate between agent-agent and agent-space interactions, as each cell contains all the needed information (either the agent's group if the cell is occupied, or nothing if the cell is empty).

Model Space

The model "space" was briefly touched on in the previous section, but let's look at it in some more detail here.

Just how we live in and interact with the space around us (our homes, neighbourhoods, cities, etc.) an agent in an ABM may interact with some form of "space" around them if the model requires it.

For some models, space is a core factor for the emergent behaviour – it would be impossible to simulate the presence of car traffic if agents had no space to create traffic in, for instance.

In other models, space is simply a way of capturing additional information; the inclusion of "space" and "distance" may allow for a stronger, more accurate model, but the underlying behaviours that are being simulated do not directly depend on the space itself.

And of course, it may be possible for some models to not require a "space" at all.

Similarly to how there are many ways of defining an agent, each with their own pros and cons, there are many ways of defining model space.

The model space can be an explicit "map" with coordinates that the agents inhabit:

```
======================================
          Map-based model space:
======================================
```

```
 ____________________
|(0,y)(1,y)(...)(x,y)|
|(...)(...)(...)(...)|
|(0,1)(1,1)(...)(x,1)|
|(0,0)(1,0)(...)(x,0)|
 ====================
```

Or it could follow a graph-based approach instead:

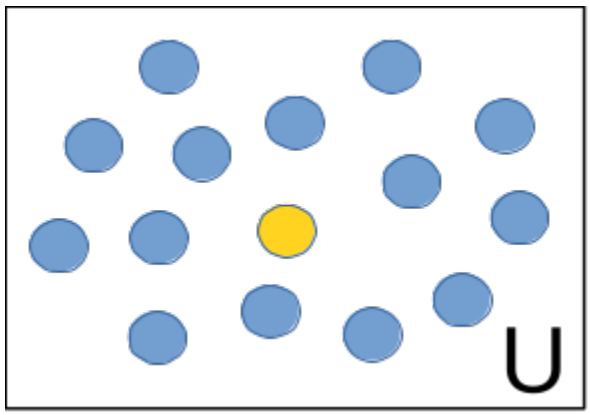


Fig. 2: **Fig.** A depiction of a graph-based model space

It is also possible for model space to be more of an abstract concept; with agents posessing attributes that note the different cities that they live in, for example.

For Schelling's model, a $N \times N$ cell grid is used, where $N \times N$ must be greater than the total number of agents to allow for agents to move into empty cells. Schelling's model is somewhat different to the "traditional" ABM when it comes to the model space, with the cells doubling as both the spaces and the agents themselves.

Model Rules

The rules of a model provide the necessary conditions for emergent behaviours to occur. To simplify things a bit, let's separate the rules into 3 different categories and look at each one individually:

1. Restrictive Rules
2. Interaction Rules
3. Behavioural Rules

Restrictive Rules

Restrictive rules can be thought of – as the name suggests – as any model rules that restrict the model's components in some way.

The use of restrictive rules can be for a number of reasons, but the 2 main reasons that you will typically be creating these rules for are: to simplify the model, and to ensure that the model is accurate.

When there is a need to simplify a model so that it can run more efficiently, it may be possible to do this by reducing the number of attributes, but this will usually not have much of an effect as attributes do not cause much of an impact to memory usage or runtime.

Instead, restrictive rules can be created to reduce or completely remove specific behaviours from the model. This is much more likely to lead to performance gains, as behaviours require interactions between agents to occur, and if the behaviour is not of particular interest, then the number of interactions can be reduced.

The other case when you would likely want to introduce restrictive rules is when the problem definition itself requires these restrictions to be more accurate.

A good example of this happens in Schelling's model, where agents can only move into empty spaces. Given that you want the model to be representative of real life, then it would not make sense for the people in one household to be able to move into a household that is already occupied.

Interaction Rules

Rules to do with interactions are primarily used for maintaining the accuracy of the model, and computationally defining the nature of the problem.

As previously mentioned, there are 2 major types of interactions in an ABM that we are interested in: agent-agent interactions, and agent-space interactions.

When we are defining how agent-agent interactions may occur, there are a number of variables to consider for any particular agent, such as:

- What other agents can it interact with?
- When will these interactions happen?
- What attributes/information are needed for this interaction to happen?
- What is the purpose of this interaction?
- Which outcomes are expected from this interaction?

Obviously, your considerations don't have to be limited to these, as these are highly dependent on your specific model. But these provide a good foundation for the rules you will be designing.

In Schelling's model, the rules of interaction are relatively simple: at every model iteration, each agent looks at its immediate neighbours and observes what group they belong to.

But nowadays, models will rarely be this straightforward… If we use Schelling's model as the example, but extend on it to include more information, we can quickly see how rules of interactions can get complicated.

If instead of being 2 groups in the community, there were 20, and each of these groups had a different level of social interaction, then the rules of interaction would be different. Instead, when it comes time to observe neighbours, agents from one group might only look at their immediate neighbours, but agents from another group would want to look at their neighbour's neighbours as well, and so on.

There may also exist interaction rules for agent-space interactions, which would take into account similar questions when being defined. The only difference being that rather than looking at an agent in the context of other agents, we are looking at the agent in the context of their environment.

And so, the rules of interaction that are defined for your model must take these things into consideration, so that you can decide how strict and specific you want to design these rules to be.

Behavioural Rules

Finally, we have behavioural rules, which determine the way in which agents and the environment react to interactions and changes in conditions.

Similar to interaction rules, behavioural rules will typically be aiming to ensure the accuracy of the model and the phenomenon being simulated. And these rules may also be applied differently depending on how the agent is reacting to other agents (agent-agent) versus how they are reacting to their environment (agent-space).

Given that behavioural rules outline the ways that agents will react to a changing situation, these are where there is most potential for impact to the model's emergent behaviours.

In Schelling's model, the behavioural rules are simple again: after following the rules of interaction and observing their neighbours, each agent then determines if there are enough agents that belong to their same group before deciding if they will move.

Random or Deterministic?

Regardless of which type of rule we are talking about, an important decision that must be made when defining them is whether the rule will follow a random or deterministic process.

Again, this decision strongly depends on your particular problem, and the computational resources that you have available,

Stochastic (random) processes will mean a larger computational overhead as a large number of random values and thresholds will have to be drawn each time that a rule is applied to an agent.

Whereas deterministic rules will simply use the same values and thresholds each time.

The big difference between stochastic or deterministic rules comes down to the model's explanatory power and the nature of the problem.

Some phenomena in nature (e.g. radioactive decay) are entirely stochastically determined, and would require stochastic processes to model them accurately.

And there are also other problems that aren't inherently random by nature, but they are better explained when some components within them are stochastically determined (e.g. social interactions).

For some models, such as Schelling's, the use of deterministic thresholds and values is enough to produce valid emergent behaviours that align with what is seen in reality.

Emergent Behaviours

You may have noticed that throughout the previous sections, we have kept mentioning the idea of "emergent behaviour", so let us explore it in more detail.

Definition: Emergent behaviour is any behaviour of interest that is produced as a final outcome of the agent-based modelling process.

If we use the traditional terminology of the scientific method, you could then say that "emergent behaviours" in an ABM are equivalent to the dependent variable (i.e. the results that are being analysed).

In most cases of ABM, you will be designing you model around an established problem, with a somewhat clear expectation of what behaviours are expected as an outcome, and an idea of what outcomes would be nonsensical and outright impossible following common sense.

However, there may be some cases where the outcome is not clearly defined, because the process that is being modelled has too many variables to accurately predict. A good example of this in real life are opinion polls prior to federal elections. In many cases, the polls may indicate that it is a "certainty" for candidate A to win the election, but then for some reason or another, candidate B ends up winning.

And this is due to multiple factors. It could be possible that by chance, the people polled were just not representative of all voters. Candidate A may have catastrophically blundered and lost voter support between the polls and the election. Or the elections may have been rigged.

But, ABMs have never been intended to be used to create deterministic predictions, rather they are used as tools to better simulate and understand the processes behind the behaviours, and how these can be affected by different factors.

Approaching the problem from this perspective, the real utility of an ABM comes from its ability to deeply explore and understand the process rather than the outcome itself.

It is much more interesting to be able to simulate the weeks leading up to the election to see how political propaganda and the candidates' actions affected the opinions of the voters.

Then assuming that the model is accurate, if the final results don't reflect what really happened in the election, we can start exploring possible explanations such as electoral fraud.

Although, this doesn't mean that ABMs couldn't be used as predictive tools to some extent. Simply knowing how certain conditions have historically impacted behaviours is enough information to be able to change those behaviours in the future.

We can also use these models to simulate hypothetical conditions, and then see what types of behaviours emerge from those conditions; which then allows us to preemptively avoid conditions leading to "negative outcomes", or foster conditions leading to "positive outcomes".

How to tell if behaviours have emerged

Now that you've seen what emergent behaviour is and what it means in the overall scheme of ABM, you may be asking yourself "How can I tell when emergent behaviours appear?".

Given that emergent behaviours are treated as the dependant variable in your model, it is not enough to simply say that any and all outcomes of the model are "emergent behaviours" in and of themselves.

Rather, you should be defining a model outcome as an "emergent behaviour" based on its usefulness towards explaining your dependant variable(s).

And typically, emergent behaviour is easy to detect. Almost all model outcomes that can be defined as emergent behaviours will: be directly or indirectly, but closely related to you dependant variable(s), separate agents into distinct "behavioural groups", and have clearly detectable patterns.

Using Schelling's model as an example, the emergent behaviour is clearly seen to be the formation of homogeneous group neighbourhoods. This progressive homogeneity is directly related to the dependant variable (segregation), separates the agents into distinct groups (the different neighbourhoods), and has a clearly detectable pattern (the agents start to have more and more neighbours belonging to their own group).

Despite emergent behaviours typically being clear to see, it is important to not become "tunnel-visioned" into a strict preconception of what the emergent behaviour "should" look like. If you focus too much on what you are expecting to see, then it becomes easy to miss emergent behaviours that – while equally valid – are not what you were expecting.

The Scientific Method

In relation to this last point, it is worth briefly touching on the "scientific method", and the general ways in which you should be aiming to carry out you research.

It has become increasingly common in academia to relentlessly pursue "significant" results that agree with some initial hypotheses. This is not how research should be carried out.

It is important to have your initial hypotheses to inform you about how to design your experiments, and the approach that you want to take in your methodology. Your initial hypotheses will also give you a good starting point for a literature review, and the areas of past research that you should focus on.

But once your experiments have been designed, you should completely forget about your hypotheses until you are analysing your results. That is to say, you should not start cherry-picking data points and constantly changing your model because you are not satisfied with the results you are getting.

You should instead try to be as neutral as possible, and interpret the results with the least amount of bias as possible. Even if you do not agree with the results, that doesn't change the fact that those results reflect reality, and denying or altering those results is the same as denying the reality we live in.

Results that directly go against your initial hypotheses, or simply do not provide strong support for them, are still valid and important results nonetheless. Knowing what *isn't* true is equally as useful as knowing what *is*...

Analysing Your Final Results

When it comes to analysing your final results, there are a number of ways that this can be done, and it depends on what your particular needs are.

In broad terms, there are two main ways that you would be analysing the results of your ABM: statistical methods, and visualisations.

Although, you will likely be using both of these methods together, and there will always be a similar basic order of analysis that you will be following:

1. Define all relevant data that you need or want from your results
2. Collect this data during and after the model's runtime
3. Perform an initial, basic exploratory analysis
4. Look for any obvious initial patterns or trends to inform further analysis
5. Carry out more complicated statistical and visual analyses

   5.1. Possibly include specific analytical tests that have been established in literature 5.2. Include any other tests or explorations that are specific to your problem
6. Finally, gather all your observations and start to describe your findings

This isn't set in stone, and you can approach the analysis of your results however works best for you, but this is a good starting point for the majority of cases.

Now let's look into the statistical methods and visualisations that you might be using for your analysis in a bit more detail.

Statistical Methods

Statistical methods will probably be the foundation of your analysis in most cases, as these use concrete mathematical equations and the quantitative data produced by your model to produce definitive metrics and significant conclusions.

The most basic of the statistical tests, which can be used for your initial exploratory analysis, are the descriptive statistics:

- **Mean** (μ) = $\frac{1}{n}(\sum_{i=1}^{n} x_i)$ = sum of all values / number of values
- **Median** = The value that appears at the exact middle of a set of values
- **Mode** = The value that appears the most frequently in a set of values
- **Standard Deviation** (σ) = $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$
- **Quartiles** = Splitting the data into evenly sized subsets

These initial statistics provide you with enough information to be able to see any obvious patterns in the data, or areas which are in need of more detailed analysis to provide useful results.

From these basic statistics, we can then go into more detailed metrics or tests which are chosen depending on the information you got from this first step.

Below we'll briefly go over some of the more commonly encountered complex statistical methods:

KMeans Clustering:

KMeans clustering is a method which is defined through the mathematical equation:

$$\arg\min_{S} \sum_{i=1}^{k} \sum_{x \in S_i} ||x - \mu_i||^2$$

where μ_i is the mean of the points in cluster S_i, defined by:

$$\mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x$$

In simple terms, the KMeans equation is looking for the values of clusters S which minimise the variance (σ^2) present across all clusters.

The main use of clustering in your analysis will be to determine if there are any major "groups" formed within the output data, and how well the output data actually separates itself into those groups.

If after performing clustering on the data, there is very little variance across the determined clusters, then you can be more confident that there are clear differences between the groups in the data.

On the other hand, if the algorithm has trouble converging on a solution, or whatever solutions it proposes have a very high variance, it cannot be said that the output data is significantly heterogeneous.

It is also worth noting that clustering can be used when constructing the initial model in some cases; allowing for the initial population of agents to be simplified into aggregate groups for simulation.

Chi-Squared (χ^2) Tests:

χ^2 tests are primarily used for null-hypothesis testing, in which one looks at how likely it is for an observed result to have been produced assuming that some initial hypothesis were true.

The most widely used of the χ^2 tests is Pearson's χ^2 test, whose test statistic is calculated through the following equation:

$$\chi^2 = N \sum_{i=1}^{n} \frac{(O_i/N - p_i)^2}{p_i}$$

where

- χ^2 = Pearson's cumulative test statistic
- O_i = the number of observations of type i
- N = the total number of observations
- p_i = the theoretical probability of type i appearing under the null hypothesis
- n = the total number of types

Once you have calculated the χ^2 test statistic, you compare it against the χ^2-distribution with the appropriate degrees of freedom, and see how far away from the center of that distribution the test statistic is.

If the test statistic is far enough away from the center of the distribution (the 95th percentile is the typical cutoff used), then there is evidence that the observed results are statistically significantly different from those that would be produced under the null hypothesis.

The χ^2 test can also be used in combination with clustering, and you can further confirm that the clusters generated by the algorithm are indeed significantly different from each other.

Log Odds:

You may already be familiar with probability (p), which is a measure of how likely it is for some event to occur, or some value to appear in a population.

The probability can be further extended into "odds", which are the relative ratio of the probability being true versus it being false, and is defined as $\frac{p}{1-p}$.

The log odds, as the name would suggest, are produced by simply applying the natural logarithm to the odds:

$$logodds(p) = \ln(\frac{p}{1-p})$$

The log odds provide the same fundamental information as the traditional odds – which is the relative ratio of p being true versus p being false.

But the benefit to using log odds over traditional odds is that it allows for a much more detailed comparison of odds values, especially in situations where the traditional odds may take extreme values (very small or very large).

Visualisations

Visualisations can play a complementary role in the initial exploratory analysis, but they are most useful for contextualising and clarifying the exploratory statistics.

While you can realistically visualise pretty much any result, in the case of ABMs you will likely be creating visualisations which can be categorised into one of two broad groups.

Model Visualisations:

Model visualisations are the more unique type of visualisations that can be generated from an ABM, and are probably what you are thinking of when you hear about visualisations in this context.

Statistic Visualisations:

Statistic visualisations, on the other hand, don't necessarily provide any new information, but they extend on what you can see in the generated statistics.

1.4.3 Beyond Traditional ABMs

Clustering Algorithms

Social Networks

> **Note**
>
> For the context of GATOH, "social network" refers to a network of people that interact with each other socially, *not* a social media platform.

Graphs

Multilayer Graphs

1.5 GATOH Fundamentals

CHAPTER

TWO

GATOH API REFERENCE

gatoh.agents.Agent(*args, **kwargs)	A class to define the Agent objects that will interact with each other in an agent-based model.

2.1 Agent

class Agent(**args: T*, ***kwargs: T*)

Bases: object

A class to define the Agent objects that will interact with each other in an agent-based model.

Supported positional arguments:

- <string> to set the Agent's id.
- <dict> of {hierarchy_name : weight} for the personal value that this Agent assigns to each social hierarchy
- <float> in the range [-1, 1] to set the Agent's initial opinion on the topic of interest
- <bool> to define if the socially contagious belief will be of personal benefit to this Agent
- <(string, float)> for the Agent's defined personality and their social susceptibility (range [0, 1])

Any arbitrary keyword arguments that are passed to initialise this object will create a new attribute for the Agent object, and store the value of that attribute as the input data type.

If any pre-existing attributes are passed as keyword arguments, this simply sets the attributes to the input value.

If both a positional argument and a corresponding keyword argument are passed for the same attribute, the value given to the keyword argument will override whatever was passed as the positional value.

Parameters

- **id** (*str, optional*) – Positional argument – provides a unique identifier for an agent.
- **social_weightings** (*dict[str, float], optional*) – Positional argument – <hierarchy : weighting> mapping indicating the personal value that the agent assigns to each social hierarchy.
- **opinion** (*float, optional*) – Positional argument – the current opinion value that the agent is being initialised with.
- **personal_benefit** (*bool, optional*) – Positional argument – a flag indicating if the socially contagious belief is of personal benefit to the agent.

- **behaviour_tuple** (*tuple[str, float], optional*) – Positional argument – a pair of variables detailing the agent's defined personality and social susceptibility (range [0, 1]), respectively.
- **index** (*int, optional*) – Keyword argument – the agent's index within an AgentSet.
- **is_silenced** (*dict[str, bool], optional*) – Keyword argument – <hierarchy : flag> mapping indicating in which social hierarchies the agent is silenced.
- **previous_opinion** (*float, optional*) – Keyword argument – the opinion value held by the agent at the previous model iteration.
- **radicalised** (*bool, optional*) – Keyword argument – a flag indicating if the agent is radicalised.
- **rw_distributions** (*dict[str, tuple[float, float]], optional*) – Keyword argument – <hierarchy : random walk params> mapping for agent-specific dynamic hierarchy weightings.
- **opinion_rw** (*tuple[float, float], optional*) – Keyword argument – (mean, variance) parameters for agent-specific stochastic opinion shifts.

Methods

add_attribute	Dynamically add an attribute to this Agent object.
change_opinion	A setter method that changes the Agent's current opinion by a given delta value.
change_opinion_rw	A setter method that changes the Agent's explicit opinion random walk parameters.
change_personality	A setter method that changes the Agent's personality type.
change_radicalisation	A setter method that changes the Agent's radicalisation value.
change_rw_distribution	A setter method that changes the Agent's explicit random walk parameters for a specific social hierarchy.
deradicalisation	Uses the agent's own opinion as well as the neighbours' opinions to determine if an already radicalised agent will deradicalise.
evolve_hierarchies	Experimental function that aims to model the constantly evolving 'intrinsic value' that Agents place on the social hierarchies that they belong in over time.
generate_agent	Randomly generate an Agent object based on the input parameters.
get_attribute	Return any existing or dynamically added attribute held by the Agent object.
life_events	Experimental function that aims to model the ways in which Agent behaviours change according to major random life events over time.
opinion_negation	Checks if the Agent has experienced sufficiently 'overwhelming' social pressure in a hierarchy leading to a complete reversal of their opinion.
opinion_silencing	Determines if an agent will become silenced in a given social hierarchy based on their attributes.

continues on next page

Table 2 – continued from previous page

radicalisation	Uses the agent's own opinion as well as the neighbours' opinions to determine if the agent has become radicalised in their actions.
set_benefit	A setter method that changes the Agent's personal_benefit parameter.
step	Step the individual agent object:
stochastic_benefit_change	Calls to this function are primarily meant to originate from *life_events()*.
stochastic_opinion	Determine the stochastic direction and magnitude of a shift in the Agent's opinion and apply it.
stochastic_personality_change	Calls to this function are primarily meant to originate from *life_events()*.
stochastic_radicalisation_change	Calls to this function are primarily meant to originate from *life_events()*.
stochastic_silencing_change	Calls to this function are primarily meant to originate from *life_events()*.
store_previous_opinion	A setter method that stores the Agent's current opinion into the previous opinion.
update	Updates the internal state of the agent after the model has stepped:

2.1.1 gatoh.agents.Agent.add_attribute

Agent.**add_attribute**(*name: str*, *value: T | dict[S, T] | tuple[T, ...] | list[T] | None = None*, *parameters: dict[str, float] | None = None*, *distribution: str | None = None*, *overwrite: bool = True*) → None

Dynamically add an attribute to this Agent object. If "value" is passed, an explicit initial value is given; if "mean" and "var" are passed, a value is generated from a random distribution. Supported random distributions are:

- "gaussian"
- "beta"
- "gamma"
- "uniform"
- "levy"

Either "value", or parameters with "mean" and "var" included should be passed to this function, not both.

If both an explicit value and distribution parameters are input to this function, the explicit value will always override the distribution parameters when setting the attribute.

Parameters

- **name** (*str*) – The name of the attribute to be added.
- **value** (*Any, optional*) – Initial value of the attribute.
- **parameters** (*dict[str, float], optional*) – The distribution parameters that will be used with the specified distribution for parameter generation.
- **distribution** (*str, optional*) – String to select which random distribution will be used to generate the value.
- **overwrite** (*bool, optional*) – A flag indicating if the added attribute should override any existing attributes of the same name.

Raises

- **ValueError** – If no valid value or distribution parameters are input, the attribute cannot be added.
- **UserWarning** – If overwrite is explicitly False but the attribute is existing, a warning is raised without completing the operation.

2.1.2 gatoh.agents.Agent.change_opinion

Agent.**change_opinion**(*opinion_delta: float*) → None

A setter method that changes the Agent's current opinion by a given delta value.

Parameters
opinion_delta (*float*) – The delta value by which to shift the Agent's current opinion.

Raises
TypeError – If opinion_delta is not a float.

2.1.3 gatoh.agents.Agent.change_opinion_rw

Agent.**change_opinion_rw**(*rw_params: tuple[float, float]*) → None

A setter method that changes the Agent's explicit opinion random walk parameters.

Parameters
rw_params (*tuple[float, float]*) – The new (mean, var) for the random walk's gaussian distribution.

Raises
TypeError – If rw_params is not a (float, float) tuple.

2.1.4 gatoh.agents.Agent.change_personality

Agent.**change_personality**(*personality: str*) → None

A setter method that changes the Agent's personality type.

Parameters
personality (*str*) – The new personality type to assign to the Agent.

Raises

- **TypeError** – If the input personality is not a string.
- **ValueError** – If the personality is not one of the supported types.

2.1.5 gatoh.agents.Agent.change_radicalisation

Agent.**change_radicalisation**(*radicalisation: bool*) → None

A setter method that changes the Agent's radicalisation value.

Parameters
radicalisation (*bool*) – The radicalisation flag to set.

Raises
TypeError – If radicalisation is not a boolean.

2.1.6 gatoh.agents.Agent.change_rw_distribution

Agent.**change_rw_distribution**(*hierarchy: str*, *parameters: tuple[float, float]*) → None

A setter method that changes the Agent's explicit random walk parameters for a specific social hierarchy.

Parameters

- **hierarchy** (*str*) – The name of the hierarchy whose rw parameters are being changes.
- **parameters** (*tuple[float, float]*) – The new (mean, var) for the random walk's gaussian distribution.

Raises

TypeError – If either of the inputs contain invalid data types.

2.1.7 gatoh.agents.Agent.deradicalisation

Agent.**deradicalisation**(*hierarchy_changes: list[float]*, *neighbour_benefits: list[bool]*, *threshold: float*) → bool

Uses the agent's own opinion as well as the neighbours' opinions to determine if an already radicalised agent will deradicalise.

Parameters

- **hierarchy_changes** (*list[float]*) – The opinion changes caused in each social hierarchy by neighbours during this iteration.
- **neighbour_benefits** (*list[bool]*) – Flags indicating the presence of personal benefit across an agent's neighbours.
- **threshold** (*float*) – The deradicalisation threshold that has been defined at the global level in the model.

Raises

- **RuntimeError** – If the agent has not yet been initialised appropriately.
- **TypeError** – If any of the input parameters contains an invalid data type.

Returns

A flag indicating if the agent has deradicalised or not.

Return type

bool

2.1.8 gatoh.agents.Agent.evolve_hierarchies

Agent.**evolve_hierarchies**(*rw_distributions: dict[str, tuple[float, float]]*) → None

Experimental function that aims to model the constantly evolving 'intrinsic value' that Agents place on the social hierarchies that they belong in over time.

Parameters

rw_distributions (*dict[str, tuple[float, float]]*) – A mapping specifying the global random walk distributions defined for each hierarchy in the model.

Raises

- **TypeError** – If the input parameter contains an invalid data type.
- **KeyError** – If a non-existent hierarchy key is passed in the input dictionary.

2.1.9 gatoh.agents.Agent.generate_agent

Agent.**generate_agent**(*id: str*, *index: int*, *hierarchies: list[str]*, *distribution: str = 'gaussian'*, *explicit_rw: bool = False*, *explicit_opinion_rw: bool = False*, *personality: str | None = None*, *parameters: dict[str, float] | None = None*, *personal_benefit: bool | None = None*) → *Agent*

Randomly generate an Agent object based on the input parameters.

Parameters

- **id** (*str*) – The id that has been assigned for this specific Agent object under the conditions of the model specifications.
- **index** (*int*) – The index of the Agent object within the model's AgentSet.
- **hierarchies** (*list[str]*) – The names of all valid social hierarchies in the model.
- **distribution** (*str, optional*) – The distribution to use for relevant attribute generation (Valid distributions include: 'gaussian', 'beta')
- **explicit_rw** (*bool, optional*) – A flag indicating if explicit hierarchy weighting random walk parameters should be generated for this Agent.
- **explicit_opinion_rw** (*bool, optional*) – A flag indicating if an explicit opinion random walk parameter should be generated for this Agent.
- **personality** (*str, optional*) – A string defining what type of personality the agent will have (defaults to 'neutral' on Agent __init__)
- **parameters** (*dict[str, float], optional*) – A dictionary containing the distribution parameters used to generate random values.
- **personal_benefit** (*bool, optional*) – A boolean indicating if the Agent would be personally benefitted by the adoption of the 'social virus' being spread.

Raises

- **TypeError** – If any of the required input parameters are of the incorrect data type.
- **ValueError** – If the input personality is not a supported type.

Returns
The generated Agent object.

Return type
Agent

2.1.10 gatoh.agents.Agent.get_attribute

Agent.**get_attribute**(*name: str*) → T | None

Return any existing or dynamically added attribute held by the Agent object.

Parameters
name (*str*) – The name of the attribute to get.

Raises
UserWarning – If the attribute does not exist.

Returns
The value stored for the attribute.

Return type
Any

2.1.11 gatoh.agents.Agent.life_events

Agent.**life_events**(*personality_changes: bool = False*, *benefit_changes: bool = False*, *radicalisation_changes: bool = False*, *silencing_changes: bool = False*, *parameters: LifeEventsDict | None = None*) → None

Experimental function that aims to model the ways in which Agent behaviours change according to major random life events over time.

Parameters

- **personality_changes** (*bool, optional*) – A flag indicating if personality changes are allowed to occur due to life events.
- **benefit_changes** (*bool, optional*) – A flag indicating if changes in personal benefit status are allowed to occur due to life events.
- **radicalisation_changes** (*bool, optional*) – A flag indicating if spontaneous changes to radicalisation status are allowed to occur due to life events.
- **silencing_changes** (*bool, optional*) – A flag indicating if spontaneous changes to hierarchy silencing status are allowed to occur due to life events.
- **parameters** (*dict[str, Any], optional*) – A <key, value> mapping which outlines all relevant thresholds or other values used in this function.

2.1.12 gatoh.agents.Agent.opinion_negation

Agent.**opinion_negation**(*hierarchy: str*, *absolute_difference: float*, *threshold: float*) → bool

Checks if the Agent has experienced sufficiently 'overwhelming' social pressure in a hierarchy leading to a complete reversal of their opinion.

Parameters

- **hierarchy** (*str*) – The name of the social hierarchy where opinion negation is being checked for.
- **absolute_difference** (*float*) – The absolute difference between the perceived opinion climate and the Agent's own opinion.
- **threshold** (*float*) – A global model threshold that has been specified for this effect to occur.

Raises

- **RuntimeError** – If the agent has not yet been initialised appropriately.
- **TypeError** – If the input parameters contain any invalid data types.
- **KeyError** – If a non-existent hierarchy is passed as an input.

Returns

A flag indicating if the Agent's opinion experienced a total negation.

Return type

bool

2.1.13 gatoh.agents.Agent.opinion_silencing

Agent.**opinion_silencing**(*estimated_opinion_climate: float*, *silencing_threshold: float | None = None*) → tuple[bool, float]

Determines if an agent will become silenced in a given social hierarchy based on their attributes.

If no silencing threshold has been passed, each Agent's own social susceptibility is used as the threshold instead.

Parameters

- **estimated_opinion_climate** (*float*) – The opinion climate perceived by the Agent in this hierarchy (not necessarily objectively 'accurate').
- **silencing_threshold** (*float, optional*) – A hierarchy or global silencing threshold that must be surpassed for silencing to occur.

Raises

- **RuntimeError** – If opinion silencing is being called before the agent has been initialised appropriately.
- **TypeError** – If the input parameters contain any invalid data types.

Returns

A pair of values indicating if silencing occurs, and the absolute difference between the perceived opinion climate and the Agent's own opinion, respectively.

Return type

tuple[bool, float]

2.1.14 gatoh.agents.Agent.radicalisation

Agent.**radicalisation**(*hierarchy_changes: list[float]*, *neighbour_benefits: list[bool]*, *threshold: float*) → bool

Uses the agent's own opinion as well as the neighbours' opinions to determine if the agent has become radicalised in their actions.

Parameters

- **hierarchy_changes** (*list[float]*) – The opinion changes caused in each social hierarchy by neighbours during this iteration.
- **neighbour_benefits** (*list[bool]*) – Flags indicating the presence of personal benefit across an agent's neighbours.
- **threshold** (*float*) – The radicalisation threshold that has been defined at the global level in the model.

Raises

- **RuntimeError** – If the agent has not yet been initialised appropriately.
- **TypeError** – If any of the input parameters contains an invalid data type.

Returns

A flag indicating if the Agent has become radicalised or not.

Return type

bool

2.1.15 gatoh.agents.Agent.set_benefit

Agent.**set_benefit**(*personal_benefit: bool*) → None

A setter method that changes the Agent's personal_benefit parameter.

Parameters
personal_benefit (*bool*) – A flag indicating if there is personal benefit for the agent in believing the contagion.

Raises
TypeError – If personal_benefit is not a boolean.

2.1.16 gatoh.agents.Agent.step

Agent.**step**(*rw_distributions: dict[str, tuple[float, float]]*, *opinion_rw: tuple[float, float]*) → None

Step the individual agent object:

1. Handle dynamic social hierarchy weightings
2. Handle the stochastic opinion changes experienced by the Agent

Parameters

- **rw_distributions** – A mapping of <hierarchy name : (mean, variance)> defining the random walk distributions of each social hierarchy weighting in the model.
- **opinion_rw** (*tuple[float, float]*) – A (mean, variance) tuple defining the random walk distribution at the model level for the Agent opinions.

2.1.17 gatoh.agents.Agent.stochastic_benefit_change

Agent.**stochastic_benefit_change**() → None

Calls to this function are primarily meant to originate from *life_events()*.

Will flip the value of the agent's personal benefit attribute when called.

Raises
RuntimeError – If the agent has not yet been initialised appropriately.

2.1.18 gatoh.agents.Agent.stochastic_opinion

Agent.**stochastic_opinion**(*opinion_rw: tuple[float, float]*) → None

Determine the stochastic direction and magnitude of a shift in the Agent's opinion and apply it.

This is representative of opinion changes in real social networks in which individual opinions may increase or decrease away from the aggregate mean (independent of external influences).

Parameters
opinion_rw (*tuple[float, float]*) – A (mean, variance) pair which parametrises the Gaussian distribution used for stochastic opinion shift.

Raises

- **RuntimeError** – If the agent has not yet been initialised appropriately.
- **TypeError** – If the input contains any invalid data types.

2.1.19 gatoh.agents.Agent.stochastic_personality_change

Agent.**stochastic_personality_change**(*personality_probs: PersonalityProbs | None = None*) → None

Calls to this function are primarily meant to originate from *life_events()*.

Will redraw a valid personality type from the ones that have been defined, and then change the agent's personality to this new type.

Parameters

personality_probs (*dict[str, float]*) – Specific per-personality type probabilities to be used for drawing the new agent personality.

Raises

- **TypeError** – If a non-float probability is supplied.
- **KeyError** – If an unsupported personality type is passed.

2.1.20 gatoh.agents.Agent.stochastic_radicalisation_change

Agent.**stochastic_radicalisation_change**() → None

Calls to this function are primarily meant to originate from *life_events()*.

Will flip the value of the agent's personal benefit attribute when called.

2.1.21 gatoh.agents.Agent.stochastic_silencing_change

Agent.**stochastic_silencing_change**(*silencing_probs: dict[str, float] | None = None*) → None

Calls to this function are primarily meant to originate from *life_events()*.

Will flip the silenced status of an agent within their hierarchies, with the possibility of per-hierarchy probabilities of this occurring.

Parameters

silencing_probs (*dict[str, float], optional*) – The per-hierarchy probabilities of a flip in the silencing status occurring.

2.1.22 gatoh.agents.Agent.store_previous_opinion

Agent.**store_previous_opinion**() → None

A setter method that stores the Agent's current opinion into the previous opinion.

2.1.23 gatoh.agents.Agent.update

Agent.**update**(*opinion_silenced: dict[str, bool]*, *negation_ocurred: bool*) → None

Updates the internal state of the agent after the model has stepped:

1. Updates what social hierarchies the Agent's opinion is currently silenced in
2. Inverts the Agent's current opinion if opinion negation ocurred

Parameters

- **opinion_silenced** – A mapping of <hierarchy : flag> indicating which social hierarchies the Agent is silencing themselves in.
- **negation_ocurred** (*bool*) – A flag indicating if opinion negation has ocurred in the current iteration.

Raises

- **RuntimeError** – If the agent has not yet been initialised in a valid manner.
- **TypeError** – If either of the input parameters are of the incorrect data type.
- **KeyError** – If a non-existent hierarchy is included in the keys for opinion_silenced.

gatoh.agents.AgentSet()	An ordered collection of Agent objects that maintains consistency for the Model.

2.2 AgentSet

class AgentSet

Bases: `object`

An ordered collection of Agent objects that maintains consistency for the Model.

Methods

add	Add an Agent to the AgentSet.
agent_at_index	Returns the Agent object at the given index in the AgentSet.
agents_at_indices	Returns a list of all the agent objects at the specified indices.
benefit_count	A getter function that returns the number of agents which experience personal benefit from social contagion in the agent set.
discard	Removes an Agent from the AgentSet which matches the input Agent; does not return an error if the Agent does not exist.
discard_index	Removes the Agent at the specified index in the AgentSet; does not return an error if the index is out of bounds.
extract_agent_pickle	A helper function that allows for multithreading of *load_agentset()*.
get_agent_by_id	Searches the AgentSet for an Agent with the given id and returns its object if it exists.
get_agent_ids	A getter function that returns the IDs of all agents contained within the agent set.
get_agents_by_ids	Searches the AgentSet for Agents with the given ids and returns their objects if they all exist.
get_index	Returns the index within the AgentSet of the input Agent object.
get_indices	Returns the indices within the AgentSet of the input Agent objects.
id_in_agentset	Report if an agent with the given unique ID exists in the agentset.
load_agentset	Loads an AgentSet that has been saved following the same process as in the *save_agentset()* function.

continues on next page

Table 4 – continued from previous page

personality_count	A getter function that returns the count of agents which hold a specific personality type in the agent set.
personality_counts	A getter function that returns the counts of all personality types among agents in the agent set.
radicalised_count	A getter function that returns the number of radicalised agents that are present in the agent set.
remove	Removes an agent from the AgentSet which matches the input agent; returning an error if such an Agent does not exist.
remove_index	Removes the Agent at the specified index in the AgentSet; returning an error if the index is out of bounds.
sample	Randomly draw n Agents from the AgentSet without replacement.
save_agentset	Save the Agent objects into a compressed subdirectory representing the saved AgentSet.
update_indices	Iterate over the AgentSet and update the current Agent object index values.
write_agent_pickle	A helper function that allows for multithreading of *save_agentset()*.

2.2.1 gatoh.agents.AgentSet.add

AgentSet.**add**(*agent:* Agent) → int

Add an Agent to the AgentSet.

Parameters
agent (Agent) – The Agent object to be added.

Returns
The index of the newly added Agent.

Return type
int

2.2.2 gatoh.agents.AgentSet.agent_at_index

AgentSet.**agent_at_index**(*index: int*) → *Agent* | None

Returns the Agent object at the given index in the AgentSet.

Parameters
index (*int*) – The index within the AgentSet to inspect.

Raises
UserWarning – If the input index is out of bounds, raise a warning and return None.

Returns
The Agent object at the specified index.

Return type
Agent

2.2.3 gatoh.agents.AgentSet.agents_at_indices

AgentSet.**agents_at_indices**(*indices: list[int]*) → list[*Agent*]

Returns a list of all the agent objects at the specified indices.

Parameters
indices (*list[int]*) – A list of all the indices for which the agent should be returned.

Raises
UserWarning – If any of the input indices are out of bounds.

Returns
All of the agent objects that correspond to the input indices.

Return type
list[*Agent*]

2.2.4 gatoh.agents.AgentSet.benefit_count

AgentSet.**benefit_count**() → int

A getter function that returns the number of agents which experience personal benefit from social contagion in the agent set.

Returns
The number of benefited agents in the agent set.

Return type
int

2.2.5 gatoh.agents.AgentSet.discard

AgentSet.**discard**(*agent:* Agent) → bool

Removes an Agent from the AgentSet which matches the input Agent; does not return an error if the Agent does not exist.

Parameters
agent (Agent) – The Agent object that should be removed from the set.

Returns
A flag indicating if the Agent was removed successfully or not.

Return type
bool

2.2.6 gatoh.agents.AgentSet.discard_index

AgentSet.**discard_index**(*index: int*) → bool

Removes the Agent at the specified index in the AgentSet; does not return an error if the index is out of bounds.

Parameters
index (*int*) – The index in the AgentSet which is to be removed.

Returns
A flag indicating if the Agent was removed successfully or not.

Return type
bool

2.2.7 gatoh.agents.AgentSet.extract_agent_pickle

AgentSet.**extract_agent_pickle**(*agent_pickle_name: str*, *subdirectory_path: str*) → *Agent*

A helper function that allows for multithreading of *load_agentset()*.

Parameters

- **agent_pickle_name** (*str*) – The name of the pickled Agent object file.
- **subdirectory_path** (*str*) – The root path to which the pickled agent object was written.

Returns
The unpickled agent.

Return type
Agent

2.2.8 gatoh.agents.AgentSet.get_agent_by_id

AgentSet.**get_agent_by_id**(*id: str*) → *Agent*

Searches the AgentSet for an Agent with the given id and returns its object if it exists.

Parameters
id (*str*) – The id that was assigned to the Agent object at creation.

Raises
KeyError – If the input id does not exist in the AgentSet.

Returns
The Agent object with the specified id.

Return type
Agent

2.2.9 gatoh.agents.AgentSet.get_agent_ids

AgentSet.**get_agent_ids**() → list[str]

A getter function that returns the IDs of all agents contained within the agent set.

Returns
All of the contained agents' IDs.

Return type
list[str]

2.2.10 gatoh.agents.AgentSet.get_agents_by_ids

AgentSet.**get_agents_by_ids**(*ids: list[str]*) → list[*Agent*]

Searches the AgentSet for Agents with the given ids and returns their objects if they all exist.

Parameters
ids (*list[str]*) – The ids that have been assigned to every Agent object at creation.

Returns
The Agent objects with the specified ids.

Return type
list[*Agent*]

2.2.11 gatoh.agents.AgentSet.get_index

AgentSet.**get_index**(*agent:* Agent) → int

Returns the index within the AgentSet of the input Agent object.

Parameters
agent (Agent) – The agent whose index is being searched for.

Raises
KeyError – If the input Agent does not exist in the AgentSet.

Returns
The index of the agent within the AgentSet.

Return type
int

2.2.12 gatoh.agents.AgentSet.get_indices

AgentSet.**get_indices**(*agents: list[*Agent*]*) → list[int]

Returns the indices within the AgentSet of the input Agent objects.

Parameters
agents (*list[*Agent*]*) – The agents whose indices are being searched for.

Returns
The indices of the agents within the AgentSet.

Return type
list[int]

2.2.13 gatoh.agents.AgentSet.id_in_agentset

AgentSet.**id_in_agentset**(*agent_id: str*) → bool

Report if an agent with the given unique ID exists in the agentset.

Parameters
agent_id (*str*) – The unique ID of the agent that is being checked for.

Returns
A flag indicating if an agent with the input ID exists in the agentset or not.

Return type
bool

2.2.14 gatoh.agents.AgentSet.load_agentset

AgentSet.**load_agentset**(*load_path: str*) → None

Loads an AgentSet that has been saved following the same process as in the *save_agentset()* function.

Parameters
load_path (*str*) – The path to the model's overall save directory.

Raises
FileNotFoundError – If no valid agentset zip was found in the load path.

2.2.15 gatoh.agents.AgentSet.personality_count

AgentSet.**personality_count**(*personality: str*) → int

A getter function that returns the count of agents which hold a specific personality type in the agent set.

Parameters
personality (*str*) – The personality type that is being counted.

Raises
ValueError – If the personality type input is unsupported.

Returns
The count of agents which hold the specified personality type.

Return type
int

2.2.16 gatoh.agents.AgentSet.personality_counts

AgentSet.**personality_counts**() → dict[str, int]

A getter function that returns the counts of all personality types among agents in the agent set.

Returns
The count of each supported personality type among agents in the agent set.

Return type
dict[str, int]

2.2.17 gatoh.agents.AgentSet.radicalised_count

AgentSet.**radicalised_count**() → int

A getter function that returns the number of radicalised agents that are present in the agent set.

Returns
The number of radicalised agents in the agent set.

Return type
int

2.2.18 gatoh.agents.AgentSet.remove

AgentSet.**remove**(*agent:* Agent) → bool

Removes an agent from the AgentSet which matches the input agent; returning an error if such an Agent does not exist.

Parameters
agent (Agent) – The agent that should be removed from the set.

Raises
KeyError – If the input Agent does not exist in the AgentSet.

Returns
A flag indicating that the Agent was removed successfully.

Return type
bool

2.2.19 gatoh.agents.AgentSet.remove_index

AgentSet.**remove_index**(*index: int*) → bool

Removes the Agent at the specified index in the AgentSet; returning an error if the index is out of bounds.

Parameters
index (*int*) – The index in the AgentSet which is to be removed.

Raises
IndexError – If the input index is out of bounds for the AgentSet.

Returns
A flag indicating that the Agent was removed successfully.

Return type
bool

2.2.20 gatoh.agents.AgentSet.sample

AgentSet.**sample**(*n: int*) → list[*Agent*]

Randomly draw n Agents from the AgentSet without replacement.

Parameters
n (*int*) – The number of agents to sample.

Returns
The agents sampled from the AgentSet.

Return type
list[*Agent*]

2.2.21 gatoh.agents.AgentSet.save_agentset

AgentSet.**save_agentset**(*directory_path: str*) → None

Save the Agent objects into a compressed subdirectory representing the saved AgentSet.

Parameters
directory_path (*str*) – The path to the directory where the agentset subdirectory should be created.

2.2.22 gatoh.agents.AgentSet.update_indices

AgentSet.**update_indices**() → None

Iterate over the AgentSet and update the current Agent object index values.

2.2.23 gatoh.agents.AgentSet.write_agent_pickle

AgentSet.**write_agent_pickle**(*agent:* Agent, *subdirectory_path: str*) → str

A helper function that allows for multithreading of *save_agentset()*.

Parameters

- **agent** (Agent) – The agent that is being saved.
- **subdirectory_path** (*str*) – The path to the subdirectory in which the agents are being saved.

Returns
The path to which the agent pickle was saved.

Return type
str

gatoh.graphs.Graph(name, rw_params[, ...])	A graph class that defines a single agent-based model layer.

2.3 Graph

class Graph(*name: str*, *rw_params: tuple[float, float]*, *generation_method: str = ''*, *suppress_warnings: bool = False*, *dynamic_rels: bool = True*)

Bases: `object`

A graph class that defines a single agent-based model layer.

This corresponds to the agents' attitudes towards one another with respect to different social hierarchies.

Parameters

- **name** (*str*) – The name of the social hierarchy that this Graph object will be representing.
- **rw_params** (*tuple[float, float]*) – The (mean, variance) of the normal distribution used for the dynamic relationships random walk.
- **generation_method** (*str, optional*) – The random graph generation method that should be used where relevant.
- **suppress_warnings** (*bool, optional*) – A flag indicating if non-critical warnings should be suppressed.
- **dynamic_rels** (*bool, optional*) – A flag indicating if dynamic relationships should be modelled for this social hierarchy.

Methods

add_edges	Creates appropriate GraphEdges from the given dictionary and then adds these to the graph.
add_nodes	Creates appropriate GraphNodes from the given Agents, and then adds these to the graph.
agent_in_graph	A simple function that checks whether an Agent exists within a Graph.
agent_opinion_change	Changes the specified Agent's current opinion by the given delta.
agent_previous_opinion	Set the specified Agent's previous opinion to be equal to the current opinion (before the current opinion changes in the current iteration).
agent_radicalisation_change	Change the specified Agent's radicalisation status.
calculate_polarisation	Calculates the level of opinion polarisation in this *Graph* based on the equation:
change_generation_params	Setter function which outlines the existing generation parameters used in generate_graph() and allows the user to alter them.
change_weights	Updates the weight of the relationship between two agents in the graph.

continues on next page

Table 6 – continued from previous page

cluster_nodes	A function that will use the betweenness centrality measures of the nodes in the graph to separate them into a specific number of clusters.
compute_subgraph_center	Calculates the center of a cluster using the betweenness centrality mapping.
dynamic_relationships	Uses the (mean, variance) passed at initialisation to draw random walk values by which each edge (relationship) in the hierarchy will be shifted.
estimate_neighbour_opinions	Return the individual opinion climate values perceived by the Agent for each other Agent within this social hierarchy.
estimate_opinion_climate	Return the unique opinion climate perceived by the Agent within this social hierarchy.
generate_graph	Randomly generate edges between existing Graph nodes and add them to the graph.
generate_group_edges	Looks at the members that make up the input groups, and their individual connections within the graph's agents to determine which groups are connected to each other.
get_agent_index	Searches for the node index in the Graph which corresponds to the input Agent object.
get_betweenness_centrality	A wrapper function that makes the rustworkx `digraph_betweenness_centrality()` function be a callable instance method for Graph.
get_dijkstra_all_pairs	A wrapper function that makes the rustworkx `digraph_all_pairs_dijkstra_path_lengths()` function be a callable instance method for Graph.
get_edge	A getter function to access GraphEdge objects.
get_edge_changes	A getter function that returns the current register of pending edge changes for this graph, and then resets the attribute to an empty entry.
get_edges	A getter function to access multiple GraphEdge objects at once.
get_neighbours	Finds all the nodes in the graph with direct relationships to the specified Agent.
get_node	A getter function to access GraphNode objects.
get_nodes	A getter function to access multiple GraphNode objects at once.
get_relationship	Return a directed relationship from one node to another.
get_relationships	Retrieves and reports the bidirectional relationship weightings between two nodes in the Graph.
load_graph	Loads a Graph object stored in the GraphML format from the given path.
neighbour_influences	Looks at all the neighbours for an Agent and uses the neighbours' own opinions plus the weight of the relationship between Agents to return a final value by which the given Agent's opinion value will increment or decrement.
node_from_agent	Returns the GraphNode object corresponding to the given Agent object.

continues on next page

Table 6 – continued from previous page

node_from_agent_id	Returns the GraphNode object corresponding to the Agent with the given ID.
node_relationships_count	Report the number of ingoing and outgoing relationships that a specific node is involved in.
register_edge_change	A helper function that records a pending edge change for this graph.
relationship_exists	Checks for the existence of a relationship (weighted edge) between two Agents (nodes).
remove_edge	Removes a single edge from the graph corresponding to the indicated directed node indices.
remove_edge_index	Removes a single edge from the graph corresponding to the input edge index.
remove_node	Removes a node from the graph, along with any relationships involving it.
save_graph	Saves the existing Graph object in the GraphML format to the given path.
step	Step the individual Graph object:
stochastic_relationships	An experimental function which aims to simulate the possible formation and disintegration of relationships within a social network.
update_edge_indices	Iterates over all the existing edges in the graph and updates their stored indices to reflect the current graph state.
update_node_indices	Iterates over all the existing nodes in the graph and updates their stored indices to reflect the current graph state.

2.3.1 gatoh.graphs.Graph.add_edges

Graph.**add_edges**(*edges: dict[str, list[Any]]*) → None

Creates appropriate GraphEdges from the given dictionary and then adds these to the graph.

The parameter `edges` has been typed as `Any` to simplify typechecking.

Parameters

edges (*dict[str, list[int | float | str | tuple[float, float]]]*) – A mapping of <key : list> where each key corresponds to (from_node, to_node, [optional] weighting, [optional] name, [optional] rw_param).

2.3.2 gatoh.graphs.Graph.add_nodes

Graph.**add_nodes**(*agents: Iterable[*Agent*]*) → None

Creates appropriate GraphNodes from the given Agents, and then adds these to the graph.

Parameters

agents (*Iterable[*Agent*]*) – The agents that will be converted to GraphNodes and added to the graph.

2.3.3 gatoh.graphs.Graph.agent_in_graph

Graph.**agent_in_graph**(*agent:* Agent) → bool

A simple function that checks whether an Agent exists within a Graph.

Parameters
agent (Agent) – the agent whose existence in the Graph is being checked for.

Returns
A flag indicating if the Agent exists in the Graph.

Return type
bool

2.3.4 gatoh.graphs.Graph.agent_opinion_change

Graph.**agent_opinion_change**(*agent:* Agent, *change_delta: float*, *deradicalisation: bool = False*) → None

Changes the specified Agent's current opinion by the given delta.

Parameters

- **agent** (Agent) – The agent whose current opinion is being changed.
- **change_delta** (*float*) – The value by which to change the Agent's current opinion.
- **deradicalisation** (*bool, optional*) – A flag indicating if this opinion change is occurring after deradicalisation.

2.3.5 gatoh.graphs.Graph.agent_previous_opinion

Graph.**agent_previous_opinion**(*agent:* Agent) → None

Set the specified Agent's previous opinion to be equal to the current opinion (before the current opinion changes in the current iteration).

Parameters
agent (Agent) – The agent whose previous opinion is being set.

2.3.6 gatoh.graphs.Graph.agent_radicalisation_change

Graph.**agent_radicalisation_change**(*agent:* Agent, *radicalisation: bool*) → None

Change the specified Agent's radicalisation status.

Parameters

- **agent** (Agent) – The agent whose radicalisation status is being changed.
- **radicalisation** (*bool*) – The new radicalisation status.

2.3.7 gatoh.graphs.Graph.calculate_polarisation

Graph.**calculate_polarisation**() → float

Calculates the level of opinion polarisation in this *Graph* based on the equation:

$$\pi(k) = \frac{1}{|K|(|K|-1)} \sum_{i \neq j}^{i \in K, j \in K} (d_{ij} - y)^2$$

where K is the set of agents within this Graph, d_{ij} is the distance between the opinions of agents i and j, and y is the mean opinion distance among all agents in this Graph.

Returns
The measure of opinion radicalisation in this social hierarchy.

Return type
float

2.3.8 gatoh.graphs.Graph.change_generation_params

Graph.**change_generation_params**(***params: int | float*) → None

Setter function which outlines the existing generation parameters used in generate_graph() and allows the user to alter them.

Parameters

- **p** (*float, optional*) – The probability of edge rewiring (small-world) or edge creation (random).
- **m** (*int, optional*) – The number of nearest neighbours that each node is connected to initially (scale-free).
- **sbm_sizes** (*int, optional*) – The size of generated blocks (blockmodel).

Raises

UserWarning – If invalid parameter keys or data types are input to the function.

2.3.9 gatoh.graphs.Graph.change_weights

Graph.**change_weights**(*node_1: int*, *node_2: int*, *value: float*) → None

Updates the weight of the relationship between two agents in the graph.

If no relationship previously exists, a new one is created.

Parameters

- **node_1** (*int*) – The index of some Agent in the graph.
- **node_2** (*int*) – The index of some other Agent in the graph.
- **value** (*float*) – The new weight to assign.

2.3.10 gatoh.graphs.Graph.cluster_nodes

Graph.**cluster_nodes**(*k: int*, *n_iters: int = 40*) → dict[*GraphNode*, int]

A function that will use the betweenness centrality measures of the nodes in the graph to separate them into a specific number of clusters.

This function uses:

- KMeans as the main clustering algorithm
- Dijkstra all-pairs path lengths for the distances between nodes
- Betweenness centrality for the KMeans evaluation metric when determining centers

Parameters

- **k** (*int*) – The number of clusters to separate the graph into.
- **n_iters** (*int, optional*) – The maximum number of iterations to attempt running the KMeans algorithm for.

Returns

An <Agent ID : Cluster ID> mapping that outlines the generated clusters for this graph.

Return type

dict[str, str]

2.3.11 gatoh.graphs.Graph.compute_subgraph_center

Graph.**compute_subgraph_center**(*subgraph_mapping: CentralityMapping*) → int

Calculates the center of a cluster using the betweenness centrality mapping.

Parameters
subgraph_mapping (*dict[int, float]*) – A <Node index : centrality measure> mapping containing the betweenness centrality for each node in the graph.

Returns
The index of the node with the greatest centrality (the center of the cluster).

Return type
int

2.3.12 gatoh.graphs.Graph.dynamic_relationships

Graph.**dynamic_relationships**() → None

Uses the (mean, variance) passed at initialisation to draw random walk values by which each edge (relationship) in the hierarchy will be shifted. Aims to simulate dynamic relationships between agents across timesteps.

2.3.13 gatoh.graphs.Graph.estimate_neighbour_opinions

Graph.**estimate_neighbour_opinions**(*agent:* Agent) → dict[str, float]

Return the individual opinion climate values perceived by the Agent for each other Agent within this social hierarchy.

Parameters
agent (Agent) – The agent which is estimating its neighbours' opinions.

Returns
A mapping of <neighbour id : estimated opinion value> for each opinion perceived by the agent in the hierarchy.

Return type
dict[str, float]

2.3.14 gatoh.graphs.Graph.estimate_opinion_climate

Graph.**estimate_opinion_climate**(*agent:* Agent) → float

Return the unique opinion climate perceived by the Agent within this social hierarchy.

Parameters
agent (Agent) – The agent which is estimating the opinion climate.

Returns
The agent's perceived 'aggregated opinion' of this whole social hierarchy.

Return type
float

2.3.15 gatoh.graphs.Graph.generate_graph

Graph.**generate_graph**(*agents: list[*Agent*]*, *method: str = ''*, *relationship_range: tuple[float, float] = (-1.0, 1.0)*, *ensure_complete: bool = True*) → Self

Randomly generate edges between existing Graph nodes and add them to the graph.

Parameters

- **agents** (*list[*Agent*]*) – The subset of Agents in the base model that are being used as the nodes for this graph.
- **method** (*str, optional*) – The random generation method to use. Possible choices include: 'small-world', 'scale-free', 'random', 'blockmodel'; Defaults to 'small-world'.
- **relationship_range** (*tuple[float, float], optional*) – The valid range of generated relationship strengths (at most, constrained to [-1.0, 1.0]).
- **ensure_complete** (*bool, optional*) – A flag indicating if the generated graph should be complete or not (in the case of 'small-world').

Raises

- **ValueError** – If no agents are being passed to this function.
- **ValueError** – If an invalid random generation method is being passed to this function.

Returns
A reference to this Graph object.

Return type
Graph

2.3.16 gatoh.graphs.Graph.generate_group_edges

Graph.**generate_group_edges**(*groups: list[*Group*]*) → list[tuple[int, int]]

Looks at the members that make up the input groups, and their individual connections within the graph's agents to determine which groups are connected to each other.

Parameters
groups (*list[*Group*]*) – The groups for which the edges are being generated.

Raises
ValueError – If any of the groups contain members which do not belong to this graph.

Returns
A list containing (from_group, to_group) indices that define the connections of the groups.

Return type
list[tuple[int, int]]

2.3.17 gatoh.graphs.Graph.get_agent_index

Graph.**get_agent_index**(*agent:* Agent) → int | None

Searches for the node index in the Graph which corresponds to the input Agent object.

Parameters
agent (Agent) – The agent whose index is being searched for.

Returns
The agent's node index within the social hierarchy Graph if it exists, or None otherwise.

Return type
int | None

2.3.18 gatoh.graphs.Graph.get_betweenness_centrality

Graph.**get_betweenness_centrality**(*subgraph: PyDiGraph[*GraphNode, GraphEdge*] | None = None*) → CentralityMapping

A wrapper function that makes the rustworkx `digraph_betweenness_centrality()` function be a callable instance method for Graph.

Parameters
subgraph (PyDiGraph, optional) – A subgraph for which to calculate the betweenness centrality rather than the full graph.

Returns
A <Node index : centrality measure> mapping containing the betweenness centrality for each node in the graph.

Return type
dict[int, float]

2.3.19 gatoh.graphs.Graph.get_dijkstra_all_pairs

Graph.**get_dijkstra_all_pairs**(*subgraph: PyDiGraph[*GraphNode, GraphEdge*] | None = None*) → AllPairsPathLengthMapping

A wrapper function that makes the rustworkx `digraph_all_pairs_dijkstra_path_lengths()` function be a callable instance method for Graph.

Parameters
subgraph (PyDiGraph, optional) – A subgraph for which to calculate the dijkstra all pairs path lengths rather than the full graph.

Returns
A <Node index : <Node index : path length>> nested mapping outlining the path lengths of all pairs in the graph.

Return type
dict[int, dict[int, float]]

2.3.20 gatoh.graphs.Graph.get_edge

Graph.**get_edge**(*edge_index: int*) → *GraphEdge* | None

A getter function to access GraphEdge objects.

Parameters
edge_index (*int*) – The index of the edge to access.

Raises
RuntimeWarning – If the edge index is out of bounds for this Graph.

Returns
The graph edge if the index was valid, or None otherwise.

Return type
GraphEdge | None

2.3.21 gatoh.graphs.Graph.get_edge_changes

Graph.**get_edge_changes**() → dict[str, *EdgeChanges*]

A getter function that returns the current register of pending edge changes for this graph, and then resets the attribute to an empty entry.

Returns
A <"ID1,ID2" : edge change> mapping that outlines the edge changes being made between two agents in the graph.

Return type
dict[str, *EdgeChanges*]

2.3.22 gatoh.graphs.Graph.get_edges

`Graph.`**`get_edges`**`(`*edge_indices: list[int]*`)` → list[*GraphEdge*]

A getter function to access multiple GraphEdge objects at once.

Parameters
`edge_indices` (*`list[int]`*) – The indices of the edges to access.

Raises
`RuntimeWarning` – If any of the edge indices is out of bounds for this Graph.

Returns
A list containing all the valid graph edges that were found.

Return type
list[*GraphEdge*]

2.3.23 gatoh.graphs.Graph.get_neighbours

`Graph.`**`get_neighbours`**`(`*agent:* Agent`)` → list[*GraphNode*] | None

Finds all the nodes in the graph with direct relationships to the specified Agent.

Parameters
`agent` (`Agent`) – The agent for which the neighbours are being examined.

Raises
`UserWarning` – If the input agent does not exist in the Graph.

Returns
A list of the graph nodes belonging to the direct neighbours of the agent.

Return type
list[*GraphNode*]

2.3.24 gatoh.graphs.Graph.get_node

`Graph.`**`get_node`**`(`*node_index: int*`)` → *GraphNode* | None

A getter function to access GraphNode objects.

Parameters
`node_index` (*`int`*) – The index of the node to access.

Raises
`RuntimeWarning` – If the node index is out of bounds for this Graph.

Returns
The graph node if the index was valid, or None otherwise.

Return type
GraphNode | None

2.3.25 gatoh.graphs.Graph.get_nodes

Graph.**get_nodes**(*node_indices: list[int]*) → list[*GraphNode*]

A getter function to access multiple GraphNode objects at once.

Parameters
node_indices (*list[int]*) – The indices of the nodes to access.

Raises
RuntimeWarning – If any of the node indices are out of bounds.

Returns
A list of all the valid graph nodes that were found.

Return type
list[*GraphNode*]

2.3.26 gatoh.graphs.Graph.get_relationship

Graph.**get_relationship**(*from_node:* Agent, *to_node:* Agent) → float

Return a directed relationship from one node to another.

Parameters

- **from_node** (Agent) – The node that the relationship originates from.
- **to_node** (Agent) – The node that the relationship points to.

Returns
The weighting of the directed relationship (from_node -> to_node).

Return type
float

2.3.27 gatoh.graphs.Graph.get_relationships

Graph.**get_relationships**(*node_1: int*, *node_2: int*) → dict[tuple[int, int], float] | None

Retrieves and reports the bidirectional relationship weightings between two nodes in the Graph.

Parameters

- **node_1** (*int*) – the node index of Agent 1.
- **node_2** (*int*) – the node index of Agent 2.

Returns
The bidirectional edge weightings between two nodes (if they exist).

Return type
dict[tuple[int, int], float] | None

2.3.28 gatoh.graphs.Graph.load_graph

Graph.**load_graph**(*path: str*, *name: str*, *rw_params: tuple[float, float] | None = None*) → None

Loads a Graph object stored in the GraphML format from the given path. The social hierarchy name must be explicitly passed with this call.

Parameters

- **path** (*str*) – Path to a stored graph file.
- **name** (*str*) – The name of the hierarchy that the stored Graph belongs to.

- **rw_params** (*tuple[float, float], optional*) – The mean and variance of the Graph's random walk distribution.

2.3.29 gatoh.graphs.Graph.neighbour_influences

Graph.**neighbour_influences**(*agent:* Agent) → float | None

Looks at all the neighbours for an Agent and uses the neighbours' own opinions plus the weight of the relationship between Agents to return a final value by which the given Agent's opinion value will increment or decrement.

Parameters
agent (Agent) – The agent for which the strength of opinion change is being determined.

Raises
UserWarning – If the input agent does not exist in the Graph.

Returns
The final change in the Agent's opinion caused by their neighbours in this hierarchy.

Return type
float

2.3.30 gatoh.graphs.Graph.node_from_agent

Graph.**node_from_agent**(*agent:* Agent) → *GraphNode* | None

Returns the GraphNode object corresponding to the given Agent object.

Parameters
agent (Agent) – The agent being searched for in the GraphNodes.

Returns
The graph node corresponding to the input Agent if it exists, or None otherwise.

Return type
GraphNode | None

2.3.31 gatoh.graphs.Graph.node_from_agent_id

Graph.**node_from_agent_id**(*agent_id: str*) → *GraphNode* | None

Returns the GraphNode object corresponding to the Agent with the given ID.

Parameters
agent_id (*str*) – The ID of the agent being searched for in the GraphNodes.

Returns
The graph node corresponding to the input agent id if it exists, or None otherwise.

Return type
GraphNode | None

2.3.32 gatoh.graphs.Graph.node_relationships_count

Graph.**node_relationships_count**(*node_index: int*) → int

Report the number of ingoing and outgoing relationships that a specific node is involved in.

Parameters
node_index (*int*) – The index of the node that is being inspected.

Raises
IndexError – If the input node index is out of bounds.

Returns
The total number of relationships involving the input node.

Return type
int

2.3.33 gatoh.graphs.Graph.register_edge_change

Graph.**register_edge_change**(*from_node: int*, *to_node: int*, *weighting: float*) → None

A helper function that records a pending edge change for this graph.

Parameters

- **from_node** (*int*) – The index of the origin node.
- **to_node** (*int*) – The index of the destination node.
- **weighting** (*float*) – The new weighting that is being assigned to the edge.

2.3.34 gatoh.graphs.Graph.relationship_exists

Graph.**relationship_exists**(*from_node: int*, *to_node: int*) → int | None

Checks for the existence of a relationship (weighted edge) between two Agents (nodes).

Parameters

- **from_node** (*int*) – the node index of the parent node.
- **to_node** (*int*) – the node index of the child node.

Returns
The index of the edge if the relationship exists, or None otherwise.

Return type
int | None

2.3.35 gatoh.graphs.Graph.remove_edge

Graph.**remove_edge**(*from_node: int*, *to_node: int*) → None

Removes a single edge from the graph corresponding to the indicated directed node indices.

Throws a warning without interrupting the runtime if the edge did not exist in the first place.

Parameters

- **from_node** (*int*) – the parent node in the edge.
- **to_node** (*int*) – the child node in the edge.

Raises
UserWarning – If the edge (from_node -> to_node) does not exist in the Graph.

2.3.36 gatoh.graphs.Graph.remove_edge_index

Graph.**remove_edge_index**(*edge_index: int*) → None

Removes a single edge from the graph corresponding to the input edge index.

Raises an error if the edge index is out of bounds of existing edges.

This function is meant as an extended wrapper to rustworkx `remove_edge_from_index()` that also handles cases where nodes are left neighbourless after the edge removal.

Parameters
edge_index (*int*) – The index of the edge to remove.

Raises
KeyError – If the input edge index is out of bounds.

2.3.37 gatoh.graphs.Graph.remove_node

Graph.**remove_node**(*node: int*) → None

Removes a node from the graph, along with any relationships involving it.

Parameters
node (*int*) – The node index to remove from the graph.

Raises
IndexError – If the node index is out of bounds.

2.3.38 gatoh.graphs.Graph.save_graph

Graph.**save_graph**(*path: str*) → None

Saves the existing Graph object in the GraphML format to the given path.

Parameters
path (*str*) – Path to which the Graph will be saved.

Raises
OSError – If the graph is not successfully written.

2.3.39 gatoh.graphs.Graph.step

Graph.**step**() → None

Step the individual Graph object:

1. Handle dynamic relationships within the graph.

2.3.40 gatoh.graphs.Graph.stochastic_relationships

Graph.**stochastic_relationships**(*formation: bool = False*, *formation_thresh: float = 0.999*, *formation_num: int = 1*, *disintegration: bool = False*, *disintegration_thresh: float = 0.999*, *disintegration_num: int = 1*) → None

An experimental function which aims to simulate the possible formation and disintegration of relationships within a social network.

Parameters

- **formation** (*bool, optional*) – A flag indicating if the spontaneous formation of new relationships should be allowed.

- **formation_thresh** (*float, optional*) – The threshold that must be surpassed for spontaneous formation to occur.
- **formation_num** (*int, optional*) – The number of new relationships to form.
- **disintegration** (*bool, optional*) – A flag indicating if the spontaneous disintegration of existing relationships should be allowed.
- **disintegration_thresh** (*float, optional*) – The threshold that must be surpassed for spontaneous disintegration to occur.
- **disintegration_num** (*int, optional*) – The number of existing relationships to disintegrate.

2.3.41 gatoh.graphs.Graph.update_edge_indices

Graph.**update_edge_indices**() → None

Iterates over all the existing edges in the graph and updates their stored indices to reflect the current graph state.

Will also update the graph edge_count attribute.

2.3.42 gatoh.graphs.Graph.update_node_indices

Graph.**update_node_indices**() → None

Iterates over all the existing nodes in the graph and updates their stored indices to reflect the current graph state.

Will also update the graph node_count attribute.

gatoh.graphs.GraphNode(agent)	A helper class that allows rustworkx to more efficiently store information about Agents in the graph nodes.

2.4 GraphNode

class GraphNode(*agent:* Agent)

Bases: object

A helper class that allows rustworkx to more efficiently store information about Agents in the graph nodes.

Parameters
agent (Agent) – The agent that is being associated with this GraphNode.

Methods

set_index	A setter method to set the GraphNode's index value.

2.4.1 gatoh.graphs.GraphNode.set_index

GraphNode.**set_index**(*idx: int*) → None

A setter method to set the GraphNode's index value.

Parameters
idx (*int*) – The index to set for this GraphNode.

gatoh.graphs.GraphEdge(hierarchy, from_node, ...)	A helper class that allows rustworkx to more efficiently store information about Agent relationships in the graph edges.

2.5 GraphEdge

class GraphEdge(*hierarchy: str*, *from_node: int*, *to_node: int*, *weighting: float = 0.0*, *rw_params: tuple[float, float] | None = None*)

Bases: `object`

A helper class that allows rustworkx to more efficiently store information about Agent relationships in the graph edges. As the social hierarchies are assumed to be DiGraphs, each GraphEdge is directional, and the social weighting that Agent A places on Agent B will not necessarily be equally reciprocated.

Parameters

- **hierarchy** (*str*) – The name of the social hierarchy that this edge belongs to.
- **from_node** (*int*) – The index of the origin node.
- **to_node** (*int*) – The index of the destination node.
- **weighting** (*float, optional*) – The relationship opinion value that is being assigned (range [-1, 1]).
- **rw_params** (*tuple[float, float], optional*) – The normal distribution (mean, variance) to assign for relationship-specific random walk effects.

Methods

has_rw_params	A function that checks if this relationship has explicit random walk parameters.
set_index	A setter function that changes this GraphEdge's index value.
set_rw_params	A setter function that changes this GraphEdge's rw_params value.
set_weighting	A setter function that changes this GraphEdge's weighting value.
update_from_node	A setter function that updates the from_node's index for this GraphEdge.
update_to_node	A setter function that updates the to_node's index for this GraphEdge.

2.5.1 gatoh.graphs.GraphEdge.has_rw_params

GraphEdge.**has_rw_params**() → bool

A function that checks if this relationship has explicit random walk parameters.

Returns
A flag indicating if random walk parameters exist.

Return type
bool

2.5.2 gatoh.graphs.GraphEdge.set_index

GraphEdge.**set_index**(*idx: int*) → None

A setter function that changes this GraphEdge's index value.

Parameters
idx (*int*) – The index to store for this GraphEdge.

2.5.3 gatoh.graphs.GraphEdge.set_rw_params

GraphEdge.**set_rw_params**(*rw_params: tuple[float, float]*) → None

A setter function that changes this GraphEdge's rw_params value.

Parameters
rw_params (*tuple[float, float]*) – A (mean, variance) tuple specifying this relationship's unique random walk distribution.

2.5.4 gatoh.graphs.GraphEdge.set_weighting

GraphEdge.**set_weighting**(*value: float*) → None

A setter function that changes this GraphEdge's weighting value.

Parameters
value (*float*) – The new weighting to store for this GraphEdge.

2.5.5 gatoh.graphs.GraphEdge.update_from_node

GraphEdge.**update_from_node**(*idx: int*) → None

A setter function that updates the from_node's index for this GraphEdge.

Parameters
idx (*int*) – The from_node's new index value to update to.

2.5.6 gatoh.graphs.GraphEdge.update_to_node

GraphEdge.**update_to_node**(*idx: int*) → None

A setter function that updates the to_node's index for this GraphEdge.

Parameters
idx (*int*) – The to_node's new index value to update to.

gatoh.graphs.GraphSet([graphs, ...])	A class that will collect all of the different social hierarchy graphs in the same structure and provide utilities using this collection.

2.6 GraphSet

class GraphSet(*graphs: list[*Graph*] | None = None*, *stochastic_relationships: dict[str, bool] | None = None*, *stochastic_rels_flags: dict[str, tuple[bool, bool]] | None = None*)

Bases: `object`

A class that will collect all of the different social hierarchy graphs in the same structure and provide utilities using this collection.

Parameters

- **graphs** (*list[*Graph*], optional*) – Existing Graph objects that should be added to the GraphSet.
- **stochastic_relationships** (*dict[str, bool], optional*) – A <hierarchy: flag> mapping indicating if the stochastic formation and disintegration of relationships should be modelled for a hierarchy.
- **stochastic_rels_flags** (*dict[str, tuple[bool, bool]], optional*) – A <hierarchy: flags> mapping providing the necessary stochastic_relationships flags per-graph.

Methods

add_graph	A setter function to add a new Graph object to the GraphSet.
agent_opinion_threshold	A utility function that iterates over the GraphSet and records for which social hierarchies a specific Agent's weighting of those hierarchies is above a certain threshold value.
calculate_polarisation	A wrapper that calls a specific hierarchy graph's calculate_polarisation function and returns its value.
get_agent_hierarchies	A helper function that determines which social hierarchies an Agent is contained in.
get_agents_hierarchies	A helper function that determines which social hierarchies multiples Agents are contained in.
get_hierarchies	A getter function that returns the Graph objects with the given hierarchy names.
get_hierarchy	A getter function to return a Graph object with the given hierarchy name.
get_index	A getter function that returns the index of a given hierarchy within the GraphSet.
get_indices	A getter function that returns the indices of the given hierarchies within the GraphSet.
graph_at_index	A getter function to return a Graph object stored at the given index in the GraphSet.
graphs_at_indices	A getter function to return the Graph objects stored at the given indices in the GraphSet.
list_hierarchies	A utility function that iterates over the GraphSet and prints out the names of all the social hierarchies that are present.
load_edge	A helper function that allows for multithreading within *load_graphset_multi()*.
load_graphset	Loads a GraphSet that has been saved following the same process as in the save_graphset() function.
load_graphset_multi	A helper function that allows for parallel processing of Graph loading for *load_graphset()*.
load_node	A helper function that allows for multithreading within *load_graphset_multi()*.
save_graphset	Save all of the graphs contained within this graphset into a compressed subdirectory representing the saved GraphSet.
set_stochastic_rels	A setter function that defines whether stochastic relationships should be modelled for a specific hierarchy.

continues on next page

Table 12 – continued from previous page

write_edge_pickle	A helper function that allows for multithreading of `save_graphset()`.
write_node_pickle	A helper function that allows for multithreading of *save_graphset()*.

2.6.1 gatoh.graphs.GraphSet.add_graph

GraphSet.**add_graph**(*graph:* Graph) → None

A setter function to add a new Graph object to the GraphSet.

Parameters
graph (Graph) – The graph to add to the GraphSet.

2.6.2 gatoh.graphs.GraphSet.agent_opinion_threshold

GraphSet.**agent_opinion_threshold**(*agent:* Agent, *threshold: float = 0.9*) → Iterable[str]

A utility function that iterates over the GraphSet and records for which social hierarchies a specific Agent's weighting of those hierarchies is above a certain threshold value.

Parameters

- **agent** (Agent) – The agent for which to check the AgentSet for.
- **threshold** (*float, optional*) – The absolute threshold value over which the Agent's opinion is considered significant.

Returns
The names of the hierarchies for which the agent's weighting is above the threshold.

Return type
Iterable[str]

2.6.3 gatoh.graphs.GraphSet.calculate_polarisation

GraphSet.**calculate_polarisation**(*hierarchy: str*) → float

A wrapper that calls a specific hierarchy graph's calculate_polarisation function and returns its value.

Parameters
hierarchy (*str*) – The name of the hierarchy for which polarisation is being calculated.

Raises
ValueError – If the input hierarchy does not exist or otherwise cannot be retrieved.

Returns
The hierarchy polarisation value.

Return type
float

2.6.4 gatoh.graphs.GraphSet.get_agent_hierarchies

GraphSet.**get_agent_hierarchies**(*agent:* Agent) → list[str]

A helper function that determines which social hierarchies an Agent is contained in.

Parameters
agent (Agent) – The agent for which hierarchy memberships are being determined.

Returns
The names of all the social hierarchies to which the input Agent belongs in.

Return type
list[str]

2.6.5 gatoh.graphs.GraphSet.get_agents_hierarchies

GraphSet.**get_agents_hierarchies**(*agents: list[*Agent*]*) → dict[str, list[str]]

A helper function that determines which social hierarchies multiples Agents are contained in.

Parameters
agents (*list[*Agent*]*) – The agents for which hierarchy memberships are being determined.

Returns
A <Agent ID : hierarchies> mapping of all the social hierarchies that Agents belong to.

Return type
dict[str, list[str]]

2.6.6 gatoh.graphs.GraphSet.get_hierarchies

GraphSet.**get_hierarchies**(*hierarchies: list[str]*) → list[*Graph*]

A getter function that returns the Graph objects with the given hierarchy names.

Parameters
hierarchies (*list[str]*) – The names of the social hierarchies represented by the Graphs to return.

Returns
The Graph objects of the specified hierarchies.

Return type
list[*Graph*]

2.6.7 gatoh.graphs.GraphSet.get_hierarchy

GraphSet.**get_hierarchy**(*hierarchy: str*) → *Graph* | None

A getter function to return a Graph object with the given hierarchy name.

Parameters
hierarchy (*str*) – The name of the social hierarchy represented by the Graph to return.

Returns
The Graph object of the specified hierarchy, or None if no matching hierarchy was found.

Return type
Graph | None

2.6.8 gatoh.graphs.GraphSet.get_index

GraphSet.**get_index**(*hierarchy: str*) → int

A getter function that returns the index of a given hierarchy within the GraphSet.

Parameters
hierarchy (*str*) – The name of the hierarchy that is being searched for.

Raises
KeyError – If the input hierarchy does not exist within the GraphSet.

Returns
The index of the hierarchy within the GraphSet.

Return type
int

2.6.9 gatoh.graphs.GraphSet.get_indices

GraphSet.**get_indices**(*hierarchies: list[str]*) → list[int]

A getter function that returns the indices of the given hierarchies within the GraphSet.

Parameters
hierarchies (*list[str]*) – The names of the hierarchies that are being searched for.

Returns
The indices of the hierarchies within the GraphSet.

Return type
list[int]

2.6.10 gatoh.graphs.GraphSet.graph_at_index

GraphSet.**graph_at_index**(*graph_index: int*) → *Graph* | None

A getter function to return a Graph object stored at the given index in the GraphSet.

Parameters
graph_index (*int*) – The index of the graph to return.

Returns
The graph object to return if it exists, or None otherwise.

Return type
Graph | None

2.6.11 gatoh.graphs.GraphSet.graphs_at_indices

GraphSet.**graphs_at_indices**(*graph_indices: list[int]*) → list[*Graph*]

A getter function to return the Graph objects stored at the given indices in the GraphSet.

Parameters
graph_indices (*list[int]*) – The indices of the graphs to return.

Returns
The graph objects at the corresponding indices.

Return type
list[*Graph*]

2.6.12 gatoh.graphs.GraphSet.list_hierarchies

GraphSet.**list_hierarchies**(*print_out: bool = False*) → list[str]

A utility function that iterates over the GraphSet and prints out the names of all the social hierarchies that are present.

Parameters
print_out (*bool, optional*) – A flag indicating if the listed hierarchies should be printed to the terminal.

Returns
The names of all social hierarchies present in the GraphSet.

Return type
list[str]

2.6.13 gatoh.graphs.GraphSet.load_edge

GraphSet.**load_edge**(*edge_dir: str*, *edge_file: str*) → tuple[*GraphEdge*, int]

A helper function that allows for multithreading within *load_graphset_multi()*.

Parameters

- **edge_dir** (*str*) – The root directory where all graph edges have been saved.
- **edge_file** (*str*) – The name of the pickle file that is being loaded.

Returns
The unpickled graph edge object and its corresponding index in the graph.

Return type
tuple[*GraphEdge*, int]

2.6.14 gatoh.graphs.GraphSet.load_graphset

GraphSet.**load_graphset**(*load_path: str*, *rw_params: dict[str, tuple[float, float]]*, *worker_pool: Pool | None = None*) → None

Loads a GraphSet that has been saved following the same process as in the save_graphset() function.

Parameters

- **load_path** (*str*) – The path to the model's overall save directory.
- **rw_params** (*dict[str, tuple[float, float]]*) – A <name : rw_params> mapping containing the relevant external information for each graph.
- **worker_pool** (Pool, optional) – A pool of workers that can distribute the processing of the graphset loading amongst themselves.

Raises
FileNotFoundError – If the input load_path does not point to a valid file.

2.6.15 gatoh.graphs.GraphSet.load_graphset_multi

GraphSet.**load_graphset_multi**(*save_dir: str*, *subdirectory_path: str*, *rw_params: tuple[float, float]*) → *Graph*

A helper function that allows for parallel processing of Graph loading for *load_graphset()*.

Parameters

- **save_dir** (*str*) – The path of the directory to which all of a graph's files have been saved to.
- **subdirectory_path** (*str*) – The path to the subdirectory in which the graph's save directory is located.

- **rw_params** (*tuple[float, float]*) – The (mean, variance) random-walk parameters for the graph that is being loaded.

Returns
A loaded graph with all included nodes and edges.

Return type
Graph

2.6.16 gatoh.graphs.GraphSet.load_node

GraphSet.**load_node**(*node_dir: str*, *node_file: str*) → tuple[*GraphNode*, int]

A helper function that allows for multithreading within *load_graphset_multi()*.

Parameters

- **node_dir** (*str*) – The root directory where all graph nodes have been saved.
- **node_file** (*str*) – The name of the pickle file that is being loaded.

Returns
The unpickled graph node object and its corresponding index in the graph.

Return type
tuple[*GraphNode*, int]

2.6.17 gatoh.graphs.GraphSet.save_graphset

GraphSet.**save_graphset**(*directory_path: str*) → None

Save all of the graphs contained within this graphset into a compressed subdirectory representing the saved GraphSet.

Parameters
directory_path (*str*) – The path to the directory where the graphset subdirectory should be created.

2.6.18 gatoh.graphs.GraphSet.set_stochastic_rels

GraphSet.**set_stochastic_rels**(*hierarchy: str*, *status: bool*, *flags: tuple[bool, bool] = (False, False)*) → None

A setter function that defines whether stochastic relationships should be modelled for a specific hierarchy.

Parameters

- **hierarchy** (*str*) – The name of the hierarchy for which the status is being set.
- **status** (*bool*) – A flag indicating whether stochastic relationships should be modelled.
- **flags** (*tuple[bool, bool], optional*) – The flags that should be input to the graph's stochastic_relationships function.

2.6.19 gatoh.graphs.GraphSet.write_edge_pickle

GraphSet.**write_edge_pickle**(*graph_edge:* GraphEdge, *subdirectory_path: str*, *graph_name: str*, *idx: int*) → str

A helper function that allows for multithreading of `save_graphset()`.

Parameters

- **graph_edge** (`GraphEdge`) – The graph edge which is being pickled.

- **subdirectory_path** (*str*) – The subdirectory to which the pickled graph edge is being written to.
- **graph_name** (*str*) – The name of the graph that the edge belongs to.
- **idx** (*int*) – The index of the edge within its parent graph.

Returns
The path to which the edge pickle was saved to.

Return type
str

2.6.20 gatoh.graphs.GraphSet.write_node_pickle

GraphSet.**write_node_pickle**(*graph_node:* GraphNode, *subdirectory_path: str*, *graph_name: str*, *idx: int*) → str

A helper function that allows for multithreading of *save_graphset()*.

Parameters

- **graph_node** (GraphNode) – The graph node which is being pickled.
- **subdirectory_path** (*str*) – The subdirectory to which the pickled graph node is being written to.
- **graph_name** (*str*) – The name of the graph that the node belongs to.
- **idx** (*int*) – The index of the node within its parent graph.

Returns
The path to which the node pickle was saved to.

Return type
str

gatoh.graphs.GroupGraph(rw_params[, name, ...])	A class that defines a graph that will be used to collect and operate on Groups.

2.7 GroupGraph

class GroupGraph(*rw_params: tuple[float, float]*, *name: str = 'Group'*, *generation_method: str = ''*, *suppress_warnings: bool = False*, *dynamic_rels: bool = True*)

Bases: object

A class that defines a graph that will be used to collect and operate on Groups.

These Groups are generated through clustering of existing Agents, and as such, the operations on Groups are representative of aggregate operations on multiple Agents.

Parameters

- **rw_params** (*tuple[float, float]*) – The (mean, variance) of the normal distribution used for the dynamic relationships random walk.
- **name** (*str, optional*) – The unique name that will be used to identify this GroupGraph.
- **generation_method** (*str, optional*) – The random graph generation method that should be used where relevant.

- **suppress_warnings** (*bool, optional*) – A flag indicating if non-critical warnings should be suppressed.
- **dynamic_rels** (*bool, optional*) – A flag indicating if dynamic relationships should be modelled for groups.

Methods

add_edges	Creates appropriate GroupEdges from the given dictionary and then adds these to the graph.
add_nodes	Creates the appropriate GroupNodes from the given Groups, and then adds these to the graph.
calculate_polarisation	Calculates the level of opinion polarisation in this *GroupGraph* based on the equation:
change_generation_params	Setter function which outlines the existing generation parameters used in generate_graph() and allows the user to alter them.
change_weights	Updates the weight of the relationship between two groups in the graph.
cluster_nodes	A function that will use the betweenness centrality measure of the nodes in the graph to separate them into a specific number of clusters.
compute_subgraph_center	Calculates the center of a cluster using the betweenness centrality mapping.
dynamic_relationships	Uses the (mean, variance) passed at initialisation to draw random walk values by which each edge (relationship) in the group graph will be shifted.
estimate_neighbour_opinions	Return the individual opinion climate values perceived by the Group for each other Group in the group graph.
estimate_opinion_climate	Return the unique opinion climate perceived by the Group within the group graph.
generate_graph	Randomly generate edges between existing GroupGraph nodes and add them to the graph.
get_betweenness_centrality	A wrapper function that makes the rustworkx `digraph_betweenness_centrality()` function be a callable instance method for GroupGraph.
get_dijkstra_all_pairs	A wrapper function that makes the `digraph_all_pairs_dijkstra_path_lengths()` function be a callable instance method for GroupGraph.
get_edge	A getter function to access GroupEdge objects.
get_edge_changes	A getter function that returns the current register of pending edge changes for this graph, and then resets the attribute to an empty entry.
get_edges	A getter function to access multiple GroupEdge objects at once.
get_group_hierarchy	A getter function that reports what specific hierarchy a Group in the group graph belongs to.
get_group_index	Searches for the node index in the GroupGraph which corresponds to the input Group object.
get_groups_in_hierarchy	A getter function that returns all graph nodes with groups that belong to the specified hierarchy.

continues on next page

Table 14 – continued from previous page

get_neighbours	Finds all the nodes in the graph with direct relationships to the specified Group.
get_node	A getter function to access GroupNode objects.
get_nodes	A getter function to access multiple GroupNode objects at once.
get_relationship	Return a directed relationship from one node to another.
get_relationships	Retrieves and reports the bidirectional relationship weightings between two nodes in the GroupGraph.
group_in_graph	A simple function that checks whether a Group exists within a GroupGraph.
group_opinion_change	Change the specified Group's current aggregate opinion by the given delta.
group_previous_opinion	Set the specified Group's previous opinion to be equal to the current opinion (before the current opinion changes in the current iteration).
group_radicalisation_change	Change the specified Group's radicalisation rate by the given delta.
hierarchy_exists	A simple function that checks if at least one group in the graph belongs to the specified hierarchy.
load_graph	Loads a GroupGraph object stored in the GraphML format from the given path.
neighbour_influences	Looks at all the neighbours for a Group and uses the neighbours' own opinions plus the weight of the relationships between Groups to return a final value by which the given Group's opinion value will increment or decrement.
node_from_group	Returns the GroupNode object corresponding to the given Group object.
node_from_group_id	Returns the GroupNode object corresponding to the Group object with the given ID.
node_relationships_count	Report the number of ingoing and outgoing relationships that a specific node is involved in.
register_edge_change	A helper function that records a pending edge change for this graph.
relationship_exists	Checks for the existence of a relationship (weighted edge) between two Groups (nodes).
remove_edge	Removes a single edge from the graph corresponding to the indicated directed node indices.
remove_edge_index	Removes a single edge from the graph corresponding to the input edge index.
remove_node	Removes a node from the graph, along with any relationships involving it.
save_graph	Saves the existing GroupGraph object in the GraphML format to the given path.
step	Step the individual GroupGraph object:
stochastic_relationships	An experimental function which aims to simulate the possible formation and disintegration of relationships within a social network.
update_edge_indices	Iterates over all existing edges in the graph and updates their stored indices to reflect the current graph state.

continues on next page

Table 14 – continued from previous page

update_node_indices	Iterates over all existing nodes in the graph and updates their stored indices to reflect the current graph state.

2.7.1 gatoh.graphs.GroupGraph.add_edges

GroupGraph.**add_edges**(*edges: dict[str, list[Any]]*) → None

Creates appropriate GroupEdges from the given dictionary and then adds these to the graph.

The parameter `edges` has been typed as `Any` to simplify typechecking.

Parameters

edges (*dict[str, list[str | int | float | tuple[float, float]]]*) – A mapping of <key : list> where each key corresponds to (hierarchy <name>, from_node, to_node, [optional] weighting, [optional] rw_param).

2.7.2 gatoh.graphs.GroupGraph.add_nodes

GroupGraph.**add_nodes**(*groups: Iterable[*Group*]*) → None

Creates the appropriate GroupNodes from the given Groups, and then adds these to the graph.

Parameters

groups (*Iterable[*Group*]*) – The groups that will be converted to GroupNodes and added to the graph.

2.7.3 gatoh.graphs.GroupGraph.calculate_polarisation

GroupGraph.**calculate_polarisation**(*hierarchy: str = ''*) → float

Calculates the level of opinion polarisation in this *GroupGraph* based on the equation:

$$\pi(k) = \frac{1}{|K|(|K|-1)} \sum_{i \neq j}^{i \in K, j \in K} (d_{ij} - y)^2$$

where K is the set of groups within this GroupGraph, d_{ij} is the distance between the opinions of groups i and j :, and : $math$: 'y is the mean opinion distance among all groups in this GroupGraph.

Parameters

hierarchy (*str, optional*) – The specific hierarchy for which group polarisation should be calculated.

Returns

The measure of opinion radicalisation in this social network.

Return type

float

2.7.4 gatoh.graphs.GroupGraph.change_generation_params

GroupGraph.**change_generation_params**(***params: int | float*) → None

Setter function which outlines the existing generation parameters used in generate_graph() and allows the user to alter them.

Parameters

- **p** (*float, optional*) – The probability of edge rewiring (small-world) or edge creation (random).

- **m** (*int, optional*) – The number of nearest neighbours that each node is connected to initially (scale-free).
- **sbm_sizes** (*int, optional*) – The size of generated blocks (blockmodel).

Raises

UserWarning – If invalid parameters keys or data types are input to the function.

2.7.5 gatoh.graphs.GroupGraph.change_weights

GroupGraph.**change_weights**(*from_node: int*, *to_node: int*, *value: float*) → None

Updates the weight of the relationship between two groups in the graph.

If no relationship previously exists, a new one is created.

Parameters

- **from_node** (*int*) – The index of some Group in the graph.
- **to_node** (*int*) – The index of some other Group in the graph.
- **value** (*float*) – The new weight to assign.

2.7.6 gatoh.graphs.GroupGraph.cluster_nodes

GroupGraph.**cluster_nodes**(*k: int*, *n_iters: int = 40*) → dict[*GroupNode*, int]

A function that will use the betweenness centrality measure of the nodes in the graph to separate them into a specific number of clusters.

This function uses:

- KMeans as the main clustering algorithm
- Dijkstra all-pairs path lengths for the distances between nodes
- Betweenness centrality for the KMeans evaluation metric when determining centers

NOTE: This function is included for GroupGraphs for continuity in functionality from what is seen in base

Graphs, but its is not expected that further KMeans clustering on the already-clustered Groups will be of much use in typical use cases…

Parameters

- **k** (*int*) – The number of clusters to separate the graph into.
- **n_iters** (*int, optional*) – The maximum number of iterations to attempt running the KMeans algorithm for.

Returns

A <Group ID : Cluster ID> mapping that outlines the generated clusters for this graph.

Return type

dict[str, str]

2.7.7 gatoh.graphs.GroupGraph.compute_subgraph_center

GroupGraph.**compute_subgraph_center**(*subgraph_mapping: CentralityMapping*) → int

Calculates the center of a cluster using the betweenness centrality mapping.

Parameters
subgraph_mapping (*dict[int, float]*) – A <Node index : centrality measure> mapping containing the betweenness centrality for each node in the graph.

Returns
The index of the node with the greatest centrality (the center of the cluster).

Return type
int

2.7.8 gatoh.graphs.GroupGraph.dynamic_relationships

GroupGraph.**dynamic_relationships**() → None

Uses the (mean, variance) passed at initialisation to draw random walk values by which each edge (relationship) in the group graph will be shifted. Aims to simulate dynamic relationships between agents across timesteps at the aggregate scale.

2.7.9 gatoh.graphs.GroupGraph.estimate_neighbour_opinions

GroupGraph.**estimate_neighbour_opinions**(*group:* Group) → dict[str, float]

Return the individual opinion climate values perceived by the Group for each other Group in the group graph.

Parameters
group (Group) – The group which is estimating its neighbours' opinions.

Returns
A <neighbour id : estimated opinion value> mapping for each opinion perceived by the group.

Return type
dict[str, float]

2.7.10 gatoh.graphs.GroupGraph.estimate_opinion_climate

GroupGraph.**estimate_opinion_climate**(*group:* Group) → float

Return the unique opinion climate perceived by the Group within the group graph.

Parameters
group (Group) – The group which is estimating the opinion climate.

Returns
The group's perceived 'aggregated opinion' of the wholse social network.

Return type
float

2.7.11 gatoh.graphs.GroupGraph.generate_graph

GroupGraph.**generate_graph**(*groups: list[*Group*]*, *method: str = ''*, *relationship_range: tuple[float, float] = (-1.0, 1.0)*, *ensure_complete: bool = True*) → Self

Randomly generate edges between existing GroupGraph nodes and add them to the graph.

Parameters

- **groups** (*list[*Group*]*) – The Groups that are being used as the nodes for this graph.
- **method** (*str, optional*) – The random generation method to use. Possible choices include: 'small-world', 'scale-free', 'random', 'blockmodel'; default to 'small-world'.

- **relationship_range** (*tuple[float, float], optional*) – The valid range of generated relationship strengths (at most, constrained to [-1.0, 1.0]).
- **ensure_complete** (*bool, optional*) – A flag indicating if the generated graph should be complete or not (in the case of 'small-world').

Raises

- **ValueError** – If no groups are being passed to this function.
- **ValueError** – If an invalid random generation method is being passed to this function.

Returns
A reference to this GroupGraph object.

Return type
GroupGraph

2.7.12 gatoh.graphs.GroupGraph.get_betweenness_centrality

GroupGraph.**get_betweenness_centrality**(*subgraph: PyDiGraph[*GroupNode, GroupEdge*] | None = None*) → CentralityMapping

A wrapper function that makes the rustworkx `digraph_betweenness_centrality()` function be a callable instance method for GroupGraph.

Parameters
subgraph (PyDiGraph, optional) – A subgraph for which to calculate the betweenness centrality rather than the full graph.

Returns
A <Node index : centrality measure> mapping containing the betweenness centrality for each node in the graph.

Return type
dict[int, float]

2.7.13 gatoh.graphs.GroupGraph.get_dijkstra_all_pairs

GroupGraph.**get_dijkstra_all_pairs**(*subgraph: PyDiGraph[*GroupNode, GroupEdge*] | None = None*) → AllPairsPathLengthMapping

A wrapper function that makes the `digraph_all_pairs_dijkstra_path_lengths()` function be a callable instance method for GroupGraph.

Parameters
subgraph (PyDiGraph, optional) – A subgraph for which to calculate the dijkstra all pairs path lengths rather than the full graph.

Returns
A <Node index : <Node index : path length>> nested mapping outlining the path lengths of all pairs in the graph.

Return type
dict[int, dict[int, float]]

2.7.14 gatoh.graphs.GroupGraph.get_edge

GroupGraph.**get_edge**(*edge_index: int*) → *GroupEdge* | None

A getter function to access GroupEdge objects.

Parameters
edge_index (*int*) – The index of the edge to access.

Raises
RuntimeWarning – If the edge index is out of bounds for this GroupGraph.

Returns
The graph edge if the index was valid, or None otherwise.

Return type
GroupEdge | None

2.7.15 gatoh.graphs.GroupGraph.get_edge_changes

GroupGraph.**get_edge_changes**() → dict[str, *EdgeChanges*]

A getter function that returns the current register of pending edge changes for this graph, and then resets the attribute to an empty entry.

Returns
A <"ID1,ID2" : edge change> mapping that outlines the edge changes being made between two groups in the graph.

Return type
dict[str, *EdgeChanges*]

2.7.16 gatoh.graphs.GroupGraph.get_edges

GroupGraph.**get_edges**(*edge_indices: list[int]*) → list[*GroupEdge*]

A getter function to access multiple GroupEdge objects at once.

Parameters
edge_indices (*list[int]*) – The indices of the edges to access.

Raises
RuntimeWarning – If any of the edge indices are out of bounds.

Returns
A list containing all valid graph edges that were found.

Return type
list[*GroupEdge*]

2.7.17 gatoh.graphs.GroupGraph.get_group_hierarchy

GroupGraph.**get_group_hierarchy**(*group:* Group) → str

A getter function that reports what specific hierarchy a Group in the group graph belongs to.

Parameters
group (Group) – The group whose hierarchy membership is being examined.

Raises
ValueError – If the input Group does not exist in this graph.

Returns
The name of the hierarchy that the group belongs to.

Return type
str

2.7.18 gatoh.graphs.GroupGraph.get_group_index

GroupGraph.**get_group_index**(*group:* Group) → int | None

Searches for the node index in the GroupGraph which corresponds to the input Group object.

Parameters
group (Group) – The group whose index is being searched for.

Returns
The group's node index within the group graph if it exists, or None otherwise.

Return type
int | None

2.7.19 gatoh.graphs.GroupGraph.get_groups_in_hierarchy

GroupGraph.**get_groups_in_hierarchy**(*hierarchy: str*) → list[*GroupNode*]

A getter function that returns all graph nodes with groups that belong to the specified hierarchy.

Parameters
hierarchy (*str*) – The name of the hierarchy that groups must be members of.

Returns
A list of all the nodes with groups belonging to the specified hierarchy.

Return type
list[*GroupNode*]

2.7.20 gatoh.graphs.GroupGraph.get_neighbours

GroupGraph.**get_neighbours**(*group:* Group) → list[*GroupNode*] | None

Finds all the nodes in the graph with direct relationships to the specified Group.

Parameters
group (Group) – The group for which the neighbours are being examined.

Raises
UserWarning – If the input group does not exist in the graph.

Returns
A list of the graph nodes belonging to the direct neighbours of the group.

Return type
list[*GroupNode*]

2.7.21 gatoh.graphs.GroupGraph.get_node

GroupGraph.**get_node**(*node_index: int*) → *GroupNode* | None

A getter function to access GroupNode objects.

Parameters
node_index (*int*) – The index of the node to access.

Raises
RuntimeWarning – If the node index is out of bounds for this GroupGraph.

Returns
The graph node if the index was valid, or None otherwise.

Return type
GroupNode | None

2.7.22 gatoh.graphs.GroupGraph.get_nodes

GroupGraph.**get_nodes**(*node_indices: list[int]*) → list[*GroupNode*]

A getter function to access multiple GroupNode objects at once.

Parameters
node_indices (*list[int]*) – The indices of all the nodes to access.

Raises
RuntimeWarning – If any of the node indices is out of bounds.

Returns
A list containing all valid graph nodes that were found.

Return type
list[*GroupNode*]

2.7.23 gatoh.graphs.GroupGraph.get_relationship

GroupGraph.**get_relationship**(*from_node:* Group, *to_node:* Group) → float

Return a directed relationship from one node to another.

Parameters

- **from_node** (Group) – The node that the relationship originates from.
- **to_node** (Group) – The node that the relationship points to.

Returns
The weighting of the directed relationship (from_node -> to_node).

Return type
float

2.7.24 gatoh.graphs.GroupGraph.get_relationships

GroupGraph.**get_relationships**(*node_1: int*, *node_2: int*) → dict[tuple[int, int], float] | None

Retrieves and reports the bidirectional relationship weightings between two nodes in the GroupGraph.

Parameters

- **node_1** (*int*) – The node index of Group 1.
- **node_2** (*int*) – The node index of Group 2.

Returns
The bidirectional edge weightings between two nodes (if they exist).

Return type
dict[tuple[int, int], float] | None

2.7.25 gatoh.graphs.GroupGraph.group_in_graph

GroupGraph.**group_in_graph**(*group:* Group) → bool

A simple function that checks whether a Group exists within a GroupGraph.

Parameters
group (Group) – The group whose existence in the group graph is being checked for.

Returns
A flag indicating if the Group exists in the GroupGraph.

Return type
bool

2.7.26 gatoh.graphs.GroupGraph.group_opinion_change

GroupGraph.**group_opinion_change**(*group:* Group, *change_delta: float*, *deradicalisation: bool = False*) → float

Change the specified Group's current aggregate opinion by the given delta.

Parameters

- **group** (Group) – The group whose aggregate opinion is being changed.
- **change_delta** (*float*) – The value by which to change the Group's aggregate opinion.
- **deradicalisation** (*bool, optional*) – A flag indicating that this opinion change is ocurring alongside deradicalisation.

Returns
The per-agent delta value that must be applied for the overall group aggregate change to occur.

Return type
float

2.7.27 gatoh.graphs.GroupGraph.group_previous_opinion

GroupGraph.**group_previous_opinion**(*group:* Group) → None

Set the specified Group's previous opinion to be equal to the current opinion (before the current opinion changes in the current iteration).

Parameters
group (Group) – The group whose previous opinion is being set.

2.7.28 gatoh.graphs.GroupGraph.group_radicalisation_change

GroupGraph.**group_radicalisation_change**(*group:* Group, *change_delta: float*) → dict[str, bool]

Change the specified Group's radicalisation rate by the given delta.

Parameters

- **group** (Group) – The group whose radicalisation rate is being changed.
- **change_delta** (*float*) – The value by which to change the Group's radicalisation rate.

Returns
A <Agent ID : radicalisation flag> mapping that outlines the new radicalisation status for each member agent.

Return type
dict[str, bool]

2.7.29 gatoh.graphs.GroupGraph.hierarchy_exists

GroupGraph.**hierarchy_exists**(*hierarchy: str*) → bool

A simple function that checks if at least one group in the graph belongs to the specified hierarchy.

Parameters
hierarchy (*str*) – The name of the hierarchy being searched for.

Returns
A flag indicating if the specified hierarchy exists in the graph.

Return type
bool

2.7.30 gatoh.graphs.GroupGraph.load_graph

GroupGraph.**load_graph**(*path: str*, *name: str = 'Group'*, *rw_params: tuple[float, float] | None = None*) → None

Loads a GroupGraph object stored in the GraphML format from the given path.

Parameters

- **path** (*str*) – Path to a stored group graph file.
- **name** (*str, optional*) – The name that is being used to uniquely identify the group graph.
- **rw_params** (*tuple[float, float], optional*) – The mean and variance of the GroupGraph's random walk distribution.

2.7.31 gatoh.graphs.GroupGraph.neighbour_influences

GroupGraph.**neighbour_influences**(*group:* Group) → float | None

Looks at all the neighbours for a Group and uses the neighbours' own opinions plus the weight of the relationships between Groups to return a final value by which the given Group's opinion value will increment or decrement.

Parameters
group (Group) – The group for which the strength of opinion change is being determined.

Raises
UserWarning – If the input group does not exist in the group graph.

Returns
The final change in the Group's aggregate opinion caused by its neighbours in the group graph.

Return type
float

2.7.32 gatoh.graphs.GroupGraph.node_from_group

GroupGraph.**node_from_group**(*group:* Group) → *GroupNode* | None

Returns the GroupNode object corresponding to the given Group object.

Parameters
group (Group) – The group being search for in the GroupNodes.

Returns
The group node corresponding to the input Group if it exists, or None otherwise.

Return type
GroupNode | None

2.7.33 gatoh.graphs.GroupGraph.node_from_group_id

GroupGraph.**node_from_group_id**(*group_id: str*) → *GroupNode* | None

Returns the GroupNode object corresponding to the Group object with the given ID.

Parameters
group_id (*str*) – The ID of the Group being searched for in the GroupNodes.

Returns
The group node corresponding to the input ID if it exists, or None otherwise.

Return type
GroupNode | None

2.7.34 gatoh.graphs.GroupGraph.node_relationships_count

GroupGraph.**node_relationships_count**(*node_index: int*) → int

Report the number of ingoing and outgoing relationships that a specific node is involved in.

Parameters
node_index (*int*) – The index of the node that is being inspected.

Raises
IndexError – If the input node index is out of bounds.

Returns
The total number of relationships involving the input node.

Return type
int

2.7.35 gatoh.graphs.GroupGraph.register_edge_change

GroupGraph.**register_edge_change**(*from_node: int*, *to_node: int*, *weighting: float*) → None

A helper function that records a pending edge change for this graph.

Parameters

- **from_node** (*int*) – The index of the origin node.
- **to_node** (*int*) – The index of the destination node.
- **weighting** (*float*) – The new weighting that is being assigned to the edge.

2.7.36 gatoh.graphs.GroupGraph.relationship_exists

GroupGraph.**relationship_exists**(*from_node: int*, *to_node: int*) → int | None

Checks for the existence of a relationship (weighted edge) between two Groups (nodes).

Parameters

- **from_node** (*int*) – The node index of the parent node.
- **to_node** (*int*) – The node index of the child node.

Returns
The index of the edge if the relationship exists, or None otherwise.

Return type
int | None

2.7.37 gatoh.graphs.GroupGraph.remove_edge

GroupGraph.**remove_edge**(*from_node: int*, *to_node: int*) → None

Removes a single edge from the graph corresponding to the indicated directed node indices.

Throws a warning without interrupting the runtime if the edge didn't exist in the first place.

Parameters

- **from_node** (*int*) – The parent node in the edge.
- **to_node** (*int*) – The child node in the edge.

Raises

UserWarning – If the edge (from_node -> to_node) doesn't exist in the GroupGraph.

2.7.38 gatoh.graphs.GroupGraph.remove_edge_index

GroupGraph.**remove_edge_index**(*edge_index: int*) → None

Removes a single edge from the graph corresponding to the input edge index.

Raises an error if the edge index is out of bounds of existing edges.

This function is meant as an extended wrapper to rustworkx `remove_edge_from_index()` that also handles cases where nodes are left neighbourless after the edge removal.

Parameters

edge_index (*int*) – The index of the edge to remove.

Raises

KeyError – If the input edge index is out of bounds.

2.7.39 gatoh.graphs.GroupGraph.remove_node

GroupGraph.**remove_node**(*node: int*) → None

Removes a node from the graph, along with any relationships involving it.

Parameters

node (*int*) – The node index to remove from the graph.

Raises

IndexError – If the node index is out of bounds.

2.7.40 gatoh.graphs.GroupGraph.save_graph

GroupGraph.**save_graph**(*path: str*) → None

Saves the existing GroupGraph object in the GraphML format to the given path.

Parameters

path (*str*) – Path to which the Graph will be saved.

Raises

OSError – If the graph is not successfully written.

2.7.41 gatoh.graphs.GroupGraph.step

GroupGraph.**step**() → None

Step the individual GroupGraph object:

1. Handle dynamic relationships within the graph.

2.7.42 gatoh.graphs.GroupGraph.stochastic_relationships

GroupGraph.**stochastic_relationships**(*formation: bool = False*, *formation_thresh: float = 0.999*, *formation_num: int = 1*, *disintegration: bool = False*, *disintegration_thresh: float = 0.999*, *disintegration_num: int = 1*) → None

An experimental function which aims to simulate the possible formation and disintegration of relationships within a social network.

Parameters

- **formation** (*bool, optional*) – A flag indicating if the spontaneous formation of new relationships should be allowed.
- **formation_thresh** (*float, optional*) – The threshold that must be surpassed for spontaneous formation to occur.
- **formation_num** (*int, optional*) – The number of new relationships to form.
- **disintegration** (*bool, optional*) – A flag indicating if the spontaneous disintegration of existing relationships should be allowed.
- **disintegration_thresh** (*float, optional*) – The threshold that must be surpassed for spontaneous disintegration to occur.
- **disintegration_num** (*int, optional*) – The number of existing relationships to disintegrate.

2.7.43 gatoh.graphs.GroupGraph.update_edge_indices

GroupGraph.**update_edge_indices**() → None

Iterates over all existing edges in the graph and updates their stored indices to reflect the current graph state.

Will also update the graph edge_count attribute.

2.7.44 gatoh.graphs.GroupGraph.update_node_indices

GroupGraph.**update_node_indices**() → None

Iterates over all existing nodes in the graph and updates their stored indices to reflect the current graph state.

Will also update the graph node_count attribute.

gatoh.graphs.GroupNode(group)	A helper class that allows rustworkx to more efficiently store information about Groups in the graph nodes.

2.8 GroupNode

class GroupNode(*group:* Group)

Bases: object

A helper class that allows rustworkx to more efficiently store information about Groups in the graph nodes.

Parameters

group (Group) – The group that is being associated with this GroupNode.

Methods

get_hierarchy	A getter method that returns the hierarchy that the contained group belongs to.
get_num_members	A wrapper for the group's *get_num_members()* method.
set_index	A setter method to set the GroupNode's index value.

2.8.1 gatoh.graphs.GroupNode.get_hierarchy

GroupNode.**get_hierarchy**() → str

A getter method that returns the hierarchy that the contained group belongs to.

Returns
The hierarchy that the node's contained group belongs to.

Return type
str

2.8.2 gatoh.graphs.GroupNode.get_num_members

GroupNode.**get_num_members**() → int

A wrapper for the group's *get_num_members()* method.

Returns
The number of members that make up the node's contained group.

Return type
int

2.8.3 gatoh.graphs.GroupNode.set_index

GroupNode.**set_index**(*idx: int*) → None

A setter method to set the GroupNode's index value.

Parameters
idx (*int*) – The index to set for this GroupNode.

gatoh.graphs.GroupEdge(hierarchy, from_node, ...)	A helper class that allows rustworkx to more efficiently store information about Group relationships in the graph edges.

2.9 GroupEdge

class GroupEdge(*hierarchy: str*, *from_node: int*, *to_node: int*, *weighting: float = 0.0*, *rw_params: tuple[float, float] | None = None*)

Bases: object

A helper class that allows rustworkx to more efficiently store information about Group relationships in the graph edges. As the social hierarchies are assumed to be DiGraphs, each GroupEdge is directional, and the aggregate weighting that the Agents in group A place on those in group B will not necessarily be equally reciprocated.

Parameters

- **hierarchy** (*str*) – The name of the social hierarchy that this edge belongs to.
- **from_node** (*int*) – The index of the origin node.
- **to_node** (*int*) – The index of the destination node.
- **weighting** (*float, optional*) – The aggregate relationship opinion value that is being assigned (range [-1, 1]).
- **rw_params** (*tuple[float, float], optional*) – The normal distribution (mean, variance) to assign for relationship-specific random walk effects.

Methods

has_rw_params	A function that checks if this relationship has explicit random walk parameters.
set_index	A setter function that changes this GroupEdge's index value.
set_rw_params	A setter function that changes this GroupEdge's rw_params value.
set_weighting	A setter function that changes this GroupEdge's weighting value.
update_from_node	A setter function that updates the from_node's index for this GroupEdge.
update_to_node	A setter function that updates the to_node's index for this GroupEdge.

2.9.1 gatoh.graphs.GroupEdge.has_rw_params

GroupEdge.**has_rw_params**() → bool

A function that checks if this relationship has explicit random walk parameters.

Returns
A flag indicating if random walk parameters exist.

Return type
bool

2.9.2 gatoh.graphs.GroupEdge.set_index

GroupEdge.**set_index**(*idx: int*) → None

A setter function that changes this GroupEdge's index value.

Parameters
idx (*int*) – The index to store for this GroupEdge.

2.9.3 gatoh.graphs.GroupEdge.set_rw_params

GroupEdge.**set_rw_params**(*rw_params: tuple[float, float]*) → None

A setter function that changes this GroupEdge's rw_params value.

Parameters
rw_params (*tuple[float, float]*) – A (mean, variance) tuple specifying this relationship's unique random walk distribution.

2.9.4 gatoh.graphs.GroupEdge.set_weighting

GroupEdge.**set_weighting**(*value: float*) → None

A setter function that changes this GroupEdge's weighting value.

Parameters
value (*float*) – The new aggregate weighting to store for this GroupEdge.

2.9.5 gatoh.graphs.GroupEdge.update_from_node

GroupEdge.**update_from_node**(*idx: int*) → None

A setter function that updates the from_node's index for this GroupEdge.

Parameters
idx (*int*) – The from_node's new index value to update to.

2.9.6 gatoh.graphs.GroupEdge.update_to_node

GroupEdge.**update_to_node**(*idx: int*) → None

A setter function that updates the to_node's index for this GroupEdge.

Parameters
idx (*int*) – The to_node's new index value to update to.

gatoh.graphs.GroupGraphSet([graph, ...])	A class that mainly serves as a way of providing utility functions for the singular group graph that would exist in the ABModel.

2.10 GroupGraphSet

class GroupGraphSet(*graph:* GroupGraph | *None* = *None*, *stochastic_relationships: dict[str, bool]* | *None* = *None*, *stochastic_rels_flags: dict[str, tuple[bool, bool]]* | *None* = *None*)

Bases: `object`

A class that mainly serves as a way of providing utility functions for the singular group graph that would exist in the ABModel.

Parameters

- **graph** (GroupGraph, *optional*) – The existing GroupGraph object that should be added to the GroupGraphSet.
- **stochastic_relationships** (*dict[str, bool], optional*) – A <graph : flag> mapping indicating if stochastic formation and disintegration of relationships should be modelled.
- **stochastic_rels_flags** (*dict[str, tuple[bool, bool]], optional*) – A <graph : flag> mapping providing the necessary stochastic_relationships flags per-graph.

Methods

add_graph	A setter function to add a new GroupGraph object to the GroupGraphSet.

continues on next page

Table 20 – continued from previous page

calculate_polarisation	A wrapper that calls the group graph's calculate_polarisation function and returns its value.
get_graph	A getter function to return the GroupGraph object stored by this GroupGraphSet.
get_group_hierarchy	A helper function that determines which social hierarchy a Group belongs to.
get_groups_hierarchies	A helper function that determines which social hierarchy multiple Groups are contained in.
group_graph_exists	A simple function that reports whether the group graph set's group graph exists or not.
list_hierarchies	A utility function that iterates over the groups in the group graph and prints out the names of all the social hierarchies that are present.
load_edge	A helper function that allows for multithreading within `load_graphset_multi()`.
load_graphset	Loads a GroupGraphSet that has been saved following the same process as in the save_graphset() function.
load_node	A helper function that allows for multithreading within `load_graphset_multi()`.
save_graphset	Save the graph contained within this graphset into a compressed subdirectory.
set_stochastic_rels	A setter function that defines whether stochastic relationships should be modelled for a specific hierarchy.
write_edge_pickle	A helper function that allows multithreading of *save_graphset()*.
write_node_pickle	A helper function that allows multithreading of `save_graphset()`.

2.10.1 gatoh.graphs.GroupGraphSet.add_graph

GroupGraphSet.**add_graph**(*graph:* GroupGraph) → None

A setter function to add a new GroupGraph object to the GroupGraphSet.

Parameters
graph (GroupGraph) – The group graph to add to the GroupGraphSet.

2.10.2 gatoh.graphs.GroupGraphSet.calculate_polarisation

GroupGraphSet.**calculate_polarisation**(*hierarchy: str = ''*) → float

A wrapper that calls the group graph's calculate_polarisation function and returns its value.

An empty string ("") will mean that the polarisation for all groups will be calculated, regardless of which hierarchy they belong to.

Parameters
hierarchy (*str, optional*) – The hierarchy for which polarisation is being calculated.

Raises

- **RuntimeError** – If no valid group graph has been set.
- **ValueError** – If no groups in the group graph belong to the input hierarchy.

Returns
The hierarchy polarisation value.

Return type
float

2.10.3 gatoh.graphs.GroupGraphSet.get_graph

GroupGraphSet.**get_graph**() → *GroupGraph* | None

A getter function to return the GroupGraph object stored by this GroupGraphSet.

Returns
The graph object to return if it has been set, or None otherwise.

Return type
GroupGraph | None

2.10.4 gatoh.graphs.GroupGraphSet.get_group_hierarchy

GroupGraphSet.**get_group_hierarchy**(*group:* Group) → str

A helper function that determines which social hierarchy a Group belongs to.

Parameters
group (Group) – The group for which hierarchy membership is being determined.

Raises
ValueError – If the input group does not exist in the group graph.

Returns
The name of the social hierarchy to which the input Group belongs in.

Return type
str

2.10.5 gatoh.graphs.GroupGraphSet.get_groups_hierarchies

GroupGraphSet.**get_groups_hierarchies**(*groups: list[*Group*]*) → dict[str, str]

A helper function that determines which social hierarchy multiple Groups are contained in.

Parameters
groups – The groups for which hierarchy membership is being determined.

Returns
A <Group ID : hierarchy> mapping of the social hierarchy that each Group belongs to.

Return type
dict[str, str]

2.10.6 gatoh.graphs.GroupGraphSet.group_graph_exists

GroupGraphSet.**group_graph_exists**() → bool

A simple function that reports whether the group graph set's group graph exists or not.

Returns
A flag indicating if the group_graph exists as a valid GroupGraph object

Return type
bool

2.10.7 gatoh.graphs.GroupGraphSet.list_hierarchies

GroupGraphSet.**list_hierarchies**(*print_out: bool = False*) → list[str]

A utility function that iterates over the groups in the group graph and prints out the names of all the social hierarchies that are present.

Parameters
print_out (*bool, optional*) – A flag indicating if the listed hierarchies should be printed out to the terminal.

Returns
The names of all social hierarchies present amongst the groups the group graph.

Return type
list[str]

2.10.8 gatoh.graphs.GroupGraphSet.load_edge

GroupGraphSet.**load_edge**(*edge_dir: str*, *edge_file: str*) → tuple[*GroupEdge*, int]

A helper function that allows for multithreading within `load_graphset_multi()`.

Parameters

- **edge_dir** (*str*) – The root directory where all graph edges have been saved.
- **edge_file** (*str*) – The name of the pickle file that is being loaded.

Returns
The unpickled group edge object and its corresponding index in the graph.

Return type
tuple[*GroupEdge*, int]

2.10.9 gatoh.graphs.GroupGraphSet.load_graphset

GroupGraphSet.**load_graphset**(*load_path: str*, *rw_params: tuple[float, float]*) → None

Loads a GroupGraphSet that has been saved following the same process as in the save_graphset() function.

Parameters

- **load_path** (*str*) – The path to the model's overall save directory.
- **rw_params** (*tuple[float, float]*) – The (mean, variance) of the normal distribution used to draw random-walk values for the graph.

Raises
FileNotFoundError – If the input load_path does not point to a valid file.

2.10.10 gatoh.graphs.GroupGraphSet.load_node

GroupGraphSet.**load_node**(*node_dir: str*, *node_file: str*) → tuple[*GroupNode*, int]

A helper function that allows for multithreading within `load_graphset_multi()`.

Parameters

- **node_dir** (*str*) – The root directory where all graph nodes have been saved.
- **node_file** (*str*) – The name of the pickle file that is being loaded.

Returns
The unpickled group node object and its corresponding index in the graph.

Return type
tuple[*GroupNode*, int]

2.10.11 gatoh.graphs.GroupGraphSet.save_graphset

GroupGraphSet.**save_graphset**(*directory_path: str*) → None

Save the graph contained within this graphset into a compressed subdirectory.

Parameters
directory_path (*str*) – The path to the directory where the graphset subdirectory should be created.

2.10.12 gatoh.graphs.GroupGraphSet.set_stochastic_rels

GroupGraphSet.**set_stochastic_rels**(*name: str*, *status: bool*, *flags: tuple[bool, bool] = (False, False)*) → None

A setter function that defines whether stochastic relationships should be modelled for a specific hierarchy.

Parameters

- **name** (*str*) – The name of the group graph for which the status is being set.
- **status** (*bool*) – A flag indicating whether stochastic relationships should be modelled.
- **flags** (*tuple[bool, bool], optional*) – The flags that should be input to the graph's stochastic_relationships function.

2.10.13 gatoh.graphs.GroupGraphSet.write_edge_pickle

GroupGraphSet.**write_edge_pickle**(*graph_edge:* GroupEdge, *subdirectory_path: str*, *graph_name: str*, *idx: int*) → str

A helper function that allows multithreading of *save_graphset()*.

Parameters

- **graph_edge** (GroupEdge) – The graph edge which is being pickled.
- **subdirectory_path** (*str*) – The subdirectory to which the pickled graph edge is being written to.
- **graph_name** (*str*) – The unique name used to identify the parent graph.
- **idx** (*int*) – The index of the edge within its parent graph.

Returns
The path to which the edge pickle was saved to.

Return type
str

2.10.14 gatoh.graphs.GroupGraphSet.write_node_pickle

GroupGraphSet.**write_node_pickle**(*graph_node:* GroupNode, *subdirectory_path: str*, *graph_name: str*, *idx: int*) → str

A helper function that allows multithreading of save_graphset().

Parameters

- **graph_node** (GroupNode) – The graph node which is being pickled.

- **subdirectory_path** (*str*) – The subdirectory to which the pickled graph node is being written to.
- **graph_name** (*str*) – The unique name used to identify the parent graph.
- **idx** (*int*) – The index of the node within its parent graph.

Returns
The path to which the node pickle was saved to.

Return type
str

gatoh.groups.Group(*args, **kwargs)	A class to define Groups of Agent objects that can serve as a way of clustering hierarchies, and providing a pseudo-vectorisation of model operations across agents.

2.11 Group

class Group(**args: T*, ***kwargs: T*)

Bases: `object`

A class to define Groups of Agent objects that can serve as a way of clustering hierarchies, and providing a pseudo-vectorisation of model operations across agents.

Supported positional arguments:

- <string> to set the Group's id.
- <int> to set the Group's maximum size.

Parameters

- **id** (*str, optional*) – Positional argument – provides a unique identifier for a group.
- **max_size** (*int, optional*) – Positional argument – the maximum number of Agents that can be members of this group.
- **index** (*int, optional*) – Keyword argument – the group's index within a GroupSet.
- **hierarchy** (*str, optional*) – Keyword argument – the social hierarchy that this group is being formed in.
- **members** (*list[str], optional*) – Keyword argument – the IDs of the agents that are members of this group.
- **aggregate_opinion** (*float, optional*) – Keyword argument – the aggregate opinion held by members of the group.
- **aggregate_susceptibility** (*float, optional*) – Keyword argument – the aggregate social susceptibility of group members.
- **cohesion** (*str, optional*) – Keyword argument – the level of cohesion that the group will act with.
- **radicalisation_rate** (*float, optional*) – Keyword argument – The proportion of agents in the group which are radicalised.
- **predominant_personality** (*str, optional*) – Keyword argument – The most common personality type among members in the group.

- **member_benefit_rate** (*float, optional*) – Keyword argument – The proportion of agents in the groups which are personally benefitted from the social contagion.
- **previous_opinion** (*float, optional*) – Keyword argument – The aggregate opinion held by the group at the immediate past iteration.
- **aggregate_hierarchy_weighting** (*float, optional*) – Keyword argument – The aggregate weighting that the members of the group give to the hierarchy they are in.

Methods

add_attribute	Dynamically add an attribute to this Group object.
add_member	Add a new member to this group (using the Agent's ID as the identifier).
add_members	Add multiple new members to this group (using the Agent IDs as the identifiers).
apply_update	A multi-setter function that simply updates all aggregate attributes with input ones that have been recalculated.
change_aggregate_hierarchy_weighting	A setter method that changes the Group's aggregate hierarchy weighting by a given delta value.
change_aggregate_opinion	A setter method that changes the Group's aggregate opinion by a given delta value.
change_aggregate_susceptibility	A setter method that changes the Group's aggregate social susceptibility by a given delta value.
change_benefit_rate	A setter method that changes the group's member_benefit_rate by a given delta value.
change_opinion_rw	A setter method that changes the group's explicit opinion random walk parameters.
change_predominant_personality	A setter method that changes the group's predominant personality type.
change_radicalisation_rate	A setter method that changes the Group's radicalisation rate by a given delta value.
change_rw_distribution	A setter method that change the group's explicit random walk parameters for its social hierarchy.
change_silencing_rate	A setter method that changes the Group's silencing rate by a given delta value.
deradicalisation	Uses the group's own aggregate opinion as well as the neighbours' opinions to determine if an already radicalised group will deradicalise.
determine_predominant_personality	A setter method that will determine what the predominant personality type amongst group members is.
evolve_hierarchy	Experimental function that aims to model the constantly evolving 'intrinsic value' that people palce on the social hierarchies that the y belong in over time.
generate_group	Generate a Group object based on the input parameters.
get_attribute	Return any existing or dynamically added attribute held by the Group object.
get_num_members	A getter method that reports the number of members that are contained in this Group.
group_at_capacity	A function that reports if the group is at its maximum capacity.

continues on next page

Table 22 – continued from previous page

is_benefited	A function that reports if the group is considered to experience personal benefit from the social contagion as a collective.
is_radicalised	A function that reports if the group is considered to be "radicalised" as a collective.
is_silenced	A function that reports if the group is considered to be silenced in its hierarchy as a collective.
life_events	Experimental function that aims to model the ways in which behaviours change according to major random life events over time.
opinion_negation	Checks if a group has experienced sufficiently 'overwhelming' social pressure in its hierarchy leading to a complete reversal of its members' opinions.
opinion_silencing	Determines if members in the group will become silenced in their hierarchy based on their collective attributes.
radicalisation	Uses the group's own aggregate opinion as well as the neighbours' opinions to determine if a non-radical group will become radicalised.
recalculate_aggregate_opinion	A setter method that will calculate a fixed aggregate opinion value from the currently held opinion values of members.
recalculate_aggregate_susceptibility	A setter method that will calculate a fixed aggregate social susceptibility value from the currently held opinion values of members.
recalculate_hierarchy_weighting	A setter method that will calculate a fixed member hierarchy weighting value from the current hierarchy weightings assigned by each group member.
recalculate_member_benefit_rate	A setter method that will calculate a fixed member benefit rate value from the current personal benefit statuses of group members.
recalculate_radicalisation_rate	A setter method that will calculate a fixed radicalisation rate value from the current radicalisation statuses of group members.
recalculate_silencing_rate	A setter method that will calculate a fixed silencing rate value from the current opinion silencing statuses of group members.
set_cohesion	A setter function to change the group's cohesion type.
set_hierarchy	A setter function to change the hierarchy that the Group belongs to.
set_index	A setter function to change the Group's index.
set_max_size	A setter function to change the maximum size of a Group.
step	Step the individual group object:
stochastic_benefit_change	Calls to this function are primarily meant to originate from *life_events()*.
stochastic_opinion	Determine the direction and magnitude of a shift in the Group's aggregate opinion and apply it.
stochastic_personality_change	Calls to this function are primarily meant to originate from *life_events()*.
stochastic_radicalisation_change	Calls to this function are primarily meant to originate from *life_events()*.

continues on next page

Table 22 – continued from previous page

stochastic_silencing_change	Calls to this function are primarily meant to originate from *life_events()*.
store_previous_opinion	A setter method that stores the Group's current opinion into the previous opinion.
update	Updates the internal state of the group after the model has stepped by recalculating all aggregate attributes to reflect any additional effects from members which may belong to multiple groups.

2.11.1 gatoh.groups.Group.add_attribute

Group.**add_attribute**(*name: str*, *value: T | dict[S, T] | tuple[T, ...] | list[T] | None = None*, *parameters: dict[str, float] | None = None*, *distribution: str | None = None*, *overwrite: bool = True*) → None

Dynamically add an attribute to this Group object. If "value" is passed, an explicit initial value is given; if "mean" and "var" are passed through parameters, a value is generated from a random distribution. Supported random distributions are:

- "gaussian"
- "beta"
- "gamma"
- "uniform"
- "levy"

Either "value", or parameters with "mean" and "var" included should be passed to this function, not both.

If both an explicit value and distribution parameters are input to this function, the explicit value will always override the distribution parameters when setting the attribute.

Parameters

- **name** (*str*) – The name of the attribute to be added.
- **value** (*Any, optional*) – Initial value of the attribute.
- **parameters** (*dict[str, float], optional*) – The distribution parameters that will be used with the specified distribution for parameter generation.
- **distribution** (*str, optional*) – The random distribution that will be used to generate the value.
- **overwrite** (*bool, optional*) – A flag indicating if the added attribute should override any existing attributes of the same name.

Raises

- **ValueError** – If no valid value or distribution parameters are input, the attribute cannot be added.
- **UserWarning** – If overwrite is explicitly False but the attribute is existing, a warning is raised without completing the operation.

2.11.2 gatoh.groups.Group.add_member

Group.**add_member**(*member: str*, *force: bool = False*) → None

Add a new member to this group (using the Agent's ID as the identifier).

Parameters

- **member** (*str*) – The ID of the Agent being added as a group member.
- **force** (*bool, optional*) – A flag indicating if this call should disregard the maximum group size.

Raises

- **TypeError** – If the input value is not a string.
- **UserWarning** – If the group is already at maximum capacity

2.11.3 gatoh.groups.Group.add_members

Group.**add_members**(*members: Iterable[str]*, *force: bool = False*) → None

Add multiple new members to this group (using the Agent IDs as the identifiers).

Parameters

- **members** (*Iterable[str]*) – An iterable containing the Agent IDs of the members to be added.
- **force** (*bool, optional*) – A flag indicating if this call should disregard the maximum group size.

Raises

- **TypeError** – If any of the input members are not string values.
- **UserWarning** – If the group is already at maximum capacity.

2.11.4 gatoh.groups.Group.apply_update

Group.**apply_update**(*new_attributes: GroupUpdateDict*) → None

A multi-setter function that simply updates all aggregate attributes with input ones that have been recalculated.

Parameters

new_attributes (*dict[str, float]*) – The group's recalculated aggregate attributes.

2.11.5 gatoh.groups.Group.change_aggregate_hierarchy_weighting

Group.**change_aggregate_hierarchy_weighting**(*weighting_delta: float*) → float

A setter method that changes the Group's aggregate hierarchy weighting by a given delta value.

This will not cause changes to the weightings of member agents to create the desired aggregate hierarchy weighting – This should be handled from within the parent model by using the returned per-agent weighting delta.

Parameters

weighting_delta (*float*) – The value by which to shift the aggregate hierarchy weighting.

Raises

- **TypeError** – If the weighting delta is not a float.

- **AttributeError** – If the aggregate_hierarchy_weighting has not yet been initialised.

Returns
The per-agent weighting delta that must be applied to each member.

Return type
float

2.11.6 gatoh.groups.Group.change_aggregate_opinion

Group.**change_aggregate_opinion**(*opinion_delta: float*) → float

A setter method that changes the Group's aggregate opinion by a given delta value.

This will not cause changes to the opinions of member agents to create the desired aggregate opinion – It should be handled from within the parent model by using the returned per-agent opinion delta value.

Parameters
opinion_delta (*float*) – The delta value by which to shift the Group's aggregate opinion.

Raises

- **TypeError** – If opinion_delta is not a float.
- **AttributeError** – If the aggregate_opinion has not been initialised yet.

Returns
The per-agent opinion_delta value that must be applied to each member.

Return type
float

2.11.7 gatoh.groups.Group.change_aggregate_susceptibility

Group.**change_aggregate_susceptibility**(*delta_value: float*) → float

A setter method that changes the Group's aggregate social susceptibility by a given delta value.

This will not cause changes to the opinions of member agents to create the desired aggregate susceptibility – It should be handled from within the parent model by using the returned per-agent susceptibility delta value.

Parameters
delta_value (*float*) – The delta value by which to shift the Group's aggregate susceptibility.

Raises

- **TypeError** – If delta_value is not a float.
- **AttributeError** – If the aggregate_susceptibility has not been initialised yet.

Returns
The per-agent delta_value that must be applied to each social susceptibility.

Return type
float

2.11.8 gatoh.groups.Group.change_benefit_rate

Group.**change_benefit_rate**(*rate_delta: float*) → dict[str, bool]

A setter method that changes the group's member_benefit_rate by a given delta value.

This will not cause changes to the personal benefit status of member agents to create the desired benefit rate – This should be handled from within the parent model by using the returned per-agent personal benefit mapping.

Parameters
rate_delta (*float*) – The value by which to shift the group's member_benefit_rate.

Raises
- **AttributeError** – If the member benefit rate has not yet been initialised for this group.
- **TypeError** – If the rate delta is not a float

Returns
A <Member ID : personal benefit flag> mapping that defines each member agent's new personal benefit status.

Return type
dict[str, bool]

2.11.9 gatoh.groups.Group.change_opinion_rw

Group.**change_opinion_rw**(*rw_params: tuple[float, float]*) → None

A setter method that changes the group's explicit opinion random walk parameters.

Parameters
rw_params (*tuple[float, float]*) – The new (mean, variance) for the random walk's gaussian distribution.

Raises
TypeError – If rw_params is not a (float, float) tuple.

2.11.10 gatoh.groups.Group.change_predominant_personality

Group.**change_predominant_personality**(*personality: str*) → dict[str, str]

A setter method that changes the group's predominant personality type.

This will not cause changes to the personalities of member agents to create the desired predominant type – This should be handled from within the parent model by using the returned per-agent personality mapping.

Parameters
personality (*str*) – The personality type that should be the new predominant personality.

Raises
- **TypeError** – If personality is not a string.
- **ValueError** – If the personality type is not currently supported.

Returns
A <Member ID : personality type> mapping reporting what each member's new personality type should be.

Return type
dict[str, str]

2.11.11 gatoh.groups.Group.change_radicalisation_rate

Group.**change_radicalisation_rate**(*rate_delta: float*) → dict[str, bool]

A setter method that changes the Group's radicalisation rate by a given delta value.

This will not cause changes to the radicalisation status of member agents – This should be handled from within the parent model by using the returned radicalisation status mapping.

Parameters
rate_delta (*float*) – The value by which to shift the radicalisation rate of the group.

Raises

- **TypeError** – If the rate delta is not a float.
- **AttributeError** – If the radicalisation_rate has not yet been initialised.

Returns
A <Member ID : radicalisation status> mapping that reports the new radicalisation status for each member.

Return type
dict[str, bool]

2.11.12 gatoh.groups.Group.change_rw_distribution

Group.**change_rw_distribution**(*parameters: tuple[float, float]*) → None

A setter method that change the group's explicit random walk parameters for its social hierarchy.

Parameters
parameters (*tuple[float, float]*) – The new (mean, variance) for the random walk's gaussian distribution.

Raises
TypeError – If the input contains any invalid data types.

2.11.13 gatoh.groups.Group.change_silencing_rate

Group.**change_silencing_rate**(*rate_delta: float*) → dict[str, bool]

A setter method that changes the Group's silencing rate by a given delta value.

This will not cause changes to the silencing status of member agents for the relevant hierarchy to reach the desired silencing rate – This should be handled from within the parent model by using the returned per-agent is_silenced mapping.

Parameters
rate_delta (*float*) – The value by which to shift the silencing_rate.

Raises

- **TypeError** – If the rate delta is not a float.
- **AttributeError** – If the silencing_rate has not yet been initialised.

Returns
A <Member ID : is_silenced flag> mapping that specifies which agents are silenced in this group's hierarchy.

Return type
dict[str, bool]

2.11.14 gatoh.groups.Group.deradicalisation

Group.**deradicalisation**(*neighbour_influences: float*, *neighbour_benefits: list[bool]*, *threshold: float | None = None*) → tuple[bool, float]

Uses the group's own aggregate opinion as well as the neighbours' opinions to determine if an already radicalised group will deradicalise.

Parameters

- **neighbour_influences** (*float*) – The total influence that the group's neighbouring opinions have had on it this iteration.
- **neighbour_benefits** (*list[bool]*) – Flags indicating the presence of personal benefit across a group's neighbours.
- **threshold** (*float, optional*) – The deradicalisation threshold that has been defined at the global level in the model.

Raises

- **RuntimeError** – If the group has not yet been initialised appropriately.
- **TypeError** – If any of the input parameters contain an invalid data type.

Returns

A (deradicalisation flag, per-agent delta) pair outlining if deradicalisation occurred, and the per-agent opinion delta to apply.

Return type

tuple[bool, float]

2.11.15 gatoh.groups.Group.determine_predominant_personality

Group.**determine_predominant_personality**(*member_personalities: list[str]*) → None

A setter method that will determine what the predominant personality type amongst group members is.

Parameters

member_personalities (*list[str]*) – The personality types for each group member.

Raises

ValueError – If the input list is not the same size as the group or one or more personalities are invalid.

2.11.16 gatoh.groups.Group.evolve_hierarchy

Group.**evolve_hierarchy**(*weighting_rw: tuple[float, float]*) → tuple[str, float]

Experimental function that aims to model the constantly evolving 'intrinsic value' that people palce on the social hierarchies that the y belong in over time.

Parameters

weighting_rw (*tuple[float, float]*) – The (mean, variance) for a normal distribution that will be used to draw random walk values.

Raises

TypeError – If the input parameter is of an incorrect type.

Returns

The group's hierarchy and the per-agent delta value that must be applied to each agent's hierarchy weighting.

Return type
tuple[str, float]

2.11.17 gatoh.groups.Group.generate_group

Group.**generate_group**(*id: str*, *index: int*, *hierarchy: str*, *members: list[*Agent*]*, *cohesion: str | None = None*, *max_size: int | None = None*) → *Group*

Generate a Group object based on the input parameters.

Parameters

- **id** (*str*) – The id that has been assigned for this specific Group object under the conditions of the model specifications.
- **index** (*int*) – The index of the Group object within the model's GroupSet.
- **hierarchy** (*str*) – The social hierarchy that the Group is being formed in.
- **members** (*list[*Agent*]*) – The Agents that this group will be made up of.
- **cohesion** (*str, optional*) – A string defining what type of cohesion the group will behave according to (defaults to 'neutral' on Group __init__).
- **max_size** (*int, optional*) – An explicit maximum group size to set for the generated group.

Raises

- **TypeError** – If any of the required input parameters are of the incorrect data type.
- **ValueError** – If the input cohesion is not a supported type.

Returns
The generated Group object.

Return type
Group

2.11.18 gatoh.groups.Group.get_attribute

Group.**get_attribute**(*name: str*) → T | None

Return any existing or dynamically added attribute held by the Group object.

Parameters
name (*str*) – The name of the attribute to get.

Raises
UserWarning – If the attribute does not exist.

Returns
The value stored for the attribute.

Return type
Any

2.11.19 gatoh.groups.Group.get_num_members

Group.**get_num_members**() → int

A getter method that reports the number of members that are contained in this Group.

Returns
The number of agents that are members of this Group.

Return type
int

2.11.20 gatoh.groups.Group.group_at_capacity

Group.**group_at_capacity**() → bool

A function that reports if the group is at its maximum capacity.

Returns
A flag indicating if the group is at its maximum capacity.

Return type
bool

2.11.21 gatoh.groups.Group.is_benefited

Group.**is_benefited**(*threshold: float = 0.4*) → bool

A function that reports if the group is considered to experience personal benefit from the social contagion as a collective.

Parameters
threshold (*float, optional*) – The threshold that the member benefit rate must surpass for the group to be considered as benefited.

Raises
AttributeError – If the member_benefit_rate has not been initialised.

Returns
A flag indicating if the group can be collectively considered to be benefited.

Return type
bool

2.11.22 gatoh.groups.Group.is_radicalised

Group.**is_radicalised**(*threshold: float = 0.2*) → bool

A function that reports if the group is considered to be "radicalised" as a collective.

Parameters
threshold (*float, optional*) – The threshold that the radicalisation rate must surpass for the group to be considered radicalised.

Raises
AttributeError – If the radicalisation_rate has not been initialised.

Returns
A flag indicating if the group can be collectively considered to be radicalised.

Return type
bool

2.11.23 gatoh.groups.Group.is_silenced

Group.**is_silenced**(*threshold: float = 0.75*) → bool

A function that reports if the group is considered to be silenced in its hierarchy as a collective.

Parameters
threshold (*float, optional*) – The threshold that the silencing rate must surpass for the group to be considered as silenced.

Raises
AttributeError – If silencing_rate has not been initialised.

Returns
A flag indicating if the group can be collectively considered to be silenced.

Return type
float

2.11.24 gatoh.groups.Group.life_events

Group.**life_events**(*personality_changes: bool = False*, *benefit_changes: bool = False*, *radicalisation_changes: bool = False*, *silencing_changes: bool = False*, *parameters: LifeEventsDict | None = None*) → dict[str, dict[str, Any]]

Experimental function that aims to model the ways in which behaviours change according to major random life events over time.

Parameters

- **personality_changes** (*bool, optional*) – A flag indicating if personality changes are allowed to occur due to life events.
- **benefit_changes** (*bool, optional*) – A flag indicating if changes in personal benefit statuses are allowed to occur due to life events.
- **radicalisation_changes** (*bool, optional*) – A flag indicating if spontaneous changes to radicalisation statuses are allowed to occur due to life events.
- **silencing_changes** (*bool, optional*) – A flag indicating if spontaneous changes to opinion silencing statuses are allowed to occur due to life events.
- **parameters** (*dict[str, Any], optional*) – A <key, value> mapping which outlines all relevant threshold or other values used in this function.

2.11.25 gatoh.groups.Group.opinion_negation

Group.**opinion_negation**(*absolute_difference: float*, *threshold: float*) → bool

Checks if a group has experienced sufficiently 'overwhelming' social pressure in its hierarchy leading to a complete reversal of its members' opinions.

Parameters

- **absolute_difference** (*float*) – The absolute difference between the perceived opinion climate and the group's aggregate opinion.
- **threshold** (*float*) – A global model threshold that has been specified for this effect to occur.

Raises

- **RuntimeError** – If the group has not yet been initialised appropriately.
- **TypeError** – If the input parameters contain any invalid data types.

Returns
A flag indicating if the group's opinion experienced a total negation.

Return type
bool

2.11.26 gatoh.groups.Group.opinion_silencing

Group.**opinion_silencing**(*estimated_opinion_climate: float*, *silencing_threshold: float | None = None*) → tuple[bool, float]

Determines if members in the group will become silenced in their hierarchy based on their collective attributes.

If no silencing threshold has been passed, the group's aggregate susceptibility is used as the threshold instead.

Using the information returned by this function, the parent model will then have to handle it appropriately and make a call to *update()* with the defined silencing_rate delta due to the reported absolute difference value.

Parameters

- **estimated_opinion_climate** (*float*) – The opinion climate perceived by the Group in its hierarchy.
- **silencing_threshold** (*float, optional*) – The silencing threshold that must be surpassed for silencing to occur.

Raises

- **RuntimeError** – If opinion silencing is called before the group has been initialised appropriately.
- **TypeError** – If the input parameters contain any invalid data types.

Returns

A pair of values indicating if silencing occurs, and the absolute difference between the perceived opinion climate and the group's aggregate opinion.

Return type

tuple[bool, float]

2.11.27 gatoh.groups.Group.radicalisation

Group.**radicalisation**(*neighbour_influences: float*, *neighbour_benefits: list[bool]*, *threshold: float | None = None*) → tuple[bool, float]

Uses the group's own aggregate opinion as well as the neighbours' opinions to determine if a non-radical group will become radicalised.

Parameters

- **neighbour_influences** (*float*) – The total influence that the group's neighbouring opinions have had on it this iteration.
- **neighbour_benefits** (*list[bool]*) – Flags indicating the presence of personal benefit across a group's neighbours.
- **threshold** (*float, optional*) – The radicalisation threshold that has been defined at the global level in the model.

Raises

- **RuntimeError** – If the group has not yet been initialised appropriately.
- **TypeError** – If any of the input parameters contain an invalid data type.

Returns

A (radicalisation flag, per-agent delta) pair outlining if radicalisation has occurred, and the per-agent opinion delta to apply.

Return type
tuple[bool, float]

2.11.28 gatoh.groups.Group.recalculate_aggregate_opinion

Group.**recalculate_aggregate_opinion**(*member_opinions: list[float]*) → None

A setter method that will calculate a fixed aggregate opinion value from the currently held opinion values of members.

Parameters
member_opinions (*list[float]*) – The currently held opinion values of group members.

Raises
ValueError – If the input list is not the same size as the group.

2.11.29 gatoh.groups.Group.recalculate_aggregate_susceptibility

Group.**recalculate_aggregate_susceptibility**(*member_susceptibilities: list[float]*) → None

A setter method that will calculate a fixed aggregate social susceptibility value from the currently held opinion values of members.

Parameters
member_susceptibilities (*list[float]*) – The current social susceptibility values of group members.

Raises
ValueError – If the input list is not the same size as the group.

2.11.30 gatoh.groups.Group.recalculate_hierarchy_weighting

Group.**recalculate_hierarchy_weighting**(*member_weightings: list[float]*) → None

A setter method that will calculate a fixed member hierarchy weighting value from the current hierarchy weightings assigned by each group member.

Parameters
member_weightings (*list[float]*) – The current hierarchy weightings assigned by each group member.

Raises
ValueError – If the input list is not the same size as the group.

2.11.31 gatoh.groups.Group.recalculate_member_benefit_rate

Group.**recalculate_member_benefit_rate**(*member_benefits: list[bool]*) → None

A setter method that will calculate a fixed member benefit rate value from the current personal benefit statuses of group members.

Parameters
member_benefits (*list[bool]*) – The current personal benefit status for each group member.

Raises
ValueError – If the input list is not the same size as the group.

2.11.32 gatoh.groups.Group.recalculate_radicalisation_rate

Group.**recalculate_radicalisation_rate**(*member_radicalisations: list[bool]*) → None

A setter method that will calculate a fixed radicalisation rate value from the current radicalisation statuses of group members.

Parameters
member_radicalisations (*list[bool]*) – The current radicalisation status for each group member.

Raises
ValueError – If the input list is not the same size as the group.

2.11.33 gatoh.groups.Group.recalculate_silencing_rate

Group.**recalculate_silencing_rate**(*members_silenced: list[bool]*) → None

A setter method that will calculate a fixed silencing rate value from the current opinion silencing statuses of group members.

Parameters
members_silenced (*list[bool]*) – The current opinion silencing status for each group member.

Raises
ValueError – If the input list is not the same size as the group.

2.11.34 gatoh.groups.Group.set_cohesion

Group.**set_cohesion**(*cohesion: str*) → None

A setter function to change the group's cohesion type.

Parameters
cohesion (*str*) – The new cohesion type to assign to the Group.

Raises

- **TypeError** – If the input cohesion type is not a string.
- **ValueError** – If the cohesion is not one of the supported types.

2.11.35 gatoh.groups.Group.set_hierarchy

Group.**set_hierarchy**(*hierarchy: str*) → None

A setter function to change the hierarchy that the Group belongs to.

Parameters
hierarchy (*str*) – The name of the hierarchy that the Group will belong in.

Raises
TypeError – If the input hierarchy name is not a string.

2.11.36 gatoh.groups.Group.set_index

Group.**set_index**(*index: int*) → None

A setter function to change the Group's index.

Parameters
index (*int*) – The new index to assign to the group.

Raises
TypeError – If the input index is not an int.

2.11.37 gatoh.groups.Group.set_max_size

Group.**set_max_size**(*max_size: int*, *no_limit: bool = False*) → None

A setter function to change the maximum size of a Group.

Parameters

- **max_size** (*int*) – The maximum number of agents that can be members of this group.
- **no_limit** (*bool, optional*) – A flag indicating if the change should remove the group's size limit.

Raises

- **TypeError** – If the input max_size is not an integer.
- **ValueError** – If the input max_size is equal or less than zero.

2.11.38 gatoh.groups.Group.step

Group.**step**(*weighting_rw: tuple[float, float]*, *opinion_rw: tuple[float, float]*) → dict[str, Any]

Step the individual group object:

1. Handle dynamic hierarchy weightings
2. Handle the stochastic opinion changes

Parameters

- **weighting_rw** (*tuple[float, float]*) – The (mean, variance) defining the random walk distribution of the aggregate hierarchy weighting.
- **opinion_rw** (*tuple[float, float]*) – The (mean, variance) defining the random walk distribution of the aggregate opinion.

Returns
A <label : information> mapping that provides all relevant information from the group's step to the parent model.

Return type
dict[str, Any]

2.11.39 gatoh.groups.Group.stochastic_benefit_change

Group.**stochastic_benefit_change**() → dict[str, bool]

Calls to this function are primarily meant to originate from *life_events()*.

Will randomly select a new number of group members that should have personal benefit, and then randomly assigns these benefit statuses to the group members.

Raises
RuntimeError – If the group has not yet been initialised appropriately.

Returns
A <member ID : benefit flag> mapping that specifies the new personal benefit flags for each agent member.

Return type
dict[str, bool]

2.11.40 gatoh.groups.Group.stochastic_opinion

Group.**stochastic_opinion**(*opinion_rw: tuple[float, float]*) → tuple[str, float]

Determine the direction and magnitude of a shift in the Group's aggregate opinion and apply it.

This is representative of opinion changes in real social networks in which individual opinions may increase or decrease away from the aggregate mean (independent of external influences).

Parameters
opinion_rw (*tuple[float, float]*) – A (mean, variance) pair which parametrises the Gaussian distribution used for stochastic opinion shift.

Raises

- **RuntimeError** – If the group has not yet been initialised appropriately.
- **TypeError** – If the input contains any invalid data types.

Returns
The group's hierarchy and per-member delta that must be applied to each member Agent to shift the aggregate opinion by the drawn delta value.

Return type
tuple[str, float]

2.11.41 gatoh.groups.Group.stochastic_personality_change

Group.**stochastic_personality_change**(*personality_probs: PersonalityProbs | None = None*) → dict[str, str]

Calls to this function are primarily meant to originate from *life_events()*.

Will redraw a valid predominant personality types from the ones that have been defined, and then change the group's predominant personality to this new type.

Following this, the minimum proportion of members which need to hold this personality is determined, and the members' personalities are randomly reassigned so that a proportion equal or greater to the minimum needed for this to be the predominant personality type are present.

Parameters
personality_probs (*dict[str, float]*) – Specific per-personality type probabilities to be used for drawing the new predominant personality type.

Raises

- **TypeError** – If a non-float probability is supplied.
- **KeyError** – If an unsupported personality type is passed.

Returns
A <member ID : personality> mapping that specifies which (if any) member personalities have changed.

Return type
dict[str, str]

2.11.42 gatoh.groups.Group.stochastic_radicalisation_change

Group.**stochastic_radicalisation_change**() → dict[str, bool]

Calls to this function are primarily meant to originate from *life_events()*.

Will randomly select a new number of group members that are radicalised, and then randomly assigns these radicalisation statuses to the group members.

Raises
RuntimeError – If the group has not yet been initialised appropriately.

Returns
A <member ID : radicalisation status> mapping that specifies the new radicalisation status for each agent member.

Return type
dict[str, bool]

2.11.43 gatoh.groups.Group.stochastic_silencing_change

Group.**stochastic_silencing_change**() → dict[str, bool]

Calls to this function are primarily meant to originate from *life_events()*.

Will randomly select a new number of group members that are silenced, and then randomly assigns these silencing statuses to the group members.

Raises
RuntimeError – If the group has not yet been initialised appropriately.

Returns
A <Member ID : silencing status> mapping that specifies the new silencing status for each agent member.

Return type
dict[str, bool]

2.11.44 gatoh.groups.Group.store_previous_opinion

Group.**store_previous_opinion**() → None

A setter method that stores the Group's current opinion into the previous opinion.

2.11.45 gatoh.groups.Group.update

Group.**update**(*members: list[*Agent*]*) → GroupUpdateDict

Updates the internal state of the group after the model has stepped by recalculating all aggregate attributes to reflect any additional effects from members which may belong to multiple groups.

Parameters
members (*list[*Agent*]*) – The Agent objects of the members that make up this group.

Raises

- **RuntimeError** – If the group has not yet been initialised appropriately.
- **TypeError** – If the input parameter is invalid.

Returns
All of the group's recalculated aggregate attributes.

Return type
dict[str, float]

gatoh.groups.GroupSet()	A collection of Group objects that maintains consistency for the Model.

2.12 GroupSet

class GroupSet

Bases: object

A collection of Group objects that maintains consistency for the Model.

Methods

add	Adds a Group to the GroupSet.
benefit_count	A getter function that returns the number of groups which experience aggregate personal benefit from social contagion in the group set.
discard	Removes a Group from the GroupSet which matches the input Group; does not return an error if the Group does not exist.
discard_index	Removes the Group at the specified index in the GroupSet; does not raise an error if the index is out of bounds.
extract_group_pickle	A helper function that allows for multithreading of *load_groupset()*.
get_group_by_id	Searches the GroupSet for a Group with the given id and returns its object if it exists.
get_group_ids	A getter function that returns the IDs of all groups contained within the group set.
get_groups_by_ids	Searches the GroupSet for Groups with the given ids and returns their objects if they all exist.
get_index	Returns the index within the GroupSet of the input Group object.
get_indices	Returns the indices within the GroupSet of the input Group objects.
get_member_ids	A getter function that returns the unique IDs of all members contained within the group set.
get_num_members	A wrapper that used the group's getter function to report the number of members of that group.
get_total_members	A getter function that iterates across all groups in the groupset and adds their member counts to report the total number of members that exist in the groupset.
group_at_index	Returns the Groups object at the given index in the GroupSet.
groups_at_indices	Returns a list of all the group objects at the specified indices.
id_in_groupset	Report if a group with the given unique ID exists in the groupset.
load_groupset	Loads a GroupSet that has been saved following the same process as in the *save_groupset()* function.

continues on next page

Table 24 – continued from previous page

personality_count	A getter function that returns the count of groups which hold a specific personality type as it predominant one in the group set.
personality_counts	A getter function that returns the counts of all predomninant personality types among groups in the group set.
radicalised_count	A getter function that returns the number of radicalised groups that are present in the group set.
remove	Removes a group from the GroupSet which matches the input group; returning an error if such a Group does not exist.
remove_index	Removes the Group at the specified index in the GroupSet; raising an error if the index is out of bounds.
sample	Randomly draw n groups from the GroupSet without replacement.
save_groupset	Save the Group objects into a compressed subdirectory representing the saved GroupSet.
update_indices	Iterate over the GroupSet and update the current Group object index values.
write_group_pickle	A helper function that allows for multithreading of *save_groupset()*.

2.12.1 gatoh.groups.GroupSet.add

GroupSet.**add**(*group:* Group) → int

Adds a Group to the GroupSet.

Parameters
group (Group) – The Group object to be added.

Returns
The index of the newly added Group.

Return type
int

2.12.2 gatoh.groups.GroupSet.benefit_count

GroupSet.**benefit_count**() → int

A getter function that returns the number of groups which experience aggregate personal benefit from social contagion in the group set.

Returns
The number of benefited groups in the group set.

Return type
int

2.12.3 gatoh.groups.GroupSet.discard

GroupSet.**discard**(*group:* Group) → bool

Removes a Group from the GroupSet which matches the input Group; does not return an error if the Group does not exist.

Parameters
group (Group) – The Group object that should be removed from the set.

Returns
A flag indicating if the Group was removed successfully or not.

Return type
bool

2.12.4 gatoh.groups.GroupSet.discard_index

GroupSet.**discard_index**(*index: int*) → bool

Removes the Group at the specified index in the GroupSet; does not raise an error if the index is out of bounds.

Parameters
index (*int*) – The index in the GroupSet which is to be removed.

Returns
A flag indicating if the Group was removed successfully or not.

Return type
bool

2.12.5 gatoh.groups.GroupSet.extract_group_pickle

GroupSet.**extract_group_pickle**(*group_pickle_name: str*, *subdirectory_path: str*) → *Group*

A helper function that allows for multithreading of *load_groupset()*.

Parameters

- **group_pickle_name** (*str*) – The name of the pickled Group object file.
- **subdirectory_path** (*str*) – The root path to which the pickled group object was written.

Returns
The unpickled group.

Return type
Group

2.12.6 gatoh.groups.GroupSet.get_group_by_id

GroupSet.**get_group_by_id**(*id: str*) → *Group*

Searches the GroupSet for a Group with the given id and returns its object if it exists.

Parameters
id (*str*) – The id that was assigned to the Group object at creation.

Raises
KeyError – If the input id does not exist in the GroupSet.

Returns
The Group object with the specified id.

Return type
Group

2.12.7 gatoh.groups.GroupSet.get_group_ids

GroupSet.**get_group_ids**() → list[str]

A getter function that returns the IDs of all groups contained within the group set.

Returns
All of the contained groups' IDs.

Return type
list[str]

2.12.8 gatoh.groups.GroupSet.get_groups_by_ids

GroupSet.**get_groups_by_ids**(*ids: list[str]*) → list[*Group*]

Searches the GroupSet for Groups with the given ids and returns their objects if they all exist.

Parameters
ids (*list[str]*) – The ids that have been assigned to every Group object at creation.

Returns
The Group objects with the specified ids.

Return type
list[*Group*]

2.12.9 gatoh.groups.GroupSet.get_index

GroupSet.**get_index**(*group:* Group) → int

Returns the index within the GroupSet of the input Group object.

Parameters
group (Group) – The group whose index is being searched for.

Raises
KeyError – If the input Group does not exist in the GroupSet.

Returns
The index of the group within the GroupSet.

Return type
int

2.12.10 gatoh.groups.GroupSet.get_indices

GroupSet.**get_indices**(*groups: list[*Group*]*) → list[int]

Returns the indices within the GroupSet of the input Group objects.

Parameters
groups (*list[*Group*]*) – The groups whose indices are being searched for.

Returns
The indices of the groups within the GroupSet.

Return type
list[int]

2.12.11 gatoh.groups.GroupSet.get_member_ids

GroupSet.**get_member_ids**() → list[str]

A getter function that returns the unique IDs of all members contained within the group set.

Returns
Every unique member ID.

Return type
list[str]

2.12.12 gatoh.groups.GroupSet.get_num_members

GroupSet.**get_num_members**(*index: int*) → int

A wrapper that used the group's getter function to report the number of members of that group.

Parameters
index (*int*) – The index of the group for which the number of members is being inspected.

Raises
IndexError – If the index is out of bounds.

Returns
The number of agents which are members of the group at the specified index.

Return type
int

2.12.13 gatoh.groups.GroupSet.get_total_members

GroupSet.**get_total_members**() → int

A getter function that iterates across all groups in the groupset and adds their member counts to report the total number of members that exist in the groupset.

Returns
The total number of members that exist in the grouspet.

Return type
int

2.12.14 gatoh.groups.GroupSet.group_at_index

GroupSet.**group_at_index**(*index: int*) → *Group* | None

Returns the Groups object at the given index in the GroupSet.

Parameters
index (*int*) – The index within the GroupSet to inspect.

Raises
UserWarning – If the input index is out of bounds, raise a warning and return None.

Returns
The Group object at the specified index.

Return type
Group

2.12.15 gatoh.groups.GroupSet.groups_at_indices

GroupSet.**groups_at_indices**(*indices: list[int]*) → list[*Group*]

Returns a list of all the group objects at the specified indices.

Parameters
indices (*list[int]*) – A list of all the indices for which the groups should be returned.

Raises
UserWarning – If any of the input indices are out of bounds.

Returns
All of the group objects that correspond to the input indices.

Return type
list[*Group*]

2.12.16 gatoh.groups.GroupSet.id_in_groupset

GroupSet.**id_in_groupset**(*group_id: str*) → bool

Report if a group with the given unique ID exists in the groupset.

Parameters
group_id (*str*) – The unique ID of the group that is being checked for.

Returns
A flag indicating if an group with the input ID exists in the groupset or not.

Return type
bool

2.12.17 gatoh.groups.GroupSet.load_groupset

GroupSet.**load_groupset**(*load_path: str*) → None

Loads a GroupSet that has been saved following the same process as in the *save_groupset()* function.

Parameters
load_path (*str*) – The path to the model's overall save directory.

Raises
FileNotFoundError – If no valid groupset zip was found in the load path.

2.12.18 gatoh.groups.GroupSet.personality_count

GroupSet.**personality_count**(*personality: str*) → int

A getter function that returns the count of groups which hold a specific personality type as it predominant one in the group set.

Parameters
personality (*str*) – The personality type that is being counted.

Raises
ValueError – If the personality type input is unsupported.

Returns
The count of groups which hold the specified predominant personality type.

Return type
int

2.12.19 gatoh.groups.GroupSet.personality_counts

GroupSet.**personality_counts**() → dict[str, int]

A getter function that returns the counts of all predomninant personality types among groups in the group set.

Returns
The count of each supported personality type which is predominant among groups in the group set.

Return type
dict[str, int]

2.12.20 gatoh.groups.GroupSet.radicalised_count

GroupSet.**radicalised_count**() → int

A getter function that returns the number of radicalised groups that are present in the group set.

Returns
The number of radicalised groups in the group set.

Return type
int

2.12.21 gatoh.groups.GroupSet.remove

GroupSet.**remove**(*group:* Group) → bool

Removes a group from the GroupSet which matches the input group; returning an error if such a Group does not exist.

Parameters
group (Group) – The group that should be removed from the set.

Raises
KeyError – If the input Group does not exist in the GroupSet.

Returns
A flag indicating that the Group was removed successfully.

Return type
bool

2.12.22 gatoh.groups.GroupSet.remove_index

GroupSet.**remove_index**(*index: int*) → bool

Removes the Group at the specified index in the GroupSet; raising an error if the index is out of bounds.

Parameters
index (*int*) – The index in the GroupSet which is to be removed.

Raises
IndexError – If the input index is out of bounds for the GroupSet.

Returns
A flag indicating that the Group was removed successfully.

Return type
bool

2.12.23 gatoh.groups.GroupSet.sample

GroupSet.**sample**(*n: int*) → list[*Group*]

Randomly draw n groups from the GroupSet without replacement.

Parameters
n (*int*) – The number of groups to sample.

Returns
The groups sampled from the GroupSet.

Return type
list[*Group*]

2.12.24 gatoh.groups.GroupSet.save_groupset

GroupSet.**save_groupset**(*directory_path: str*) → None

Save the Group objects into a compressed subdirectory representing the saved GroupSet.

Parameters
directory_path (*str*) – The path to the directory where the groupset subdirectory should be created.

2.12.25 gatoh.groups.GroupSet.update_indices

GroupSet.**update_indices**() → None

Iterate over the GroupSet and update the current Group object index values.

2.12.26 gatoh.groups.GroupSet.write_group_pickle

GroupSet.**write_group_pickle**(*group:* Group, *subdirectory_path: str*) → str

A helper function that allows for multithreading of *save_groupset()*.

Parameters

- **group** (Group) – The group that is being saved.
- **subdirectory_path** (*str*) – The path to the subdirectory in which the groups are being saved.

Returns
The path to which the group pickle was saved.

Return type
str

gatoh.logging.GATOHLogger(max_iterations, ...)	The logging module will contain all functions related to logging and/or printing model progress and information both during and after simulation.

2.13 GATOHLogger

class GATOHLogger(*max_iterations: int*, *hierarchies: list[str]*, *verbose: bool = False*, *print_interval: int = 10*, *print_outside_interval: bool = True*, *write_file: bool = True*, *debug: bool = False*)

Bases: object

The logging module will contain all functions related to logging and/or printing model progress and information both during and after simulation.

Parameters

- **max_iterations** (*int*) – The maximum number of iterations that the parent model is running its simulation for.
- **hierarchies** (*list[str]*) – The names of the social hierarchies present in the parent model.
- **verbose** (*bool, optional*) – A flag indicating if extended information should be printed during logging.
- **print_interval** (*int, optional*) – The number of model iterations to run in between each printed logging output.
- **print_outside_interval** (*bool, optional*) – A flag indicating if a simple string indicating just the iteration number should be printed outside the print_interval.
- **write_file** (*bool, optional*) – A flag indicating if a log file should be written to disk at the end of logging.
- **debug** (*bool, optional*) – A flag indicating if additional developer statistics should be logged and reported.

Methods

debug_iteration	Store all relevant developer variables based on the level of logging that has been specified.
debug_iteration_print	A method which formats the debug statistics for printing.
format_non_interval_print	Returns a formatted string to be printed out on iterations which fall outside the print interval (to still provide some feedback on iteration progress)
iteration	Store all relevant model variables and states based on the level of logging that has been specified.
iteration_print	A method which formats the model statistics at the appropriate print_interval.
log_function_call	A wrapper to the DevStats log_function_call.
new_iteration	A wrapper that calls LoggerVariables new_iteration().
save_data	Saves all logged data to the specified path.
track_model_parameters	A utility function that adds new model parameters to be tracked to the logger variables.

2.13.1 gatoh.logging.GATOHLogger.debug_iteration

GATOHLogger.**debug_iteration**() → None

Store all relevant developer variables based on the level of logging that has been specified.

2.13.2 gatoh.logging.GATOHLogger.debug_iteration_print

GATOHLogger.**debug_iteration_print**() → str

A method which formats the debug statistics for printing.

Returns
An appropriately formatted textual representation of debug statistics to be printed out by the model for this iteration.

Return type
str

2.13.3 gatoh.logging.GATOHLogger.format_non_interval_print

GATOHLogger.**format_non_interval_print**() → str

Returns a formatted string to be printed out on iterations which fall outside the print interval (to still provide some feedback on iteration progress)

Defined as its own function to allow for easy modification in the future.

Returns
The formatted text to print outside of the print interval.

Return type
str

2.13.4 gatoh.logging.GATOHLogger.iteration

GATOHLogger.**iteration**(*aggregate_opinion: float*, *radicalised_agents: int*, *radicalised_groups: int*, *radicalisation_logodds: float*, *layer_interdependences: dict[str, float]*, *layer_polarisations: dict[str, float]*, *agent_attributes: dict[str, dict[str, Any]]*, *model_parameters: dict[str, Any] | None = None*) → None

Store all relevant model variables and states based on the level of logging that has been specified.

Parameters

- **aggregate_opinion** (*float*) – The aggregate network opinion that has been observed in the model at the end of this iteration.
- **radicalised_agents** (*int*) – The number of radicalised agents in the model at the end of this iteration.
- **radicalised_groups** (*int*) – The number of radicalised groups in the model at the end of this iteration.
- **radicalisation_logodds** (*float*) – The log odds of an agent being radicalised in the model at the end of this iteration.
- **layer_interdependences** (*dict[str, float]*) – A <hierarchy : value> mapping containing the calculated layer interdependency for each hierarchy in the model at the end of this iteration.
- **layers_polarisation** (*dict[str, float]*) – A <hierarchy : value> mapping containing the calculated polarisation for each hierarchy in the model at the end of this iteration.
- **agent_attributes** (*dict[str, dict[str, Any]]*) – A <agent : <attribute : value>> nested mapping containing all relevant attributes for agents which are being tracked.
- **model_parameters** (*dict[str, Any], optional*) – A <parameter : value> mapping containing a tracked model parameter's value at the end of this iteration.

2.13.5 gatoh.logging.GATOHLogger.iteration_print

GATOHLogger.**iteration_print**() → str

A method which formats the model statistics at the appropriate print_interval.

Returns
An appropriately formatted textual representation of variables to be printed out by the model for this iteration.

Return type
str

2.13.6 gatoh.logging.GATOHLogger.log_function_call

GATOHLogger.**log_function_call**(*function_name: str*) → None

A wrapper to the DevStats log_function_call.

Parameters
function_name (*str*) – The name of the function that was called.

2.13.7 gatoh.logging.GATOHLogger.new_iteration

GATOHLogger.**new_iteration**(*init: bool = False*) → None

A wrapper that calls LoggerVariables new_iteration().

Parameters
init (*bool, optional*) – A flag indicating if this function is being called from the first iteration of the model.

2.13.8 gatoh.logging.GATOHLogger.save_data

GATOHLogger.**save_data**(*save_path: str*) → bool

Saves all logged data to the specified path.

Parameters
save_path (*str*) – The path to save the logger's data to.

Returns
A flag indicating if data was successfully saved or not.

Return type
bool

2.13.9 gatoh.logging.GATOHLogger.track_model_parameters

GATOHLogger.**track_model_parameters**(*parameters: list[str]*) → None

A utility function that adds new model parameters to be tracked to the logger variables.

Parameters
parameters (*list[str]*) – The names of the model parameters to be tracked.

gatoh.logging.LoggerVariables(...)	Dataclass that defines the simulation variables that are tracked and stored by the Logger.

2.14 LoggerVariables

class LoggerVariables(*max_iterations: int*, *hierarchies: list[str]*)

Bases: `object`

Dataclass that defines the simulation variables that are tracked and stored by the Logger.

Parameters

- **max_iterations** (*int*) – The maximum number of iterations that the model will run its simulation for.
- **hierarchies** (*list[str]*) – The names of all social hierarchies present in the model.

Store the number of max iterations and initialise all lists and dictionaries with the appropriate hierarchy names and sizes to match the number of iterations.

Methods

current_iteration_repr	Extract all variable information for the current iteration and format it into a string to be printed to the terminal.
current_layers_repr	Extract all the per-hierarchy variables for the current iteration and format it into a substring to be appended to the main iteration output.
current_model_params_repr	Extract all the explicitly tracked model parameters for the current iteration and format them into a substring to be appended to the main iteration output.
get_fieldnames	A helper function that provides a place to collect and return all of the dataclass' attribute names for use as CSV column names.
increment_deradicalised	A simple setter function that checks the input flag and updates the deradicalisation event count accordingly.
increment_deradicalised_group	A simple setter function that checks the input flag and updates the group deradicalisation event count accordingly.
increment_negated	A simple setter function that checks the input flag and updates the opinion negation events count accordingly.
increment_negated_group	A simple setter function that checks the input flag and updates the group negation events count accordingly.
increment_radicalised	A simple setter function that checks the input flag and updates the radicalisation event count accordingly.
increment_radicalised_group	A simple setter function that checks the input flag and updates the group radicalisation event count accordingly.
increment_silenced	A simple setter function that checks the input flag and updates the opinion silencing events count accordingly.
increment_silenced_group	A simple setter function that checks the input flag and updates the group silencing events count accordingly.
new_iteration	Increment the current_iteration counter and then copy all the values from the previous iteration to their respective list indexes for the new iteration.

continues on next page

Table 28 – continued from previous page

store_aggregate_opinion	A setter function that simplifies the storing of aggregate opinion values at each iteration.
store_group_radicalisation	A setter function that simplifies the storing of the model's radicalised groups count at each iteration.
store_layer_interdependences	A setter function that simplifies the storing of the model's layer interdependences at each iteration.
store_layer_polarisations	A setter function that simplifies the storing of the model's layer polarisations at each iteration.
store_model_parameters	A setter function that simplifies the storing of the model's tracked parameters at each iteration.
store_radicalisation	A setter function that simplifies the storing of the model's radicalised agents count at each iteration.
store_radicalisation_logodds	A setter function that simplifies the storing of the model's radicalisation log odds at each iteration.

2.14.1 gatoh.logging.LoggerVariables.current_iteration_repr

LoggerVariables.**current_iteration_repr**() → str

Extract all variable information for the current iteration and format it into a string to be printed to the terminal.

Returns
A formatted text representation containing all the variables for the current model iteration.

Return type
str

2.14.2 gatoh.logging.LoggerVariables.current_layers_repr

LoggerVariables.**current_layers_repr**() → str

Extract all the per-hierarchy variables for the current iteration and format it into a substring to be appended to the main iteration output.

Returns
A formatted text representation containing all the per-hierarchy variables for the current model iteration.

Return type
str

2.14.3 gatoh.logging.LoggerVariables.current_model_params_repr

LoggerVariables.**current_model_params_repr**() → str

Extract all the explicitly tracked model parameters for the current iteration and format them into a substring to be appended to the main iteration output.

Returns
A formatted text representation containing all the tracked model parameters for the current model iteration.

Return type
str

2.14.4 gatoh.logging.LoggerVariables.get_fieldnames

LoggerVariables.**get_fieldnames**() → list[str]

A helper function that provides a place to collect and return all of the dataclass' attribute names for use as CSV column names.

Returns
The names of all the attributes in this dataclass.

Return type
list[str]

2.14.5 gatoh.logging.LoggerVariables.increment_deradicalised

LoggerVariables.**increment_deradicalised**(*flag: bool*) → None

A simple setter function that checks the input flag and updates the deradicalisation event count accordingly.

Parameters
flag (*bool*) – A flag indicating if deradicalisation occurred.

2.14.6 gatoh.logging.LoggerVariables.increment_deradicalised_group

LoggerVariables.**increment_deradicalised_group**(*flag: bool*) → None

A simple setter function that checks the input flag and updates the group deradicalisation event count accordingly.

Parameters
flag (*bool*) – A flag indicating if deradicalisation ocurred.

2.14.7 gatoh.logging.LoggerVariables.increment_negated

LoggerVariables.**increment_negated**(*flag: bool*) → None

A simple setter function that checks the input flag and updates the opinion negation events count accordingly.

Parameters
flag (*bool*) – A flag indicating if opinion negation ocurred.

2.14.8 gatoh.logging.LoggerVariables.increment_negated_group

LoggerVariables.**increment_negated_group**(*flag: bool*) → None

A simple setter function that checks the input flag and updates the group negation events count accordingly.

Parameters
flag (*bool*) – A flag indicating if opinion negation ocurred.

2.14.9 gatoh.logging.LoggerVariables.increment_radicalised

LoggerVariables.**increment_radicalised**(*flag: bool*) → None

A simple setter function that checks the input flag and updates the radicalisation event count accordingly.

Parameters
flag (*bool*) – A flag indicating if radicalisation ocurred.

2.14.10 gatoh.logging.LoggerVariables.increment_radicalised_group

LoggerVariables.**increment_radicalised_group**(*flag: bool*) → None

A simple setter function that checks the input flag and updates the group radicalisation event count accordingly.

Parameters
flag (*bool*) – A flag indicating if radicalisation ocurred.

2.14.11 gatoh.logging.LoggerVariables.increment_silenced

LoggerVariables.**increment_silenced**(*flag: bool*) → None

A simple setter function that checks the input flag and updates the opinion silencing events count accordingly.

Parameters
flag (*bool*) – A flag indicating if opinion silencing ocurred.

2.14.12 gatoh.logging.LoggerVariables.increment_silenced_group

LoggerVariables.**increment_silenced_group**(*flag: bool*) → None

A simple setter function that checks the input flag and updates the group silencing events count accordingly.

Parameters
flag (*bool*) – A flag indicating if opinion silencing ocurred.

2.14.13 gatoh.logging.LoggerVariables.new_iteration

LoggerVariables.**new_iteration**(*init: bool = False*) → None

Increment the current_iteration counter and then copy all the values from the previous iteration to their respective list indexes for the new iteration.

Parameters
init (*bool, optional*) – A flag indicating if the call is being made during the first model iteration (no previous values to copy)

2.14.14 gatoh.logging.LoggerVariables.store_aggregate_opinion

LoggerVariables.**store_aggregate_opinion**(*agg_opp: float*) → None

A setter function that simplifies the storing of aggregate opinion values at each iteration.

Parameters
agg_opp (*float*) – The aggregate opinion value to store for the current iteration.

2.14.15 gatoh.logging.LoggerVariables.store_group_radicalisation

LoggerVariables.**store_group_radicalisation**(*radical_n: int*) → None

A setter function that simplifies the storing of the model's radicalised groups count at each iteration.

Parameters
radical_n (*int*) – The number of radicalised groups to store for the current iteration.

2.14.16 gatoh.logging.LoggerVariables.store_layer_interdependences

LoggerVariables.**store_layer_interdependences**(*layer_interdeps: dict[str, float]*) → None

A setter function that simplifies the storing of the model's layer interdependences at each iteration.

Parameters

layer_interdeps (*dict[str, float]*) – A <hierarchy : interdependence value> mapping that tracks the layer interdependences to be stored for this iteration.

2.14.17 gatoh.logging.LoggerVariables.store_layer_polarisations

LoggerVariables.**store_layer_polarisations**(*layer_polars: dict[str, float]*) → None

A setter function that simplifies the storing of the model's layer polarisations at each iteration.

Parameters

layer_polars (*dict[str, float]*) – A <hierarchy : polarisation value> mapping that tracks the layer polarisations to be stored for this iteration.

2.14.18 gatoh.logging.LoggerVariables.store_model_parameters

LoggerVariables.**store_model_parameters**(*model_parameters: dict[str, Any] | None = None*) → None

A setter function that simplifies the storing of the model's tracked parameters at each iteration.

Parameters

model_parameters (*dict[str, Any], optional*) – A <parameter : value> mapping for the tracked model parameters to be logged.

2.14.19 gatoh.logging.LoggerVariables.store_radicalisation

LoggerVariables.**store_radicalisation**(*radical_n: int*) → None

A setter function that simplifies the storing of the model's radicalised agents count at each iteration.

Parameters

radical_n (*int*) – The number of radicalised agents to store for the current iteration.

2.14.20 gatoh.logging.LoggerVariables.store_radicalisation_logodds

LoggerVariables.**store_radicalisation_logodds**(*r_logodds: float*) → None

A setter function that simplifies the storing of the model's radicalisation log odds at each iteration.

Parameters

r_logodds (*float*) – The model's radicalisation log odds value to store for the current iteration.

Attributes

current_iteration: int = 0

max_iterations: int

aggregate_opinions: list[float]

radicalised_agents: list[int]

radicalisation_events: list[int]

radicalised_groups: list[int]

group_radicalisation_events: list[int]

deradicalised_agents: list[int]

deradicalised_groups: list[int]

silenced_agents: list[int]

silenced_groups: list[int]

negated_agents: list[int]

negated_groups: list[int]

layer_interdependences: dict[str, list[float]]

layer_polarisations: dict[str, list[float]]

radicalisation_logodds: list[float]

model_parameters: dict[str, list[Any]]

gatoh.model.ABModel(hierarchy_names, ...[, ...])	An agent-based model class that is capable of handling multiple layers that affect agent behaviour.

2.15 ABModel

class ABModel(*hierarchy_names: list[str]*, *hierarchy_rw_distributions: list[tuple[float, float]]*, *agent_opinion_rw: tuple[float, float] = (0.0, 0.1)*, *group_opinion_rw: tuple[float, float] = (0.0, 0.1)*, *group_rw_distribution: tuple[float, float] = (0.0, 0.1)*, *iterations: int = 100*, *silencing_threshold: float = 0.95*, *silencing_link: tuple[str, float] = ('', 0.0)*, *negation_threshold: float = 0.999*, *negation_link: tuple[str, float] = ('', 0.0)*, *radicalisation_threshold: float = 0.99*, *radicalisation_link: tuple[str, float] = ('', 0.0)*, *suppress_warnings: bool = False*, *print_interval: int = 10*, *debug: bool = False*, *visualise: bool = True*, *visualisation_dir: str = ''*, *vis_aggregation_method: str = 'median'*, *checkpointing: bool = True*, *init_graphs: bool = False*, *partial_iterations: bool = False*, *simulate_groups: bool = False*, *parameters_to_track: list[str] | None = None*, *save_dir: str = ''*, *data_file: str = ''*, *model_id: str = ''*)

Bases: `object`

An agent-based model class that is capable of handling multiple layers that affect agent behaviour.

Parameters

- **hierarchy_names** (*list[str]*) – The names of all social hierarchies that will exist in the model.
- **hierarchy_rw_distributions** (*list[tuple[float, float]]*) – (mean, variance) parameters used for random walk effects within each hierarchy.
- **agent_opinion_rw** (*tuple[float, float], optional*) – Shared (mean, variance) parameters used for stochastic opinion changes across all agents at each timestep.
- **group_opinion_rw** (*tuple[float, float], optional*) – Shared (mean, variance) parameters used for stochastic opinion changes across all groups at each timestep.

- **group_rw_distribution** (*tuple[float, float], optional*) – (mean, variance) parameters used for random walk effects within the group graph.
- **iterations** (*int, optional*) – The number of iterations that the model will run for.
- **silencing_threshold** (*float, optional*) – A threshold that, when surpassed by agents, will cause them to cease expressing their opinions in a given hierarchy.
- **silencing_link** (*tuple[str, float], optional*) – The link function to apply to the silencing threshold, and the link value to use.
- **negation_threshold** (*float, optional*) – A threshold that, when surpassed by agents, will cause their opinion to become its additive inverse.
- **negation_link** (*tuple[str, float], optional*) – The link function to apply to the negation threshold, and the link value to use.
- **radicalisation_threshold** (*float, optional*) – A threshold that determines how strong of an absolute opinion an agent must hold before they begin to consider becoming radicalised.
- **radicalisation_link** (*tuple[str, float], optional*) – The link function to apply to the radicalisation threshold, and the link value to use.
- **suppress_warnings** (*bool, optional*) – A flag indicating if non-critical runtime warnings should be suppressed.
- **print_interval** (*int, optional*) – The iteration interval at which the model's logger should be printing detailed variable reports.
- **debug** (*bool, optional*) – A flag indicating if additional developer statistics should be tracked and reported during runtime.
- **visualise** (*bool, optional*) – A flag indicating if the model should visualise emergent behaviour in real time.
- **visualisation_dir** (*str, optional*) – The path to a directory in which all of this model's visualiser outputs should be saved to.
- **vis_aggregation_method** (*str, optional*) – The aggregation method that should be used when relevant for visualisation (i.e. "median", "mean", etc.).
- **checkpointing** (*bool, optional*) – A flag indicating if the model's progress should be saved at the end of each iteration (useful in case of interrupted runtimes).
- **init_graphs** (*bool, optional*) – A flag indicating if empty graphs should be initialised with the input hierarchy information.
- **partial_iterations** (*bool, optional*) – A flag indicating if the model's iterations should only simulate effects for a random subset of the agents.
- **simulate_groups** (*bool, optional*) – A flag indicating if the model's iterations will be simulating aggregate agent groups rather than individual agents.
- **parameters_to_track** (*list[str], optional*) – The names of initial model parameters that should be tracked and recorded by the logger.
- **save_dir** (*str, optional*) – The path to a directory in which all of this model's non-logger data should be saved to.
- **data_file** (*str, optional*) – The path to which the logger's data should be saved to after iterations are run.

- **model_id** (*str, optional*) – A unique ID assigned to this model to make it referencable amongst other models.

Methods

add_agent	Add a single new Agent to the model's AgentSet, returning its index within the AgentSet.
add_agents	Add new Agents to the Model's AgentSet.
add_agents_to_hierarchy	A helper function that can directly add a collection of agents to a specific hierarchy in the model's graph set.
add_base_graph_edges	A function that takes a Graph object and adds all of its weighted edges to the model's base graph.
add_graph	Add a new Graph to the Model's GraphSet.
add_graphs	Add new Graphs to the Model's GraphSet.
add_group	Add a single new Group to the model's GroupSet, returning its index within the GroupSet.
add_group_graph_edge	A function that takes two groups and adds a relationship between them in the model's groupgraph.
add_groups	Add new Groups to the Model's GroupSet.
add_relationships_to_hierarchy	A helper function that can directly add relationships to a specific hierarchy in the model's graph set.
apply_link_functions	Apply all relevant parameter link functions as needed.
calculate_aggregate_opinion	Calculates the aggregate network opinion by iterating over each Agent in the model.
calculate_aggregate_opinion_deviation	Calculates the standard deviation of the aggregate network opinion by iterating over each Agent in the model.
calculate_density	Calculate the density of the entire network.
calculate_interdependence	Calculate the layer interdependence; a measure of how much impact a specific layer has in the overall social network.
calculate_interdependences	A helper function that allows for multiprocessing of `calculate_interdependence()`.
calculate_layers_polarisation	Calculate the polarisation of the opinion climate within each hierarchy by calling each graph's calculate_polarisation() method.
calculate_navigability	Calculate the difficulty of navigating from an arbitrary node s in some layer a to another arbitrary node t in some layer b, where a and b may or may not be the same layer.
calculate_radicalisation_logodds	Calculates the log odds of an Agent being radicalised within the model.
generate_agents	Randomly generates a number of Agent objects.
generate_graphs	Randomly generates graphs for the given social hierarchy names using the specified method.
generate_groups	Randomly generates a number of Group objects from the model's agent and graph sets, using KMeans clustering.
get_agent_ids	A wrapper for the AgentSet's get_agent_ids function.

continues on next page

Table 30 – continued from previous page

get_base_indices_from_edge	A helper function for the base graph that takes in a GraphEdge object from a hierarchy graph and transforms the node indices from hierarchy graph indices to the respective index of the Agent objects in the model's AgentSet.
get_benefited_count	A wrapper for *benefit_count()*.
get_group_benefited_count	A wrapper for *benefit_count()*.
get_group_personality_count	A wrapper for *personality_count()*.
get_group_personality_counts	A wrapper for *personality_counts()*.
get_group_radical_count	A wrapper for *radicalised_count()*.
get_groupgraph_indices_from_endpoints	A helper function for the groupset that takes in two Group objects and retrieves their node indices within the model's groupgraph.
get_personality_count	A wrapper for *personality_count()*.
get_personality_counts	A wrapper for *personality_counts()*.
get_radical_count	A wrapper for *radicalised_count()*.
get_tracked_agent_attributes	A getter function that checks for any individual agents that are being tracked over the model runtime, and then gathers all relevant attributes for those agents.
get_tracked_parameters	A getter function that checks for any explicitly tracked model parameters, and forms a return accordingly.
group_iteration_opinion_calculation	A helper function that calculates the per-group changes to opinions for the iteration, returning all necessary information for `group_iteration_opinion_changes()` to apply the opinion changes.
group_iteration_opinion_changes	A helper function for group_iterate that simply applies all group opinion changes and then performs the appropriate checks.
group_members_opinion_changes	A helper function that applies the group's aggregate opinion changes to the individual Agents which make it up.
init_base_graph	Iterates over all the relationships in the existing social hierarchies and creates corresponding edges within the model's base graph.
iterate	A wrapper for the model's main iteration function.
iteration_opinion_calculation	A helper function that calculates the per-agent, per-hierarchy changes to opinions for the iteration, returning all necessary information for `iteration_opinion_changes()` to apply the opinion changes.
iteration_opinion_changes	A helper function for iterate that simply applies all agent opinion changes and then checks for radicalisation.
layers_polarisation_multi	A helper function that allows for multiprocessing of `calculate_layers_polarisation()`.
load_model	Loads a model and all its components which have been saved following the processes in the save_model() function.
logger_debug_iteration	A helper function that handles the iteration of the logger's debugging component.

continues on next page

Table 30 – continued from previous page

logger_iteration	Calculate any relevant aggregate statistics and then pass these to the logger's iteration() function to be stored.
override_current_iteration	A setter function that overrides the model's current iteration value with a new one.
save_model	Saves the model's GraphSet and AgentSet objects, including all objects recursively contained.
set_agent_opinion_rw	A setter function that changes the model's agent_opinion_rw attribute.
set_checkpointing	A setter function that changes the model's checkpointing flag.
set_data_file	A setter function that changes the path to which the final data file should be written.
set_hierarchy_information	A setter function that adds a new entry into the model's hierarchy information attribute.
set_max_iterations	A setter function that changes the model's max iterations attribute.
set_model_id	A setter function that changes the model's unique ID.
set_negation_threshold	A setter function that changes the model's negation threshold.
set_partial_iterations	A setter function that changes the model's partial_iterations flag.
set_radicalisation_threshold	A setter function that changes the model's radicalisation threshold.
set_save_dir	A setter function that changes the model's save directory.
set_silencing_threshold	A setter function that changes the model's silencing threshold.
set_simulate_groups	A setter function that changes the model's simulate_groups flag.
set_suppress_warnings	A setter function that changes the model's suppress warnings flag.
set_visualisation_dir	A setter function that changes the model's visualisation directory.
set_visualise	A setter function that changes the model's visualise flag.
step	Steps the model forward one iteration.
track_agent_custom_attribute	Used to specify that a custom attribute of a particular agent should be logged and tracked throughout the model runtime.
track_agent_opinion	Used to specify that the aggregate opinion at the current iteration of a particular agent should be logged and tracked throughout the model runtime.
track_agent_previous_opinion	Used to specify that the aggregate opinion at the previous iteration of a particular agent should be logged and tracked throughout the model runtime.
track_agent_radicalisation	Used to specify that the radicalisation status of a particular agent should be logged and tracked throughout the model runtime.
track_agent_silencing	Used to specify that the silencing status of a particular agent in its hierarchies should be logged and tracked throughout the model runtime.

continues on next page

Table 30 – continued from previous page

track_agent_social_weightings	Used to specify that the social weightings of a particular agent should be logged and tracked throughout the model runtime.
track_model_parameters	Note specific model parameters that should be tracked by the logger, and included in the logger's output file.
update	Updates the agents' internal states to match the model step.
update_base_graph	Iterates over all the social hierarchy graphs and checks for pending edge changes.
update_group_multi	A helper function that allows for multiprocessing of the *update()* function when simulating groups.
update_multi	A helper function that allows for multiprocessing of the *update()* function.

2.15.1 gatoh.model.ABModel.add_agent

ABModel.**add_agent**(*agent:* Agent) → int

Add a single new Agent to the model's AgentSet, returning its index within the AgentSet.

Parameters
agent (Agent) – The agent to add to the AgentSet.

Returns
The index of the newly added Agent in the AgentSet.

Return type
int

2.15.2 gatoh.model.ABModel.add_agents

ABModel.**add_agents**(*agents: list[*Agent*]*) → *AgentSet*

Add new Agents to the Model's AgentSet.

Parameters
agents (*list[*Agent*]*) – The agents to be added to the AgentSet.

Returns
The model's newly updated agent set.

Return type
AgentSet

2.15.3 gatoh.model.ABModel.add_agents_to_hierarchy

ABModel.**add_agents_to_hierarchy**(*agents: list[*Agent*]*, *hierarchy: str*) → None

A helper function that can directly add a collection of agents to a specific hierarchy in the model's graph set.

This function will also add any input agents to the model's overarching AgentSet if they do not already exist in it.

Parameters

- **agents** (*list[*Agent*]*) – The collection of agents that should be added as nodes to the hierarchy.

- **hierarchy** (*str*) – The hierarchy that the agents are being added to.

Raises

- **KeyError** – If the specified hierarchy does not exist in the graph set.
- **ValueError** – If any object in the agents iterable is of an invalid type.

2.15.4 gatoh.model.ABModel.add_base_graph_edges

ABModel.**add_base_graph_edges**(*graph:* Graph) → None

A function that takes a Graph object and adds all of its weighted edges to the model's base graph.

Parameters
graph (Graph) – The new graph that is being added to self.graphs.

2.15.5 gatoh.model.ABModel.add_graph

ABModel.**add_graph**(*graph:* Graph) → *GraphSet*

Add a new Graph to the Model's GraphSet. It is assumed that this is a generated Graph object which already has a name and rw_params assigned to it.

Parameters
graph (Graph) – The graph to be added to the Model's GraphSet.

Returns
The model's newly updated graph set.

Return type
GraphSet

2.15.6 gatoh.model.ABModel.add_graphs

ABModel.**add_graphs**(*graphs: list[*Graph*] | list[str]*, *names: list[str]*, *rw_params: list[tuple[float, float]]*) → *GraphSet*

Add new Graphs to the Model's GraphSet.

Parameters

- **graphs** (*list[*Graph*] | list[str]*) – Graph objects or filepaths to stored GraphML objects.
- **names** (*list[str]*) – The corresponding social hierarchy names to give to the Graphs.
- **rw_params** (*list[tuple[float, float]]*) – The (mean, variance) to assign to the hierarchy when determining normal distributions for random walk dynamic relationships.

Returns
The model's newly updated graph set.

Return type
GraphSet

2.15.7 gatoh.model.ABModel.add_group

ABModel.**add_group**(*group:* Group) → int

Add a single new Group to the model's GroupSet, returning its index within the GroupSet.

Parameters
group (Group) – The group to add to the GroupSet.

Returns
The index of the newly added Group in the GroupSet.

Return type
int

2.15.8 gatoh.model.ABModel.add_group_graph_edge

ABModel.**add_group_graph_edge**(*from_group:* Group, *to_group:* Group, *weighting: float | None = None*, *rw_params: tuple[float, float] | None = None*) → None

A function that takes two groups and adds a relationship between them in the model's groupgraph.

Parameters

- **from_group** (Group) – The node representing the parent node of the relationship.
- **to_group** (Group) – The node representing the child node of the relationship.
- **weighting** (*float, optional*) – The weighting that should be assigned to the relationship.
- **rw_params** (*tuple[float, float], optional*) – The relationship-specific random-walk parameters that should be assigned.

Raises
ValueError – If either of the groups do not exist in the groupgraph as nodes.

2.15.9 gatoh.model.ABModel.add_groups

ABModel.**add_groups**(*groups: list[*Group*]*) → *GroupSet*

Add new Groups to the Model's GroupSet.

Parameters
groups (*list[*Group*]*) – The groups to be added to the GroupSet.

Returns
The model's newly updated group set.

Return type
GroupSet

2.15.10 gatoh.model.ABModel.add_relationships_to_hierarchy

ABModel.**add_relationships_to_hierarchy**(*relationships: dict[str, list[Any]]*, *hierarchy: str*) → None

A helper function that can directly add relationships to a specific hierarchy in the model's graph set.

Validation of from_node and to_node indices is performed within the called Graph function when adding the relationships.

Parameters

- **relationships** (*dict[str, list[int | float]]*) – A <key, values> mapping providing 'to_node', 'from_node', and 'weighting' information.
- **hierarchy** (*str*) – The hierarchy that the relationships are being added to.

Raises

- **KeyError** – If the specified hierarchy does not exist in the graph set.
- **ValueError** – If any required key is missing, or the information is of a mismatching data type.

2.15.11 gatoh.model.ABModel.apply_link_functions

ABModel.**apply_link_functions**() → None

Apply all relevant parameter link functions as needed.

2.15.12 gatoh.model.ABModel.calculate_aggregate_opinion

ABModel.**calculate_aggregate_opinion**() → float

Calculates the aggregate network opinion by iterating over each Agent in the model.

Returns
The aggregate network opinion value.

Return type
float

2.15.13 gatoh.model.ABModel.calculate_aggregate_opinion_deviation

ABModel.**calculate_aggregate_opinion_deviation**() → float

Calculates the standard deviation of the aggregate network opinion by iterating over each Agent in the model.

Returns
The standard deviation of the aggregate network opinion.

Return type
float

2.15.14 gatoh.model.ABModel.calculate_density

ABModel.**calculate_density**() → float

Calculate the density of the entire network.

The density of a graph is defined as:

$$D = \frac{l}{\frac{n(n-1)}{2}}$$

which simply refers to the proportion of existing relationships versus the total possible relationships.

Given that gatoh is multilayer, and bidirectional relationships can exist, the density calculation must instead be defined as:

where the summation is the total possible relationships across all layers.

Returns
The density metric for the entire social network.

Return type
float

2.15.15 gatoh.model.ABModel.calculate_interdependence

ABModel.**calculate_interdependence**(*layer: int*) → float

Calculate the layer interdependence; a measure of how much impact a specific layer has in the overall social network.

The general formula for layer interdependence is defined as:

$$\lambda^a = \frac{\sum_i \sum_{j \neq i} \Psi_{ij}^a}{\sum_i \sum_{j \neq i} \Psi_{ij}}$$

where Ψ_{ij}^a describes the number of shortest paths between nodes i and j using two or more layers, where at least one of the layers passed through is a.

For the case of multilayer social contagion modeling, it has been defined here as:

$$\lambda^a = \frac{\sum_i \sum_{j \neq i} |OC_i'(j)^a|}{\sum_i \sum_{j \neq i} |OC_i'(j)|}$$

where $|OC_i'(j)^a|$ is Agent j's opinion climate value as perceived by Agent i in the social hierarchy layer a. Although, the absolute of this value should be taken, as this is representative of the real "strength" of a layer.

Parameters
layer (*int*) – The index of the layer of interest.

Returns
The layer interdependence measure for the layer of interest.

Return type
float

2.15.16 gatoh.model.ABModel.calculate_interdependences

ABModel.**calculate_interdependences**(*worker_pool: Pool | None = None*) → list[tuple[str, float]]

A helper function that allows for multiprocessing of `calculate_interdependence()`.

Parameters
worker_pool (Pool) – A pool of workers that can distribute the interdependence calculations amongst themselves.

Returns
A list containing the necessary results.

Return type
list[tuple[str, float]]

2.15.17 gatoh.model.ABModel.calculate_layers_polarisation

ABModel.**calculate_layers_polarisation**(*worker_pool: Pool | None = None*) → dict[str, float]

Calculate the polarisation of the opinion climate within each hierarchy by calling each graph's calculate_polarisation() method.

Parameters
worker_pool (Pool) – A pool of workers that can distribute the polarisation calculations amongst themselves.

Returns
A <hierarchy : value> mapping containing the polarisation value for each hierarchy.

Return type
dict[str, float]

2.15.18 gatoh.model.ABModel.calculate_navigability

ABModel.**calculate_navigability**(*from_node: tuple[int, int]*, *to_node: tuple[int, int]*) → float

Calculate the difficulty of navigating from an arbitrary node s in some layer a to another arbitrary node t in some layer b, where a and b may or may not be the same layer.

The general formulae for the navigatability are defined by:

$$S(s \to t) = -\log_2 \sum_{\{p(s,t)\}} P[p(s,t)]$$

$$P[p(s,t)] = \frac{1}{k_s} \prod_{j \in p(s,t)} \frac{1}{k_j - 1}$$

Although, the relationships across the layers in gatoh are bidirectional and weighted, and as such, $P[p(s,t)]$ is redefined as:

$$P[p(s,t)] = \frac{w_{s \to j_o}}{k_s} \prod_{j \in p(s,t)} \frac{w_{j \to j+1}}{k_j - 1}$$

Parameters

- **from_node** (*tuple[int, int]*) – (agent_index, graph_index) for the starting node.
- **to_node** (*tuple[int, int]*) – (agent_index, graph_index) for the end node.

Returns
The navigability value for the specified path.

Return type
float

2.15.19 gatoh.model.ABModel.calculate_radicalisation_logodds

ABModel.**calculate_radicalisation_logodds**() → float

Calculates the log odds of an Agent being radicalised within the model.

Returns
The log odds of agent radicalisation.

Return type
float

2.15.20 gatoh.model.ABModel.generate_agents

ABModel.**generate_agents**(*id_base: str*, *personality_probs: dict[str, float]*, *distribution: str = 'gaussian'*, *parameters: dict[str, float] | None = None*, *number: int = 100*) → None

Randomly generates a number of Agent objects.

Parameters

- **id_base** (*str*) – a 4-character alphabetic string that serves as the base of the XXXXnnnn id for each Agent.
- **personality_probs** (*dict[str, float]*) – A <personality : probability> mapping specifying the probability of an Agent having any given personality.
- **distribution** (*str, optional*) – The distribution from which any random values will be drawn.

- **parameters** (*dict, optional*) – Any explicit parameters that the distribution should use when being created.
- **number** (*int, optional*) – Number of agents to be randomly created.

2.15.21 gatoh.model.ABModel.generate_graphs

ABModel.**generate_graphs**(*hierarchies: list[str]*, *agents: list[*Agent*]*, *method: str = 'small-world'*, *agent_subsetting: bool = False*, *rw_params: list[tuple[float, float]] | None = None*, *individual_methods: dict[str, str] | None = None*, *ensure_connected: dict[str, bool] | None = None*) → None

Randomly generates graphs for the given social hierarchy names using the specified method. Hierarchies will only contain the agents whose names are passed to the function.

Parameters

- **hierarchies** (*list[str]*) – The names of the social hierarchy graphs to be created.
- **agents** (*list[*Agent*]*) – The agents to be included in the hierarchies.
- **method** (*str, optional*) – The social network graph generation method to use. Options include: 'small-world', 'scale-free', 'random', 'blockmodel'. Defaults to 'small-world'.
- **agent_subsetting** (*bool, optional*) – A flag indicating if the agents should be sampled into random subsets when generating each graph.
- **rw_params** (*list[tuple[float, float]], optional*) – (mean, variance) parameters containing the random-walk distributions for each of the generated graphs.
- **individual_methods** (*dict[str, str], optional*) – A <hierarchy : generation method> mapping indicating the per-hierarchy generation methods that should be used.
- **ensure_connected** (*dict[str, bool], optional*) – A <hierarchy : flag> mapping indicating if the graphs generated per hierarchy should be fully connected or not where relevant.

2.15.22 gatoh.model.ABModel.generate_groups

ABModel.**generate_groups**(*id_base: str*, *cohesion_probs: dict[str, float]*, *n_groups: int = 10*, *max_iters: int = 40*) → None

Randomly generates a number of Group objects from the model's agent and graph sets, using KMeans clustering.

This function requires for both valid agents and graphs to already exist in the model.

Parameters

- **id_base** (*str*) – a 4-character alphabetic string that serves as the base of the XXXXnnnn id for each Group.
- **cohesion_probs** (*dict[str, float]*) – A <cohesion : probability> mapping specifying the probability of a Group having any given cohesion type.
- **n_groups** (*int, optional*) – The number of groups to create in each hierarchy.
- **max_iters** (*int, optional*) – The maximum number of iterations that the KMeans algorithm can run for.

Raises

RuntimeError – If no valid agents or graphs exist in the model.

2.15.23 gatoh.model.ABModel.get_agent_ids

ABModel.**get_agent_ids**() → list[str]

A wrapper for the AgentSet's get_agent_ids function.

Returns
A list containing the unique IDs of every agent in the model's agent set.

Return type
list[str]

2.15.24 gatoh.model.ABModel.get_base_indices_from_edge

ABModel.**get_base_indices_from_edge**(*hierarchy_graph:* Graph, *edge:* GraphEdge) → tuple[int, int]

A helper function for the base graph that takes in a GraphEdge object from a hierarchy graph and transforms the node indices from hierarchy graph indices to the respective index of the Agent objects in the model's AgentSet.

Parameters

- **hierarchy_graph** (Graph) – The corresponding hierarchy graph that the graph edge belongs in.
- **edge** (GraphEdge) – A graph edge from one of the model's hierarchy graphs in the GraphSet.

Returns
The index in the AgentSet of the parent and child nodes involved in the hierarchy graph's relationship.

Return type
tuple[int, int]

2.15.25 gatoh.model.ABModel.get_benefited_count

ABModel.**get_benefited_count**() → int

A wrapper for *benefit_count()*.

Returns
The number of benefited agents in the model's agent set.

Return type
int

2.15.26 gatoh.model.ABModel.get_group_benefited_count

ABModel.**get_group_benefited_count**() → int

A wrapper for *benefit_count()*.

Returns
The number of benefited groups in the model's group set.

Return type
int

2.15.27 gatoh.model.ABModel.get_group_personality_count

ABModel.**get_group_personality_count**(*personality: str*) → int

A wrapper for *personality_count()*.

Parameters
personality (*str*) – The personality type that is being counted.

Raises
ValueError – If the wrapped function receives an invalid personality type.

Returns
The count of groups which hold the specified predominant personality type.

Return type
int

2.15.28 gatoh.model.ABModel.get_group_personality_counts

ABModel.**get_group_personality_counts**() → dict[str, int]

A wrapper for *personality_counts()*.

Returns
The count of each supported personality type among groups in the group set.

Return type
dict[str, int]

2.15.29 gatoh.model.ABModel.get_group_radical_count

ABModel.**get_group_radical_count**() → int

A wrapper for *radicalised_count()*.

Returns
The number of radicalised groups in the model's group set.

Return type
int

2.15.30 gatoh.model.ABModel.get_groupgraph_indices_from_endpoints

ABModel.**get_groupgraph_indices_from_endpoints**(*from_group:* Group, *to_group:* Group) → tuple[int, int]

A helper function for the groupset that takes in two Group objects and retrieves their node indices within the model's groupgraph.

Parameters

- **from_group** (Group) – The group that the edge starts from.
- **to_group** (Group) – The group that the edge points to.

Returns
The index in the groupgraph of the parent and child nodes involved in the edge's relationship.

Return type
tuple[int, int]

2.15.31 gatoh.model.ABModel.get_personality_count

ABModel.**get_personality_count**(*personality: str*) → int

A wrapper for *personality_count()*.

Parameters
personality (*str*) – The personality type that is being counted.

Raises
ValueError – If the wrapped function receives an invalid personality type.

Returns
The count of agents which hold the specified personality type.

Return type
int

2.15.32 gatoh.model.ABModel.get_personality_counts

ABModel.**get_personality_counts**() → dict[str, int]

A wrapper for *personality_counts()*.

Returns
The count of each supported personality type among agents in the agent set.

Return type
dict[str, int]

2.15.33 gatoh.model.ABModel.get_radical_count

ABModel.**get_radical_count**() → int

A wrapper for *radicalised_count()*.

Returns
The number of radicalised agents in the model's agent set.

Return type
int

2.15.34 gatoh.model.ABModel.get_tracked_agent_attributes

ABModel.**get_tracked_agent_attributes**() → dict[str, dict[str, Any]]

A getter function that checks for any individual agents that are being tracked over the model runtime, and then gathers all relevant attributes for those agents.

Returns
A <agent : <attribute : value>> nested mapping containing all relevant attributes for agents which are being tracked.

Return type
dict[str, dict[str, Any]]

2.15.35 gatoh.model.ABModel.get_tracked_parameters

ABModel.**get_tracked_parameters**() → dict[str, Any] | None

A getter function that checks for any explicitly tracked model parameters, and forms a return accordingly.

Returns
A <parameter : value> mapping for this iteration for each tracked model parameter.

Return type
dict[str, Any]

2.15.36 gatoh.model.ABModel.group_iteration_opinion_calculation

ABModel.**group_iteration_opinion_calculation**(*group:* Group) → tuple[str, tuple[float, list[bool]]]

A helper function that calculates the per-group changes to opinions for the iteration, returning all necessary information for `group_iteration_opinion_changes()` to apply the opinion changes.

This function was primarily created to allow for multiprocessing in the main `group_iterate()` function.

Parameters
group (Group) – The group for which the opinion changes are being calculated.

Returns
A <Group ID : Changes info> mapping that provides all necessary information to apply the opinion changes for a specific group.

Return type
tuple[str, tuple[float, list[bool]]]

2.15.37 gatoh.model.ABModel.group_iteration_opinion_changes

ABModel.**group_iteration_opinion_changes**(*changes_dict: dict[str, tuple[float, list[bool]]]*, *radicalisation_thresh: float | None = None*) → None

A helper function for group_iterate that simply applies all group opinion changes and then performs the appropriate checks.

Parameters

- **changes_dict** (*dict[str, tuple[float, list[bool]]]*) – A <group ID : opinion change information> mapping of the opinion values to apply.
- **radicalisation_thresh** (*float, optional*) – An explicit threshold that should be used when determining (de)radicalisation.

Raises
TypeError – If the explicit radicalisation_thresh is invalid.

2.15.38 gatoh.model.ABModel.group_members_opinion_changes

ABModel.**group_members_opinion_changes**(*agent_changes: dict[str, AgentChanges]*) → None

A helper function that applies the group's aggregate opinion changes to the individual Agents which make it up.

Parameters

- **opinion_delta** (*float*) – The calculated per-agent opinion delta that must be applied.
- **radicalisations** (*dict[str, bool]*) – The new radicalisation statuses that have been determined for each member.

2.15.39 gatoh.model.ABModel.init_base_graph

ABModel.**init_base_graph**() → None

Iterates over all the relationships in the existing social hierarchies and creates corresponding edges within the model's base graph.

2.15.40 gatoh.model.ABModel.iterate

ABModel.**iterate**(*worker_pool: Pool | None = None*) → None

A wrapper for the model's main iteration function.

This is made to distinguish the logic of the iteration function as an entry point versus the model simulation itself (in case this is needed in the future…)

Parameters
worker_pool (Pool, optional) – A pool of workers that can distribute the iteration processing amongst themselves.

2.15.41 gatoh.model.ABModel.iteration_opinion_calculation

ABModel.**iteration_opinion_calculation**(*agent:* Agent) → tuple[str, tuple[float, list[float], list[bool]]]

A helper function that calculates the per-agent, per-hierarchy changes to opinions for the iteration, returning all necessary information for `iteration_opinion_changes()` to apply the opinion changes.

This function was primarily created to allow for multiprocessing in the main `iterate()` function.

Parameters
agent (Agent) – The agent for which the opinion changes are being calculated.

Returns
An <Agent ID : Changes info> mapping that provides all necessary information to apply the opinion changes for a specific agent.

Return type
tuple[str, tuple[float, list[float], list[bool]]]

2.15.42 gatoh.model.ABModel.iteration_opinion_changes

ABModel.**iteration_opinion_changes**(*changes_dict: dict[str, tuple[float, list[float], list[bool]]]*) → None

A helper function for iterate that simply applies all agent opinion changes and then checks for radicalisation.

Parameters
changes_dict (*dict[str, tuple[float, list[float], list[bool]]]*) – A <agent ID : opinion change information> mapping of the opinion values to apply.

2.15.43 gatoh.model.ABModel.layers_polarisation_multi

ABModel.**layers_polarisation_multi**(*hierarchy_name: str*) → tuple[str, float]

A helper function that allows for multiprocessing of `calculate_layers_polarisation()`.

Parameters
hierarchy_graph (*str*) – The name of the hierarchy graph for which the polarisation is being calculated.

Returns
The name and polarisation value for the given hierarchy.

Return type
tuple[str, float]

2.15.44 gatoh.model.ABModel.load_model

ABModel.**load_model**(*load_dir: str*) → None

Loads a model and all its components which have been saved following the processes in the save_model() function.

Parameters
load_dir (*str*) – The path to the directory where all model data was saved.

Raises
FileNotFoundError – If the input load_dir is invalid.

2.15.45 gatoh.model.ABModel.logger_debug_iteration

ABModel.**logger_debug_iteration**() → None

A helper function that handles the iteration of the logger's debugging component.

2.15.46 gatoh.model.ABModel.logger_iteration

ABModel.**logger_iteration**(*worker_pool: Pool | None = None*) → None

Calculate any relevant aggregate statistics and then pass these to the logger's iteration() function to be stored.

Statistics calculated currently:

1. Aggregate network opinion
2. Network radicalisation log odds
3. Layer navigability for each hierarchy
4. Layer interdependence for each hierarchy

Parameters
worker_pool (Pool) – A pool of workers that can distribute the processing of the logger iteration between them.

2.15.47 gatoh.model.ABModel.override_current_iteration

ABModel.**override_current_iteration**(*current_iteration: int*) → None

A setter function that overrides the model's current iteration value with a new one.

This function should not be used if a custom iteration function is not in use, as it will cause unexpected simulation behaviours if not handled appropriately.

2.15.48 gatoh.model.ABModel.save_model

ABModel.**save_model**() → None

Saves the model's GraphSet and AgentSet objects, including all objects recursively contained.

2.15.49 gatoh.model.ABModel.set_agent_opinion_rw

ABModel.**set_agent_opinion_rw**(*agent_opinion_rw: tuple[float, float]*) → None

A setter function that changes the model's agent_opinion_rw attribute.

Parameters
agent_opinion_rw (*tuple[float, float]*) – The (mean, variance) parameters used for the agent opinions random-walk effect.

2.15.50 gatoh.model.ABModel.set_checkpointing

ABModel.**set_checkpointing**(*checkpointing: bool*) → None

A setter function that changes the model's checkpointing flag.

Parameters

checkpointing (*bool*) – A flag indicating if the model should save checkpoints at the end of every iteration.

2.15.51 gatoh.model.ABModel.set_data_file

ABModel.**set_data_file**(*data_file: str*) → None

A setter function that changes the path to which the final data file should be written.

Parameters

data_file (*str*) – The path to where the model's final data .csv file should be written.

2.15.52 gatoh.model.ABModel.set_hierarchy_information

ABModel.**set_hierarchy_information**(*hierarchy: str*, *rw_params: tuple[float, float]*) → None

A setter function that adds a new entry into the model's hierarchy information attribute.

This function will always overwrite the parameters for an existing hierarchy in the model.

Parameters

- **hierarchy** (*str*) – The name of the new hierarchy being recorded.
- **rw_params** (*tuple[float, float]*) – The random-walk (mean, variance) parameters for the new hierarchy.

2.15.53 gatoh.model.ABModel.set_max_iterations

ABModel.**set_max_iterations**(*max_iterations: int*) → None

A setter function that changes the model's max iterations attribute.

Parameters

max_iterations (*int*) – The total number of iterations that the model should run for.

Raises

ValueError – If max_iterations is not a valid integer.

2.15.54 gatoh.model.ABModel.set_model_id

ABModel.**set_model_id**(*model_id: str*) → None

A setter function that changes the model's unique ID.

Parameters

model_id (*str*) – The new identifier to assign to the model.

2.15.55 gatoh.model.ABModel.set_negation_threshold

ABModel.**set_negation_threshold**(*negation_thresh: float*) → None

A setter function that changes the model's negation threshold.

Parameters

negation_thresh (*float*) – The threshold that the model should use for the opinion negation effect.

Raises
ValueError – If the negation threshold is outside the range [0.0, 1.0].

2.15.56 gatoh.model.ABModel.set_partial_iterations

ABModel.**set_partial_iterations**(*partial_iterations: bool*) → None

A setter function that changes the model's partial_iterations flag.

Parameters
partial_iterations (*bool*) – A flag indicating if each of the model's iterations should only simulate a random subset of the agents.

2.15.57 gatoh.model.ABModel.set_radicalisation_threshold

ABModel.**set_radicalisation_threshold**(*radical_thresh: float*) → None

A setter function that changes the model's radicalisation threshold.

Parameters
radical_thresh (*float*) – The threshold that the model should use when determining agent (de)radicalisation.

Raises
ValueError – If the radicalisation threshold is outside the range [0.0, 1.0].

2.15.58 gatoh.model.ABModel.set_save_dir

ABModel.**set_save_dir**(*save_dir: str*, *force: bool = False*) → None

A setter function that changes the model's save directory.

Parameters

- **save_dir** (*str*) – The path to the root directory where all model components and subcomponents should be saved.
- **force** (*bool, optional*) – A flag indicating if the function should explicitly create the directory if it does not exist.

Raises
NotADirectoryError – If the input directory is not valid and the operation is not being forced.

2.15.59 gatoh.model.ABModel.set_silencing_threshold

ABModel.**set_silencing_threshold**(*silencing_thresh: float*) → None

A setter function that changes the model's silencing threshold.

Parameters
silencing_thresh (*float*) – The threshold that the model should use for the opinion silencing effect.

Raises
ValueError – If the silencing threshold is outside the range [0.0, 1.0].

2.15.60 gatoh.model.ABModel.set_simulate_groups

ABModel.**set_simulate_groups**(*simulate_groups: bool*) → None

A setter function that changes the model's simulate_groups flag.

Parameters
simulate_groups (*bool*) – A flag indicating if the model will be simulating aggregate agent groups rather than individual agents.

2.15.61 gatoh.model.ABModel.set_suppress_warnings

ABModel.**set_suppress_warnings**(*suppress_warnings: bool*) → None

A setter function that changes the model's suppress warnings flag.

Parameters
suppress_warnings (*bool*) – A flag indicating if non-critical runtime warnings should be suppressed.

2.15.62 gatoh.model.ABModel.set_visualisation_dir

ABModel.**set_visualisation_dir**(*visualisation_dir: str*, *force: bool = False*) → None

A setter function that changes the model's visualisation directory.

Parameters
- **visualisation_dir** (*str*) – The path to the directory where model visualisations should be saved.
- **force** (*bool, optional*) – A flag indicating if the function should explicitly create the directory if it does not exist.

Raises
NotADirectoryError – If the input directory is not valid and the operation is not being forced.

2.15.63 gatoh.model.ABModel.set_visualise

ABModel.**set_visualise**(*visualise: bool*) → None

A setter function that changes the model's visualise flag.

Parameters
visualise (*bool*) – A flag indicating if the model should visualise its runtime and outputs.

2.15.64 gatoh.model.ABModel.step

ABModel.**step**() → None

Steps the model forward one iteration.

This does not handle agent opinion changes, but rather dynamic agent relationships and hierarchy weightings.

2.15.65 gatoh.model.ABModel.track_agent_custom_attribute

ABModel.**track_agent_custom_attribute**(*agent_id: str*, *attribute_name: str*) → None

Used to specify that a custom attribute of a particular agent should be logged and tracked throughout the model runtime.

Parameters
- **agent_id** (*str*) – The unique ID of the agent for which the attribute will be tracked.
- **attribute_name** (*str*) – The name of the custom attribute to be tracked.

Raises
KeyError – If no agent with the specified ID exists in the model's agentset or no valid attribute is found.

2.15.66 gatoh.model.ABModel.track_agent_opinion

ABModel.**track_agent_opinion**(*agent_id: str*) → None

Used to specify that the aggregate opinion at the current iteration of a particular agent should be logged and tracked throughout the model runtime.

Parameters
agent_id (*str*) – The unique ID of the agent for which the opinion will be tracked.

Raises
KeyError – If no agent with the specified ID exists in the model's agentset.

2.15.67 gatoh.model.ABModel.track_agent_previous_opinion

ABModel.**track_agent_previous_opinion**(*agent_id: str*) → None

Used to specify that the aggregate opinion at the previous iteration of a particular agent should be logged and tracked throughout the model runtime.

Parameters
agent_id (*str*) – The unique ID of the agent for which the opinion will be tracked.

Raises
KeyError – If no agent with the specified ID exists in the model's agentset.

2.15.68 gatoh.model.ABModel.track_agent_radicalisation

ABModel.**track_agent_radicalisation**(*agent_id: str*) → None

Used to specify that the radicalisation status of a particular agent should be logged and tracked throughout the model runtime.

Parameters
agent_id (*str*) – The unique ID of the agent for which the radicalisation status will be tracked.

Raises
KeyError – If no agent with the specified ID exists in the model's agentset.

2.15.69 gatoh.model.ABModel.track_agent_silencing

ABModel.**track_agent_silencing**(*agent_id: str*) → None

Used to specify that the silencing status of a particular agent in its hierarchies should be logged and tracked throughout the model runtime.

Parameters
agent_id (*str*) – The unique ID of the agent for which the silencing status will be tracked.

Raises
KeyError – If no agent with the specified ID exists in the model's agentset.

2.15.70 gatoh.model.ABModel.track_agent_social_weightings

ABModel.**track_agent_social_weightings**(*agent_id: str*) → None

Used to specify that the social weightings of a particular agent should be logged and tracked throughout the model runtime.

Parameters
agent_id (*str*) – The unique ID of the agent for which the social weightings will be tracked.

Raises
KeyError – If no agent with the specified ID exists in the model's agentset.

2.15.71 gatoh.model.ABModel.track_model_parameters

ABModel.**track_model_parameters**(*parameters: list[str]*) → None

Note specific model parameters that should be tracked by the logger, and included in the logger's output file.

Parameters
parameters (*list[str]*) – The names of the parameters to track (these should be identical to their names within the model object).

2.15.72 gatoh.model.ABModel.update

ABModel.**update**(*worker_pool: Pool | None = None*, *partial_indices: list[int] | None = None*) → None

Updates the agents' internal states to match the model step. This mainly handles the construction of agents' perceived opinion climates within their hierarchies, and the simulation of opinion silencing behaviours depending on these climates.

Parameters

- **worker_pool** (Pool, optional) – A pool of workers that can distribute the processing of the update function amongst themselves.
- **partial_indices** (*list[int], optional*) – A collection of all the agent indices that were partially simulated in the iteration.

2.15.73 gatoh.model.ABModel.update_base_graph

ABModel.**update_base_graph**() → None

Iterates over all the social hierarchy graphs and checks for pending edge changes.

If pending changes exist, use all of the given information to apply the changes appropriately to the relationships present in the model's base graph.

2.15.74 gatoh.model.ABModel.update_group_multi

ABModel.**update_group_multi**(*group:* Group) → GroupUpdateDict

A helper function that allows for multiprocessing of the *update()* function when simulating groups.

Parameters
group (Group) – The group being updated.

Returns
All recalculated group aggregate attributes (reflecting the changes that members may have experienced from their other groups).

Return type
dict[str, Any]

2.15.75 gatoh.model.ABModel.update_multi

ABModel.**update_multi**(*agent:* Agent) → tuple[dict[str, bool], bool, bool]

A helper function that allows for multiprocessing of the *update()* function.

Parameters
agent (Agent) – The agent being updated.

Returns
The new is_silenced flags for the agent, and flags indicating if opinion silencing and negation ocurred this iteration.

Return type
tuple[dict[str, bool], bool, bool]

gatoh.visualisation.ABVisualiser(...[, ...])	The visualisations will include:

2.16 ABVisualiser

class ABVisualiser(*visualisation_dir: str*, *aggregation_method: str = 'median'*, *save_visualisations: bool = True*)

Bases: `object`

The visualisations will include:

- Graph structure plotting
- Realtime model runtime display
- Post-runtime summarisation graphs
- Per-agent lifetime information
- Per-graph lifetime information

Parameters

- **visualisation_dir** (*str*) – A path to a subdirectory to which all relevant visualisation outputs should be saved to.
- **aggregation_method** (*str, optional*) – The method to use for aggregating parameters (i.e. "mean", "median", etc.).
- **save_visualisations** (*bool, optional*) – A flag indicating if the visualisation outputs should be saved.

Methods

graph_node_opinion	A helper function that takes in a GraphNode object (node payloads in the Graphs), and reports the string conversion of the agent's opinion in the GraphNode.
graph_node_values	A helper function that takes in a GraphNode object (node payloads in the Graphs), and reports the plotting information for the agent in the GraphNode.
visualise_hierarchy	A standalone function to visualise the graph for a specific social hierarchy within the ABModel.
visualise_hierarchy_mpl	**DEPRECATED** This function was originally used when `mpl_draw()` was the graph plotting function in use.
visualiser_iteration	Visualise the radicalisation of agents across the entire agent population; called from the parent model's iterate function at every model iteration.

continues on next page

Table 32 – continued from previous page

visualiser_iteration_mpl	**DEPRECATED** This function was originally used when `mpl_draw()` was the graph plotting function in use.

2.16.1 gatoh.visualisation.ABVisualiser.graph_node_opinion

`ABVisualiser.`**`graph_node_opinion`**(*graph_node:* GraphNode) → str

A helper function that takes in a GraphNode object (node payloads in the Graphs), and reports the string conversion of the agent's opinion in the GraphNode.

Parameters
graph_node (GraphNode) – The graph node for which the agent opinion is being reported.

Returns
A text representation of the node's agent opinion.

Return type
str

2.16.2 gatoh.visualisation.ABVisualiser.graph_node_values

`ABVisualiser.`**`graph_node_values`**(*graph_node:* GraphNode) → tuple[str, bool, float]

A helper function that takes in a GraphNode object (node payloads in the Graphs), and reports the plotting information for the agent in the GraphNode.

Parameters
graph_node (GraphNode) – The graph node for which the agent opinion is being reported.

Raises
ValueError – If any of the agent values are of the incorrect type for plotting.

Returns
The radicalisation status of the agent and the float value of the node's agent opinion.

Return type
tuple[bool, float]

2.16.3 gatoh.visualisation.ABVisualiser.visualise_hierarchy

`ABVisualiser.`**`visualise_hierarchy`**(*hierarchy_graph:* Graph, *save_graph: bool = True*, *show_graph: bool = True*) → None

A standalone function to visualise the graph for a specific social hierarchy within the ABModel.

Parameters

- **hierarchy_graph** (Graph) – The hierarchy graph that is being visualised.
- **save_graph** (*bool, optional*) – A flag indicating if the hierarchy visualisation should be saved.
- **show_graph** (*bool, optional*) – A flag indicating if the hierarchy visualisation should be shown in the output terminal.

2.16.4 gatoh.visualisation.ABVisualiser.visualise_hierarchy_mpl

ABVisualiser.**visualise_hierarchy_mpl**(*hierarchy_graph:* Graph, *save_graph: bool = True*, *show_graph: bool = True*) → None

DEPRECATED This function was originally used when `mpl_draw()` was the graph plotting function in use.

The visualisation module was reworked in July 2026 to work on a more flexible, custom visualisation system developed specifically for GATOH.

The direct counterpart of this function that should be used in the new system is `visualise_hierarchy()`.

—

A standalone function to visualise the graph for a specific social hierarchy within the ABModel.

Parameters

- **hierarchy_graph** (Graph) – The hierarchy graph that is being visualised.
- **save_graph** (*bool, optional*) – A flag indicating if the hierarchy visualisation should be saved.
- **show_graph** (*bool, optional*) – A flag indicating if the hierarchy visualisation should be shown in the output terminal.

2.16.5 gatoh.visualisation.ABVisualiser.visualiser_iteration

ABVisualiser.**visualiser_iteration**(*base_graph:* Graph, *current_iteration: int*, *model_name: str | None = None*) → None

Visualise the radicalisation of agents across the entire agent population; called from the parent model's iterate function at every model iteration.

Parameters

- **base_graph** (Graph) – The model's base graph to visualise.
- **current_iteration** (*int*) – The model's current runtime iteration number.
- **model_name** (*str, optional*) – The name or identification assigned to the model that is being visualised.

2.16.6 gatoh.visualisation.ABVisualiser.visualiser_iteration_mpl

ABVisualiser.**visualiser_iteration_mpl**(*base_graph:* Graph, *current_iteration: int*) → None

DEPRECATED This function was originally used when `mpl_draw()` was the graph plotting function in use.

The visualisation module was reworked in July 2026 to work on a more flexible, custom visualisation system developed specifically for GATOH.

The direct counterpart of this function that should be used in the new system is `visualiser_iteration()`.

—

Visualise the radicalisation of agents across the entire agent population; called from the parent model's iterate function at every model iteration.

Parameters

- **base_graph** (Graph) – The model's base graph to visualise.

- **current_iteration** (*int*) – The model's current runtime iteration number.

gatoh.utils.NodeChanges(opinion, ...)	Dataclass that stores all the relevant information needed to update GraphNodes in the model's base graph.
gatoh.utils.EdgeChanges(hierarchy, weighting)	Dataclass that stores all the relevant information needed to update GraphEdges in the model's base graph.
gatoh.utils.pygraph_to_pydigraph(input_graph)	Transform an arbitrary `rustworxk.PyGraph` to a `rustworkx.PyDiGraph` object where each unidirectional edge in the `PyGraph` becomes two opposing monodirectional edges with weight equal to the original.
gatoh.utils.watts_strogatz_graph(n, k, p, seed)	Returns an undirected Watts-Strogatz small-world graph generated using `rustworkx`.
gatoh.utils.watts_strogatz_creator(n, k, p, ...)	A helper function that performs the actual Graph object creation for the multidispatched functions above.
gatoh.utils.connected_watts_strogatz_graph(n k, p)	Returns a connected, directed Watts-Strogatz small-world graph.

2.17 NodeChanges

class NodeChanges(*opinion: float*, *previous_opinion: float*, *radicalised: bool*, *social_weightings: dict[str, float]*)

Bases: `object`

Dataclass that stores all the relevant information needed to update GraphNodes in the model's base graph.

Parameters

- **opinion** (*float*) – The newest opinion value that the agent in the node should have.
- **previous_opinion** (*float*) – The newest previous_opinion value that the agent in the node should have.
- **radicalised** (*bool*) – The newest radicalisation status that the agent in the node should have.
- **social_weightings** (*dict[str, float]*) – The newest social weightings that the agent in the node should have.

Attributes

opinion: float

previous_opinion: float

radicalised: bool

social_weightings: dict[str, float]

2.18 EdgeChanges

class EdgeChanges(*hierarchy: str*, *weighting: float*)

Bases: `object`

Dataclass that stores all the relevant information needed to update GraphEdges in the model's base graph.

Parameters

- **hierarchy** (*str*) – The name of the hierarchy to which the relationships belongs.
- **weighting** (*float*) – The newest weighting value that the relationship should have.

Attributes

hierarchy: **str**

weighting: **float**

2.19 gatoh.utils.pygraph_to_pydigraph

pygraph_to_pydigraph(*input_graph: PyGraph*) → PyDiGraph

Transform an arbitrary rustworxk.PyGraph to a rustworkx.PyDiGraph object where each unidirectional edge in the PyGraph becomes two opposing monodirectional edges with weight equal to the original. Mainly used exclusively to transform undirected *watts_strogatz_graph()* returns to directed ones for use in the model.

Parameters
input_graph (rustworkx.PyGraph) – The undirected input graph to be transformed into a directed version.

Returns
A directed version of the input graph where each edge has been transformed into two edges pointing in opposing directions.

Return type
rustworkx.PyDiGraph

2.20 gatoh.utils.watts_strogatz_graph

watts_strogatz_graph(*n: 'int'*, *k: 'int'*, *p: 'float'*, *seed: 'int'*) → 'rx.PyGraph'

Returns an undirected Watts-Strogatz small-world graph generated using rustworkx. An adapted version of watts_strogatz_graph() from the NetworkX library.

Parameters

- **n** (*int*) – The number of nodes in the graph.
- **k** (*int*) – The number of nearest neighbours that each node is joined to initially.
- **p** (*float*) – The probability of rewiring each edge of the original ring lattice.
- **seed** (*int*) – The random seed to use for random generation.

Returns
An undirected Watts-Strogatz small-world graph that has been generated.

Return type
rustworkx.PyGraph

watts_strogatz_graph(n: ‘int’, k: ‘int’, p: ‘float’, seed: ‘np.random.RandomState’) -> ‘rx.PyGraph’

Returns an undirected Watts-Strogatz small-world graph generated using rustworkx. An adapted version of watts_strogatz_graph() from the NetworkX library.

Parameters

- **n** (*int*) – The number of nodes in the graph.

- **k** (*int*) – The number of nearest neighbours that each node is joined to initially.
- **p** (*float*) – The probability of rewiring each edge of the original ring lattice.
- **seed** (RandomState) – The random seed to use for random generation.

Returns
The created graph.

Return type
rustworkx.PyGraph

watts_strogatz_graph(n: 'int', k: 'int', p: 'float') -> 'rx.PyGraph'

Returns an undirected Watts-Strogatz small-world graph generated using rustworkx. An adapted version of watts_strogatz_graph() from the NetworkX library.

Parameters

- **n** (*int*) – The number of nodes in the graph.
- **k** (*int*) – The number of nearest neighbours that each node is joined to initially.
- **p** (*float*) – The probability of rewiring each edge of the original ring lattice.

Returns
The created graph.

Return type
rustworkx.PyGraph

2.21 gatoh.utils.watts_strogatz_creator

watts_strogatz_creator(*n: int*, *k: int*, *p: float*, *random_gen: Random | RandomState*) → PyGraph

A helper function that performs the actual Graph object creation for the multidispatched functions above.

Parameters

- **n** (*int*) – The number of nodes in the graph.
- **k** (*int*) – The number of nearest neighbours that each node is joined to initially.
- **p** (*float*) – The probability of rewiring each edge of the original ring lattice.
- **random_gen** (random.Random | numpy.random.RandomState) – The random generator to use for Graph creation.

Raises
ValueError – If k is equal to or greater than n.

Returns
The created graph.

Return type
rustworkx.PyGraph

2.22 gatoh.utils.connected_watts_strogatz_graph

connected_watts_strogatz_graph(*n: int*, *k: int*, *p: float*, *tries: int = 100*, *seed: int | RandomState | None = None*) → PyDiGraph

Returns a connected, directed Watts-Strogatz small-world graph. An adapted version of connected_watts_strogatz_graph() from the NetworkX library.

Parameters

- **n** (*int*) – The number of nodes in the graph.
- **k** (*int*) – The number of nearest neighbours that each node is joined to initially.
- **p** (*float*) – The probability of rewiring each edge of the original ring lattice.
- **tries** (*int, optional*) – The number of times to try producing a connected graph after rewiring, before raising an exception.
- **seed** (int | numpy.random.RandomState, optional) – The random seed to use for random generation.

Raises

RuntimeError – If the function reaches the maximum number of attempts without producing a connected graph.

Returns

The created graph, which is assured to be connected.

Return type

rustworkx.PyDiGraph

gatoh.utils.beta_value_attenuation(input_value	Takes an input value and rescales it using a beta distribution.
gatoh.utils.linear_link(input_param, link_val)	Utility function that performs constant linear scaling on an input parameter.
gatoh.utils.multiplicative_link(input_param, ...)	Utility function that performs a multiplicative scaling on an input parameter.

2.23 gatoh.utils.beta_value_attenuation

beta_value_attenuation(*input_value: float*, *a: float = 0.9*, *b: float = 0.9*) → float

Takes an input value and rescales it using a beta distribution. This function is intended to be used exclusively for the attenuation of indirect neighbouring opinions when an agent is estimating an opinion climate in a hierarchy.

Importantly, the output values are not used for constructing the opinion climate, instead they serve as a threshold used to decide if the original opinion values are included or not.

a and b should generally be equal to each other and less than 1.0 to approximate a binomial distribution whose PDF still has non-zero values in the range (0, 1). This is to enable modeling of the tendency for opinions to become polarised over time; with this distribution meaning that agents will place much higher importance on extreme observed opinions and less importance on moderate opinions.

Parameters

- **input_value** (*float*) – The value to be attenuated. Should always be in the range [-1, 1].
- **a** (*float, optional*) – The alpha parameter for the beta distribution.
- **b** (*float, optional*) – The beta parameter for the beta distribution.

Returns

The attenuated input value.

Return type

float

2.24 gatoh.utils.linear_link

linear_link(*input_param: float*, *link_val: float*, *minimum: float = 0.0*, *maximum: float = 1.0*) → float

Utility function that performs constant linear scaling on an input parameter.

This function is primarily intended to be used as a type of 'link' function for ABModel parameters throughout iterations.

Parameters

- **input_param** (*float*) – The input model parameter that is being scaled.
- **link_val** (*float*) – The constant value by which to increase or decrease the input parameter by.
- **minimum** (*float, optional*) – The minimum valid value that the input parameter may take.
- **maximum** (*float, optional*) – The maximum valid value that the input parameter may take.

Returns
The input parameter after the link function has been applied.

Return type
float

2.25 gatoh.utils.multiplicative_link

multiplicative_link(*input_param: float*, *link_val: float*, *minimum: float = 0.0*, *maximum: float = 1.0*) → float

Utility function that performs a multiplicative scaling on an input parameter.

This function is primarily intended to be used as a type of 'link' function for ABModel parameters throughout iterations.

Parameters

- **input_param** (*float*) – The input model parameter that is being scaled.
- **link_val** (*float*) – The factor by which to multiply the input parameter with.
- **minimum** (*float, optional*) – The minimum valid value that the input parameter may take.
- **maximum** (*float, optional*) – The maximum valid value that the input parameter may take.

Returns
The input parameter after the link function has been applied.

Return type
float

gatoh.utils.create_config_file(save_path, ...)	Creates a structured config file from the input config data, and then saves it to the specified path.

2.26 gatoh.utils.create_config_file

create_config_file(*save_path: str*, *config_data: ConfigData*) → None

Creates a structured config file from the input config data, and then saves it to the specified path.

Parameters

- **save_path** (*str*) – The path in which to save the config file.
- **config_data** (*dict*) – A <name : value> mapping specifying the values of specific parameters to be stored in the config file.

gatoh.utils.get_keys_by_value(input_dict, ...)	A utility function that returns all keys in a dictionary which have the specified value.

2.27 gatoh.utils.get_keys_by_value

get_keys_by_value(*input_dict: dict[K, V]*, *value_to_find: V*) → list[K]

A utility function that returns all keys in a dictionary which have the specified value.

Parameters

- **input_dict** (*dict[Any, Any]*) – The dictionary which is being searched.
- **value_to_find** (*Any*) – The value that is being searched for in the dictionary.

Returns

A list of all keys which have the specified value in the dictionary.

Return type

List[Any]

gatoh.utils.draw_random_value(distribution)	Utility function that handles random value generation from multiple distributions in the same function.
gatoh.utils.random_coinflip(return_type)	Simulates a random coinflip, returning the result as either a boolean, an integer, a float, or a string.
gatoh.utils.value_rw_delta(input_value, ...)	Draws a random walk delta from a normal distribution with the specified parameters, and then adds this delta to the input value before returning.

2.28 gatoh.utils.draw_random_value

draw_random_value(*distribution: str*, *parameters: dict[str, float] | None = None*) → float

Utility function that handles random value generation from multiple distributions in the same function.

All values generated by this function will be in the range [0, 1], with any necessary scaling ocurring in the calling functions.

Note, "scale" and "loc" parameters should still be included as their default values (0 for loc, and 1 for scale) when calling with parameters to prevent dictionary key errors.

Parameters

- **distribution** (*str*) – The name of the distribution to draw from.

- **parameters** (*dict[str, float]*) – A <parameter, value> mapping that contains any relevant parameters to be specified for a given distribution.

Raises
ValueError – If the input distribution is not valid or unsupported.

Returns
A value drawn from the random distribution.

Return type
float

2.29 gatoh.utils.random_coinflip

random_coinflip(*return_type: str*) → T

Simulates a random coinflip, returning the result as either a boolean, an integer, a float, or a string.

The exact return type strings supported are:

- "bool"
- "int"
- "float"
- "string"

Parameters
return_type (*str*) – The data type of the returned coinflip result.

Returns
The outcome of the random coinflip.

Return type
bool | int | float | str

2.30 gatoh.utils.value_rw_delta

value_rw_delta(*input_value: float*, *mean: float*, *variance: float*) → float

Draws a random walk delta from a normal distribution with the specified parameters, and then adds this delta to the input value before returning.

Parameters

- **input_value** (*float*) – The value from which the random walk begins.
- **mean** (*float*) – The mean of the normal distribution from which the delta is drawn.
- **variance** (*float*) – The variance of the normal distribution from which the delta is drawn.

Returns
The result of the random walk.

Return type
float

gatoh.utils.plot_graph(x_vals, y_vals[, ...])	A helper function that handles 2D plotting of data point, with possible separation by categories.

2.31 gatoh.utils.plot_graph

plot_graph(*x_vals: dict[str, list[int | float]]*, *y_vals: dict[str, list[int | float]]*, *plot_type: str = 'line'*, *show_fig: bool = False*, *x_label: str | None = None*, *y_label: str | None = None*, *title: str | None = None*, *save_path: str | None = None*, *vertical_x: int | None = None*, *vertical_name: str | None = None*, *horizontal_y: int | None = None*, *horizontal_name: str | None = None*) → None

A helper function that handles 2D plotting of data point, with possible separation by categories.

Parameters

- **x_vals** (*dict[str, list]*) – A <category : values> mapping that defines the values for the x-axis.
- **y_vals** (*dict[str, list]*) – A <category : values> mapping that defines the values for the y-axis.
- **plot_type** (*str, optional*) – The type of graph to plot.
- **show_fig** (*bool, optional*) – A flag indicating if the graph should be displayed in the script output.
- **x_label** (*str, optional*) – The label to give to the x-axis.
- **y_label** (*str, optional*) – The label to give to the y-axis.
- **title** (*str, optional*) – The title to give to the graph.
- **save_path** (*str, optional*) – The path to save the graph image to.
- **vertical_x** (*int, optional*) – An X-axis value specifying where a vertical line should be added to the graph.
- **vertical_name** (*str, optional*) – A label for the graph's vertical line.
- **horizontal_y** (*int, optional*) – A Y-axis value specifying where a horizontal line should be added to the graph.
- **horizontal_name** (*str, optional*) – A label for the graph's horizontal line.

Raises

NotImplementedError – If the input plot_type is invalid or not currently supported.

CHAPTER

THREE

CONTRIBUTING TO GATOH

CHAPTER

FOUR

GATOH LICENSE

```
MIT License

Copyright (c) 2025 Manuel Munizaga Sepúlveda

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

CHAPTER

FIVE

RELEASE NOTES

5.1 2026.09.0

Further integration of the new Groups module with existing code, and rewrites of the existing validation experiments to include groups.

5.1.1 New Features

- New version of the GraphAlgorithms experiment with groups
- New version of the InfluentialAgents experiment with groups
- New version of the OpinionChanges experiment with groups
- New version of the RandomWalk experiment with groups
- New version of the SocialSusceptibility experiment with groups
- Multiple QoL reporting functions for things such as number of radicalised agents in an agentset, etc.
- Integrated the new *Group* class with the `GATOHLogger`
- Fully implemented all baseline iteration features for *Group* (now on par with *Agent*)
- Fully integrated the clustered groups functionality with the CaseStudy experiment script

5.1.2 Fixes

- Fixed the way that (de)radicalisation thresholds were being determined with social susceptibility
- Fixed the weighting that was used for dijkstra path lengths in clustering (abs(edge.weighting) -> 1 - abs(edge.weighting))
- Fixed the way that the logger was tracking radicalised agents (now correctly stores the number of radicalised agents for that iteration)

5.2 2026.08.0

Continued addition of new features – mostly QoL convenience features, but some new core features.

Also, ongoing bugfixes and optimisations where possible.

5.2.1 New Features

- Added the ability to explicitly define if generated graphs should be complete from within *generate_graphs()*
- Started using coverage to track the coverage of tests across the library modules
- Finished the test cases for *Agent* creation and attributes
- Added input parameter data type checks to various methods in *Agent*
- Finished the test cases for :class:'~gatoh.agents.Agent' populated and empty objects
- Ability to have partial agent iterations in the *ABModel*
- Additional getter methods defined for *AgentSet*
- Additional getter methods defined for *GraphSet*
- Ability to log and track specific attributes of individual agents throughout model runtimes
- **MAJOR** New module groups which adds the ability to further group agents within hierarchies, and pseudo-vectorise operations
- Addition of KMeans clustering algorithm to graphs
- Test cases for Group attributes and creation

5.2.2 Fixes

- Fixed the test cases in test_model_multiprocessed.py and test_model_simulation.py (All 151 existing test cases up to 01/08/2026 are working correctly)
- Fixed a specific *add_edges()* case in which edges were not being added to the graph
- Fixed the build parameters for the docs (will now always add new methods when building)
- Fixed the *calculate_density()* to account for the bidirectional nature of relationships
- Tidying up of multiple typechecks

5.3 2026.07.0

GATOH changes from a semantic to a calendar versioning scheme with this release for simplicity of version control.

This version involves a large number of logistic or 'housekeeping' changes in preparation for an official release as a PyPI package.

5.3.1 New Features

- Changed from semantic to calendar versioning
- Changed the layout of the package modules to simplify import statements
- Extended full docstrings to case study experiment code
- Implemented the *calculate_navigability()* function
- Implemented an initial prototype for *life_events()*
- Implemented a function to calculate graph density metrics with *calculate_density()*
- Implemented functionality to automatically remove any nodes in graphs without neighbours after relevant edges are removed
- Added functions that allow for the spontaneous formation and disintegration of relationships in graphs

- Implemented a mechanism that allows for thresholded agent deradicalisation
- Now accounting for the idea that like-minded agents will push each other towards more extreme opinions even if the average opinion is currently stable (*iteration_opinion_calculation()*)
- Added new checks to *agent_opinion_change()* that account for agent deradicalisation
- Added multiple QoL setter functions to *ABModel*
- Rudimentary prototype for a debugging system in *GATOHLogger*
- Expanded test coverage to include all the new QoL setter functions in *ABModel*
- QoL method *add_agents_to_hierarchy()*
- Created new tests to cover the various calculation methods in *ABModel*
- QoL method *add_relationships_to_hierarchy()*
- Defined multiple global constants throughout modules to clarify "magic numbers"
- Prototype for ability to add "link functions" to model parameters
- Prototype for the ability to track specific model parameters throughout iterations within the *GATOHLogger*

5.3.2 Fixes

- Fixed all the existing imports across scripts to reflect the module restructuring
- Fixed all the code references in the existing documentation
- Fixed all the save/load paths in the experiment scripts to reflect the new layout
- Fixed the layout of agents in the radicalisation heatmap (from shapes of (10, x) to (x, 10))
- Numerous bugs fixed throughout the existing test cases (errors in the test scripts themselves, not in the core code)

5.3.3 Breaking Changes

- Any pickled GATOH objects from previous versions cannot be used in future versions starting with this one. This is due to the simplification from gatoh.X.X.x to gatoh.X.x imports.

5.4 0.2.0-alpha

This release involves the optimisation of core package functionality to reduce model runtimes and simplify the modelling process.

5.4.1 New Features

- A new way of tracking changes and updating models' base graphs, composed of the following:
  - `NodeChanges`
  - `EdgeChanges`
  - `register_edge_change()`
  - `get_edge_changes()`
  - `pending_edge_changes`
  - `init_base_graph()`
  - `update_base_graph()`

- Multiprocessing added to `iterate()`
- Multiprocessing added to `update()`
- Multiprocessing added to the custom iteration function in the `CaseStudy` experiment
- Multiprocessing added to `calculate_layers_polarisation()`
- Multithreading added to `save_agentset()`
- Multithreading added to `save_graphset()`
- Multiprocessing added to `calculate_interdependence()`
- Creation of a quickstart guide for the documentation
- Multiprocessing and multithreading added to `load_agentset()`
- Multiprocessing and multithreading added to `load_graphset()`
- Tests added for AgentSet objects
- Implemented a custom visualisation system and started using this (previously the one provided by rustworkx was used)

5.4.2 Fixes

- Added edge case handling for division by zero errors in `ABModel`
- Corrected the object that was being passed to `rustworkx.visualization.mpl_draw()` in `visualise_hierarchy()`
- Fixed value setting error caused by mismatching `NDArray` data types in `ResponseParser.py` for the planned case study experiment
- Fixed the accessing of Agent `personal_benefit` attributes in `ResponseParser.custom_iterate()` for the planned case study experiment
- Fixed the radicalisation checks in `iteration_opinion_changes()` to stop repeated deradicalisation of agents
- **MAJOR:** Fixed numerous variable declarations and added manual garbage collection throughout various functions to prevent memory leaks during model runtimes
- Fixed modulo operations in new `ABVisualiser` methods
- Removed most uses of "Any" in the code typing
- Stricter type annotations throughout the code

5.5 0.1.0-alpha

This is the first official release of GATOH.

The main purpose of this release is to start preparing the code and its supplementary materials for publication as a proper package.

5.5.1 New Features

- Start of the creation of documentation material. This includes:
  - index.rst
  - license.rst
  - Full API reference

  - Developer and user manuals
  - Getting Started section
  - CITATION.cff
- Mostly working functionality across all main package modules
- Basic model visualisation functionality
- Support for four commonly used random graph generation algorithms:
  - small-world
  - scale-free
  - random
  - stochastic blockmodel
- Support for random generation of `Agent` objects
- A working data persistence system that allows for loading and saving any models created with the framework
- Design and implementation of five framework validation experiments
- Initial prototype for a future case study experiment
- Implementation of unit tests for all core module functionalities
- Enforcing of consistent type checking across modules
- Full docstring coverage across the existing code
- Changed the way that default names were generated for model config files to ensure functionality across different operating systems
- Initial, basic conversion of some Python modules to Rust code. This will likely be expanded on in the future, with the package adopting a hybrid approach, and resource-intensive modules being ported to and optimised in Rust.
- Random-walk mechanisms used to simulate the dynamic nature of agent relationships in social networks, as well as the variable attitude of agents towards the different social networks.
- Publication of the package code to Zenodo

5.5.2 Fixes

- Fixed the way that `radicalisation()` was determining agent radicalisation for "rational" personality types. Now, the function should properly use the "aggregate benefit" of neighbours rather than the agent's personal benefit.
- Multiple restructurings of the ./gatoh file layout; clarifying the separation of package features each time.
- Creation of dedicated setter functions for `Agent` objects to fix the inability of setting `Agent` attributes from `Graph` objects which persist:
  - `store_previous_opinion()`
  - `change_opinion()`
  - `change_radicalisation()`
- Creation of dedicated setter functions for `GraphNode` and `GraphEdge` objects to fix the inability of setting persistent attributes from within `Graph` objects:
  - `set_index()`

  - `set_index()`
  - `set_weighting()`
  - `update_from_node()`
  - `update_to_node()`
- Use of `deepcopy()` across multiple functions to fix data inheritance bugs

BIBLIOGRAPHY

[1] John Conway. Conway's game of life. *Scientific American*, 223:120–123, 1970.

[2] Thomas C. Schelling. Dynamic models of segregation†. *The Journal of Mathematical Sociology*, 1(2):143–186, 1971. URL: https://doi.org/10.1080/0022250X.1971.9989794, arXiv:https://doi.org/10.1080/0022250X.1971.9989794, doi:10.1080/0022250X.1971.9989794.